
Software Requirements Specification

for

Fitness and Workout Tracker System

Version 1.0

Prepared by Prateekshaa N

15/10/2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	2
1.5 References.....	2
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	3
2.4 Operating Environment.....	9
2.5 Design and Implementation Constraints	9
2.6 User Documentation	10
2.7 Assumptions and Dependencies	10
3. External Interface Requirements	10
3.1 User Interfaces	10
3.2 Hardware Interfaces	11
3.3 Software Interfaces	11
3.4 Communications Interfaces	11
4. System Features	11
4.1 System Feature 1	11
4.2 System Feature 2	12
4.3 System Feature 3	12
4.4 System Feature 4	12
4.5 System Feature 5	12
4.6 System Feature 6	13
5. Other Nonfunctional Requirements.....	14
5.1 Performance Requirements	14
5.2 Safety Requirements	14
5.3 Security Requirements	14
5.4 Software Quality Attributes	15
5.5 Business Rules	15
6. Other Requirements	15
Appendix A: Glossary.....	15
Appendix B: Analysis Models.....	16

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification document describes the functional and non-functional requirements of the Fitness and Workout Tracker System. The purpose of this document is to provide a clear description of the system, its features, user interactions, external interfaces, constraints, and expected behaviour. The document serves as a reference for understanding the requirements of the application during development, testing, and evaluation. The system is designed as a web-based application that enables users to maintain their fitness information, record workout activities, track weight and Body Mass Index (BMI), set fitness goals, and monitor their progress through suitable reports and graphical representations.

1.2 Document Conventions

This document follows a structured Software Requirements Specification format based on the organization of the IEEE SRS reference. The headings and subheadings are numbered hierarchically so that each requirement can be easily identified and referenced. Functional requirements describe the services and operations that the Fitness and Workout Tracker System provides to its users, while non-functional requirements describe characteristics such as performance, security, usability, reliability, and maintainability. The terms “User” and “System” are used throughout the document to represent the person using the application and the software application itself.

1.3 Intended Audience and Reading Suggestions

This document is intended for the project developers, project guide, software testers, evaluators, and end users of the Fitness and Workout Tracker System. Developers can use this document to understand the required functionality and system behaviour, while testers can use it to verify whether the implemented features satisfy the specified requirements. The project guide and evaluator can use the document to understand the overall purpose, scope, design, and functionality of the system. Users can refer to the document to understand the major functions available in the application. Readers are recommended to begin with the Introduction section, followed by the Overall Description and External Interface Requirements, and then proceed to the System Features and Nonfunctional Requirements. The Analysis Models in Appendix B provide the UML diagrams associated with the system.

1.4 Product Scope

The Fitness and Workout Tracker System is a web-based application developed to help users record and monitor their fitness activities in an organized manner. The system provides facilities for user registration and login, profile management, workout recording, workout history viewing, weight tracking, BMI calculation, fitness goal management, and progress monitoring. The system stores the user's fitness-related information so that previously entered records can be viewed and analysed over time. Progress information can be represented through suitable charts to make changes in fitness activities and measurements easier to understand. The system aims to provide a simple and centralized platform through which users can maintain their fitness records and monitor their progress.

1.5 References

- IEEE Std 830-1998, Software Requirements Specification (SRS).
- Pressman, R. S. and Sommerville, I., Software Engineering.
- Python and Django Official Documentation.

The development and documentation of the Fitness and Workout Tracker System are based on the principles of Software Requirements Specification and software engineering. The IEEE Software Requirements Specification structure has been used as the reference for organizing this document. The system is implemented using HTML5, CSS3, JavaScript, Bootstrap 5, and Chart.js for the user interface and visualization, while Python with Flask is used for backend processing. SQLAlchemy is used for database interaction and SQLite is used for storing application data. Documentation and learning resources related to these technologies are also considered during system development.

2. Overall Description

2.1 Product Perspective

The Fitness and Workout Tracker System is a web-based software application that provides a centralized environment for managing personal fitness information. The system consists of a user interface through which users enter and view information, a backend application responsible for processing requests and applying system logic, and a database responsible for storing user and fitness records. Users interact with the application through a web browser. The backend processes registration, authentication, workout records, weight information, goals, BMI calculations, and progress information before storing or retrieving the required data from the database. The system therefore provides an integrated environment for maintaining fitness-related records.

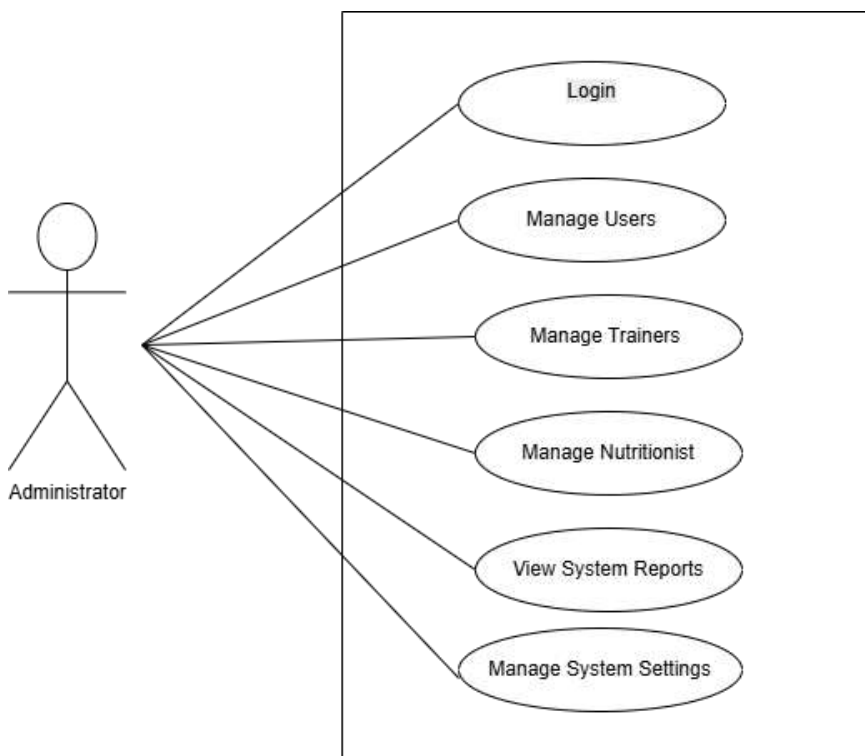
2.2 Product Functions

The system provides several functions required for personal fitness tracking. A new user can register by providing the required personal and login information and can subsequently log in to

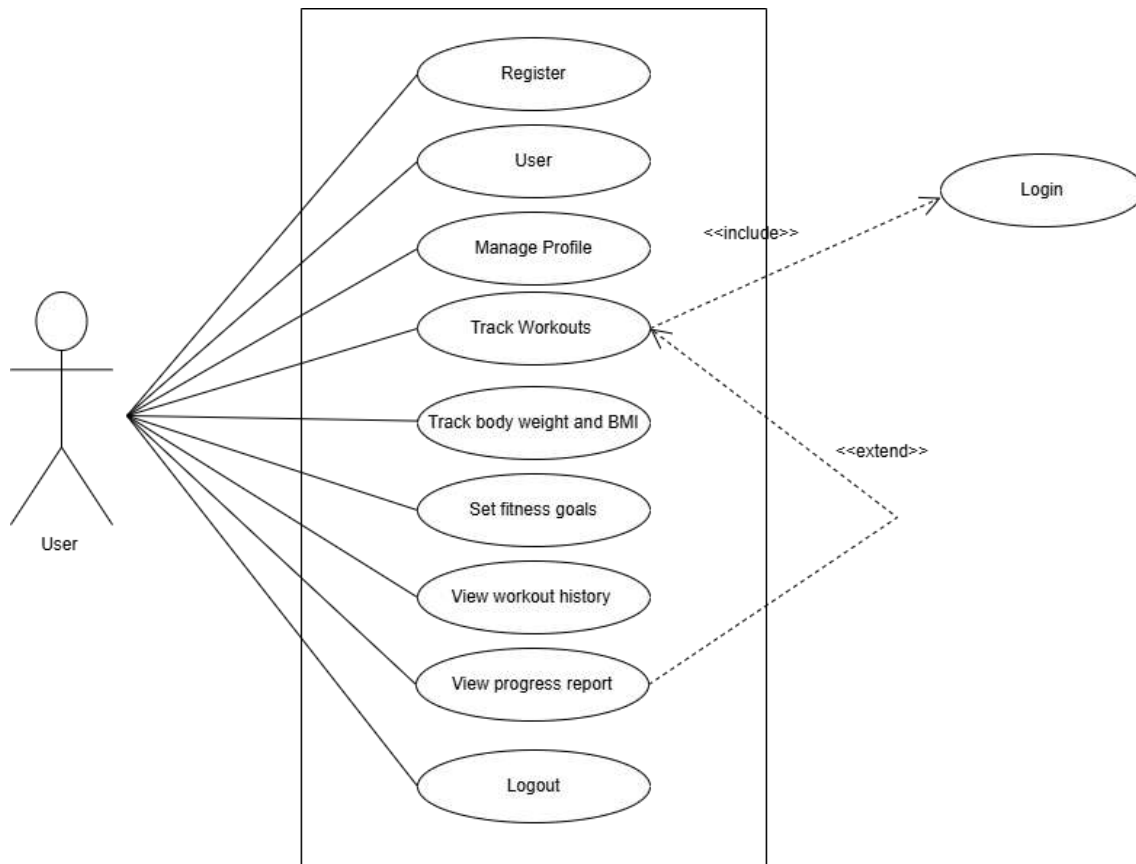
access the application. After authentication, the user can access the dashboard and profile information. The system allows the user to add workout records and view previously recorded workout history. Users can also enter their weight information and use the available information to calculate and monitor BMI. The goal management functionality allows users to create and view personal fitness goals. The progress monitoring functionality retrieves relevant fitness records and presents the user's progress in an understandable form, including graphical representation where applicable. These functions are designed to work together so that users can maintain their fitness information in one application. The Use Case Model represents the major interactions between the user and the Fitness and Workout Tracker System. It provides a high-level view of the services available to the user, including registration, login, profile management, workout tracking, weight and BMI tracking, goal management, and progress monitoring.

2.3 User Classes and Characteristics

The primary user class of the current Fitness and Workout Tracker System is the User. The user is an individual who uses the application to maintain personal fitness records and monitor progress. The user is expected to have basic knowledge of using a web browser, entering information into forms, navigating between web pages, and viewing charts or reports. The user can register an account, authenticate into the system, manage profile information, record workouts, maintain weight information, calculate BMI, create fitness goals, and view progress. The current implementation is focused on personal user-based fitness tracking. Administrative functionality may be considered as a future extension of the system, but it is not treated as an implemented feature in this version of the application.

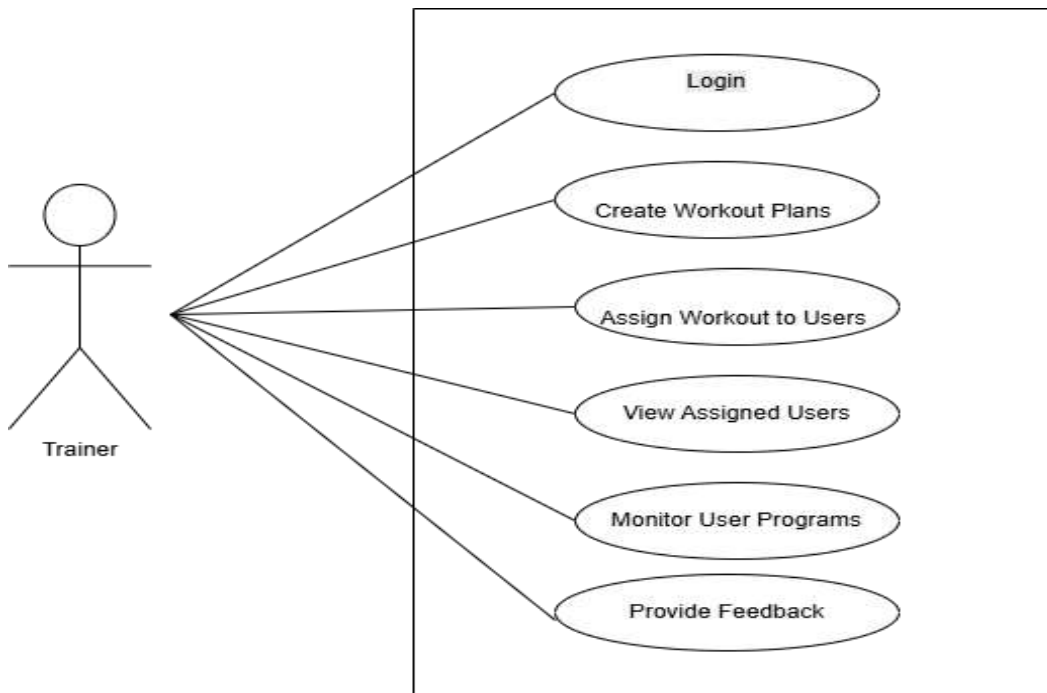


FIELD	DESCRIPTION
Use Case Name	Manage Users
Primary Actor	Administrator
Description	The administrator manages all user accounts in the Fitness and Workout Tracker System. This includes adding new users, viewing user details, updating user information, activating or deactivating accounts, and deleting users when necessary. This ensures secure and efficient management of user data.
Precondition	The administrator must be logged into the system.
Postcondition	User account information is successfully updated, and the changes are stored in the database.
Trigger	The administrator selects the manage users option from the administrator dashboard.
Main Success Scenario	1.The administrator logs into the system. 2.The administrator opens the Manage users module. 3.The system displays the list of registered users. 4.Adds, views, or deletes user information. 5.Saves the changes, validates the information and updates the database
Exceptional Scenario	1.If invalid user information is entered, the system displays an error message and requests valid data. 2.If the selected user account does not exist, the system notifies the administrator. 3.If the database or server is unavailable, the system displays an appropriate error message and the changes are not saved.



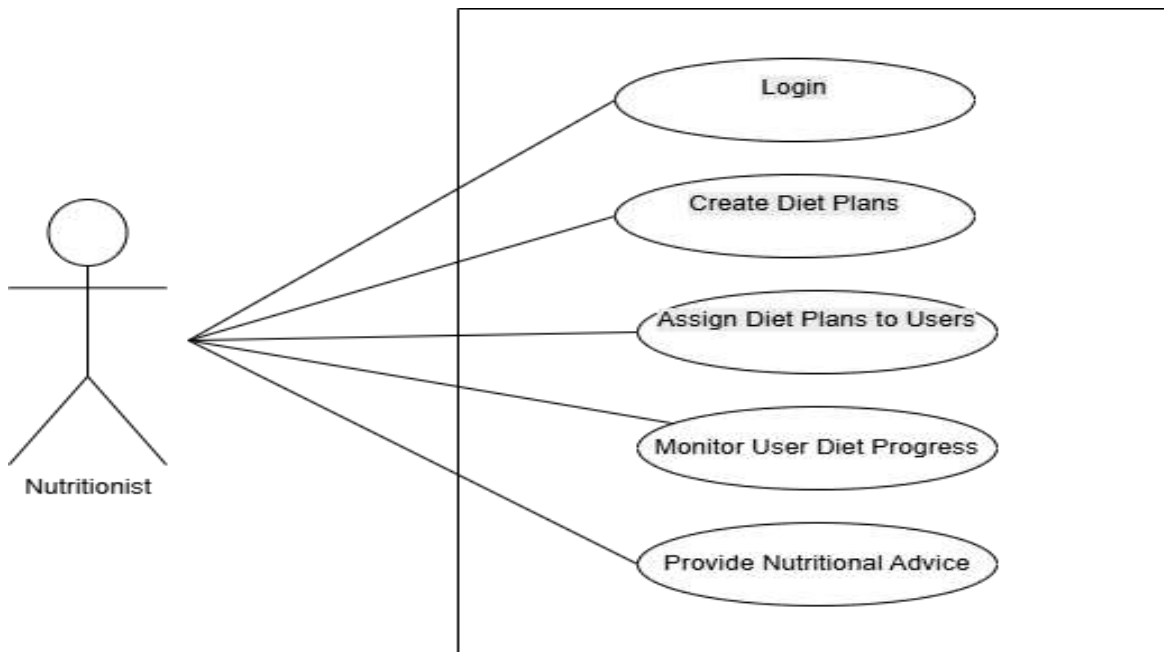
FIELD	DESCRIPTION
Use Case Name	Trace Workouts
Primary Actor	User
Description	The user records daily workout activities such as walking, running, gym exercises, yoga, or cycling. The system stores the workout details and updates the user's fitness progress, allowing them to monitor their performance over time.
Precondition	The user must be logged and resistered into the system.
Postcondition	Workout details are saved successfully, and the users progress is updated
Trigger	The user selects the option to log or record a new workout on the application dashboard.

Main Success Scenario	1. User navigates to the "Log Workout" section. 2. System displays a form asking for workout type, duration, and intensity. 3. User enters the workout details and clicks "Save". 4. System validates the data fields. 5. System stores the workout details in the database. 6. System recalculates and updates the user's overall fitness progress metrics. 7. System displays a success message confirming the workout is saved.
Exceptional Scenario	A. Invalid or Missing Data: 1. At step 4, the system detects required fields (e.g., workout type or duration) are empty or formatted incorrectly. 2. System displays a clear validation error message highlighting the specific fields. 3. System prompts the user to correct the input. B. Database/Network Connection Error: 1. At step 5, the system fails to connect to the database. 2. System displays an error message stating "Failed to save workout. Please try again later." 3. System retains the user's typed input locally so data is not completely lost
Include Relationship .	Login – The user must log in before accessing the Track Workouts functionality
Extend Relationship	View Progress Report – After recording workouts, the user may choose to view their progress report.



FIELD	DESCRIPTION
Use Case Name	Create Workout Plans
Primary Actor	Trainer
Description	The trainer creates personalized workout plans based on the user's fitness level, health condition, and fitness goals. These plans help users follow a structured exercise routine to achieve their desired results.
Precondition	The trainer must be logged into the system.
Postcondition	The workout plan is created successfully and is available for assignment to users.
Trigger	Trainer clicks the "Create New Workout Plan" button.
Main Success Scenario	1. Form: System displays the workout plan builder interface. 2. Input: Trainer inputs exercises, sets, reps, and target goals. 3. Validate: System verifies that all required workout fields are filled. 4. Save: System records the structured plan to the database. 5. Publish: System flags the plan as ready to assign to users.

Exceptional Scenario	1. Empty Fields: System blocks save if no exercises are added. 2. Invalid Data: System rejects negative numbers for sets or reps. 3. Name Conflict: System prevents saving if plan name already exists. 4. Session Timeout: System redirects to login if connection drops. 5. System Error: System prompts user to retry if save fails.
----------------------	---



FIELD	DESCRIPTION
Use Case Name	Create Diet Plans
Primary Actor	Nutritionist
Description	Creates personalized diet plans based on the user's health condition, nutritional requirements, and fitness goals.
Precondition	The nutritionist must be logged into the system.
Postcondition	The diet plan is created successfully and is available for assignment

	to users.
Trigger	Nutritionist clicks the "Create New Diet Plan" button.
Main Success Scenario	1. Form: System displays the diet plan creation form. 2. Input: Nutritionist enters target calories and macronutrients. 3. Validate: System checks if nutritional values meet rules. 4. Save: System stores the completed plan in database. 5. Confirm: System confirms creation and adds to list.
Exceptional Scenario	1. Missing Fields: System blocks save if targets are blank. 2. Invalid Data: System flags negative or unrealistic values. 3. Name Conflict: System rejects plan if name already exists. 4. Timeout: System prompts re-login if session expires. 5. Save Failure: System alerts user if database connection drops.

2.4 Operating Environment

The Fitness and Workout Tracker System operates as a web application and can be accessed using a modern web browser. The frontend uses HTML5 for page structure, CSS3 for styling, JavaScript for client-side interaction, Bootstrap 5 for responsive interface components, and Chart.js for graphical representation of progress information. The backend is implemented using Python and the Flask framework. SQLAlchemy is used as the Object Relational Mapping layer for database operations, while SQLite is used for persistent storage of application data. The application can be developed and executed in a standard desktop or laptop environment capable of running the required Python environment and web browser.

2.5 Design and Implementation Constraints

The system is constrained by the technologies selected for the project. The frontend must use web technologies including HTML5, CSS3, JavaScript, and Bootstrap 5, while Python with Flask is used for backend processing. SQLAlchemy is used for database interaction and SQLite is used as the database. The system must maintain appropriate separation between the user interface, application logic, and data storage. User information and fitness records must be validated before being stored. Authentication must be used to prevent unauthorized access to user-specific information. The application also depends on the availability of the required software environment, Python packages, browser, and database.

2.6 User Documentation

The user documentation for the Fitness and Workout Tracker System explains how users can access and use the major functions of the application. It provides guidance for creating an account, logging in, accessing the dashboard, managing profile information, adding workout records, viewing workout history, entering weight information, calculating BMI, creating fitness goals, and viewing progress. The documentation also explains the general navigation of the application and the purpose of the major screens so that users can operate the system without requiring advanced technical knowledge.

2.7 Assumptions and Dependencies

The system assumes that users have access to a computer or compatible device with a modern web browser and that the application environment is correctly configured. It is assumed that users enter accurate information when recording workouts, weight, height, and fitness goals because the quality of progress information depends on the correctness of the data provided by the user. The system depends on the availability of the Python runtime environment, Flask framework, required Python packages, SQLAlchemy, SQLite database, and browser environment. The application also depends on successful communication between the frontend, backend, and database components.

3. External Interface Requirements

3.1 User Interfaces

The Fitness and Workout Tracker System provides a web-based graphical user interface through which users can interact with the application. The interface includes registration and login pages for authentication, a dashboard for accessing major functions, a profile page for viewing user information, workout pages for recording and viewing activities, weight and BMI pages for maintaining body measurements, goal pages for managing fitness goals, and progress pages for viewing fitness trends. The interface is designed to provide a clear and consistent user experience. Input fields are provided with suitable labels and validation, while error or confirmation messages are displayed when required. Responsive design techniques are used so that the interface can be accessed on different screen sizes. Bootstrap components may be used where appropriate to maintain a consistent and user-friendly layout. Progress information can be represented using charts to make fitness trends easier to understand.

3.2 Hardware Interfaces

The Fitness and Workout Tracker System does not require any specialized hardware devices for its basic operation. The system can be accessed using a standard desktop computer or laptop with sufficient processing capability, memory, display, keyboard, and pointing device. The system interacts with hardware indirectly through the operating system and web browser. No dedicated fitness sensor or external hardware device is required for entering or viewing the fitness information in the current version of the application.

3.3 Software Interfaces

The system interacts with several software components that support its operation. The frontend uses HTML5, CSS3, JavaScript, and Bootstrap 5 to create the user interface and provide client-side interaction. Chart.js is used where graphical visualization of progress information is required. The backend uses Python and Flask to process user requests, perform application logic, handle authentication, and communicate with the database. SQLAlchemy provides the database abstraction layer, while SQLite stores user, workout, weight, goal, and related application information. The application is accessed through a modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

3.4 Communications Interfaces

The Fitness and Workout Tracker System uses web-based communication between the client browser and the Flask backend. The browser sends requests to the application when a user submits a form, selects an operation, or requests information. The Flask backend processes the request and returns the appropriate response or page to the browser. HTTP is used for communication during local development, while secure HTTPS communication can be used when the application is hosted in a production environment. The system does not require specialized communication protocols for its core functionality.

4. System Features

4.1 System Feature 1

4.1.1 Description and Priority

User registration and authentication provide the basic access mechanism for the Fitness and Workout Tracker System. A new user can create an account by entering the required registration information. After successful registration, the user can log in using the registered credentials. Authentication ensures that the user can access personal fitness information through an authorized session.

4.1.2 Stimulus/Response Sequences

When a user opens the registration page and submits valid information, the system validates the entered details and creates the user account. If the information is invalid or incomplete, the system displays an appropriate validation message and requests correction. When a registered user submits valid login credentials, the system authenticates the user and provides access to the application dashboard. If invalid credentials are entered, the system rejects the login attempt and displays an appropriate message.

4.1.3 Functional Requirements

FR-01: The system shall allow a new user to register by providing the required information.

FR-02: The system shall validate the information entered during registration.

FR-03: The system shall store valid registered user information in the database.

FR-04: The system shall allow registered users to log in using valid credentials.

FR-05: The system shall reject invalid login credentials and display an appropriate message.

FR-06: The system shall allow an authenticated user to log out of the application.

4.2 System Feature 2

4.2.1 Description and Priority

The Profile Management feature allows an authenticated user to view personal information associated with the account. The profile provides a centralized view of the user's basic information and supports the management of information required by the fitness tracking system.

4.2.2 Stimulus/Response Sequences

When an authenticated user selects the profile option, the system retrieves the user's stored information from the database and displays it. When profile information is modified, the system validates the entered values and updates the corresponding information in the database.

4.2.3 Functional Requirements

FR-07: The system shall allow an authenticated user to view profile information.
FR-08: The system shall retrieve the user's profile information from the database.
FR-09: The system shall validate profile information before storing changes.
FR-10: The system shall maintain updated profile information for the authenticated user.

4.3 System Feature 3

4.3.1 Description and Priority

The Workout Tracking feature allows users to maintain records of their physical activities. Users can enter workout information and save it for future reference. The system provides a workout history so that users can review their previously recorded activities and monitor their workout patterns over time.

4.3.2 Stimulus/Response Sequences

When a user selects the option to add a workout, the system displays the required workout fields. After the user submits the information, the system validates the input and stores the workout record. When the user requests workout history, the system retrieves the relevant records and displays them in an organized form.

4.3.3 Functional Requirements

FR-11: The system shall allow an authenticated user to add a workout record.
FR-12: The system shall validate the workout information entered by the user.
FR-13: The system shall store valid workout records in the database.
FR-14: The system shall associate workout records with the corresponding user.
FR-15: The system shall allow users to view their previously recorded workouts.

4.4 System Feature 4

4.4.1 Description and Priority

The Weight and BMI Tracking feature enables users to maintain records of their body weight and obtain BMI information based on the required body measurements. The system stores weight records so that users can observe changes over time. BMI information provides an additional numerical measurement that can be monitored along with other fitness records.

4.4.2 Stimulus/Response Sequences

When the user enters the required weight and height information, the system validates the values and performs the BMI calculation using the appropriate formula. The calculated BMI is displayed to the user, and relevant weight information is stored for future tracking. When the user views weight history, the system retrieves previously stored records and presents them in chronological or suitable order.

4.4.3 Functional Requirements

FR-16: The system shall allow an authenticated user to enter weight information.

FR-17: The system shall validate weight and height values before processing them.

FR-18: The system shall calculate BMI using the user's entered measurements.

FR-19: The system shall display the calculated BMI to the user.

FR-20: The system shall store weight records for future reference.

FR-21: The system shall allow users to view their weight history.

4.5 System Feature 5

4.5.1 Description and Priority

The Fitness Goal Management feature allows users to define and maintain personal fitness goals. Goals provide users with a way to identify specific targets and monitor their progress towards those targets. The system stores the goal information and makes it available to the user through the goal management interface.

4.5.2 Stimulus/Response Sequences

When a user selects the goal management option, the system provides the required fields for entering a fitness goal. After submission, the system validates the information and stores the goal. The user can subsequently access the goal section to view previously created goals and monitor their status.

4.5.3 Functional Requirements

FR-22: The system shall allow an authenticated user to create a fitness goal.

FR-23: The system shall validate the information entered for a goal.

FR-24: The system shall store valid goal information in the database.

FR-25: The system shall allow the user to view previously created goals.

FR-26: The system shall associate each goal with the corresponding user.

4.6 System Feature 6

4.6.1 Description and Priority

The Progress Monitoring feature provides a consolidated view of the user's fitness information. The system uses stored workout, weight, BMI, and goal-related information to help the user understand changes over time. Where applicable,

progress information is presented using graphical charts so that trends can be understood more easily than through raw values alone.

4.6.2 Stimulus/Response Sequences

When the user opens the progress section, the system retrieves the relevant fitness records from the database. The retrieved information is processed and presented through suitable visual or textual representations. If sufficient records are available, the system displays progress information through charts or other graphical representations. The user can use the displayed information to review fitness activity over time.

4.6.3 Functional Requirements

FR-27: The system shall retrieve relevant fitness records for the authenticated user.

FR-28: The system shall display the user's fitness progress.

FR-29: The system shall provide graphical representation of suitable progress data.

FR-30: The system shall ensure that progress information displayed to a user belongs to that user's records.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The Fitness and Workout Tracker System should respond to normal user operations within a reasonable period under the expected local development environment. Registration, login, profile access, workout submission, weight entry, goal creation, and retrieval of stored records should be processed without unnecessary delay. Database operations should be designed efficiently so that retrieving a user's fitness history does not result in significant performance degradation as the amount of stored data increases.

5.2 Safety Requirements

The Fitness and Workout Tracker System is intended for recording and monitoring user-provided fitness information and is not intended to replace professional medical advice or diagnosis. The system should clearly present calculated or recorded information as fitness-tracking data. The application should validate user inputs to prevent invalid or unexpected values from causing incorrect system behaviour. The system should also prevent accidental loss of entered information wherever appropriate through suitable validation and confirmation mechanisms.

5.3 Security Requirements

The system shall require user authentication and protect user credentials. Access to personal fitness data shall be restricted to authorized users. User input shall be validated to prevent invalid or harmful data. User sessions shall be securely managed, and logout shall terminate access. User sessions shall be managed appropriately, and logout shall terminate the user's authenticated access.

5.4 Software Quality Attributes

The Fitness and Workout Tracker System should provide usability, reliability, maintainability, portability, and correctness. The user interface should be simple enough for users with basic web application knowledge to understand and navigate. The system should provide consistent results when valid information is entered. The application should be maintainable so that additional fitness features can be incorporated without requiring major changes to the existing structure. The use of Flask, SQLAlchemy, and modular frontend technologies supports separation of responsibilities and improves maintainability. The system should also be portable across common operating systems and modern web browsers that support the technologies used by the application.

5.5 Business Rules

Each user must register before accessing protected features of the application. A user must provide valid authentication information before accessing personal fitness records. Workout, weight, BMI, and goal records must be associated with the appropriate user account. Required input values must be validated before they are stored or processed. BMI should be calculated using the required height and weight values rather than being entered as an arbitrary value when the calculation functionality is used. Progress information should be generated only from records available for the authenticated user.

6. Other Requirements

The system shall maintain its application data in a structured database so that user information and fitness records can be stored and retrieved efficiently. The database shall maintain the relationship between users and their corresponding workout, weight, BMI, goal, and progress information. The application shall perform appropriate validation before storing data in the database. The system should be designed so that future improvements can be incorporated without requiring a complete redesign. Additional fitness tracking features, enhanced reporting, improved visualization, and additional user-management capabilities may be incorporated in future versions. The current version focuses on the core personal fitness tracking requirements defined in this SRS.

Appendix A: Glossary

Term	Description
Fitness Tracker	A software system used to record and monitor fitness-related activities and measurements.
Workout	A physical exercise activity recorded by the user.
BMI	Body Mass Index, a numerical value calculated using weight and height.
Goal	A fitness target defined by the user.
Dashboard	The main application interface from which the user can access major system functions.

Authentication	The process of verifying a user's identity using login credentials.
User	A registered person who uses the Fitness and Workout Tracker System.
Database	A structured storage system used to maintain application and fitness information.
Flask	A Python web framework used to implement the backend of the application.
SQLAlchemy	A Python-based ORM used for interaction with the database.
SQLite	A lightweight relational database used to store application data.
Chart.js	A JavaScript library used to display data through charts and graphical representations.
Bootstrap	A frontend framework used to create responsive and consistent user interfaces.
Progress	The change or development of a user's fitness records over a period of time.

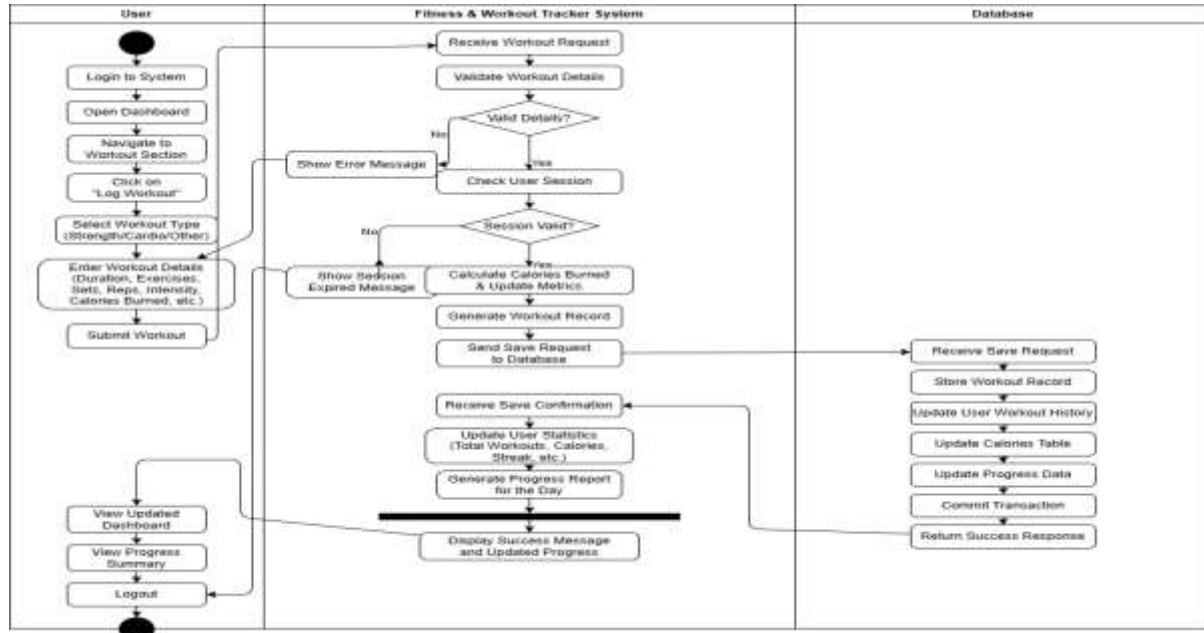
Appendix B: Analysis Models

CLASS DIAGRAM :

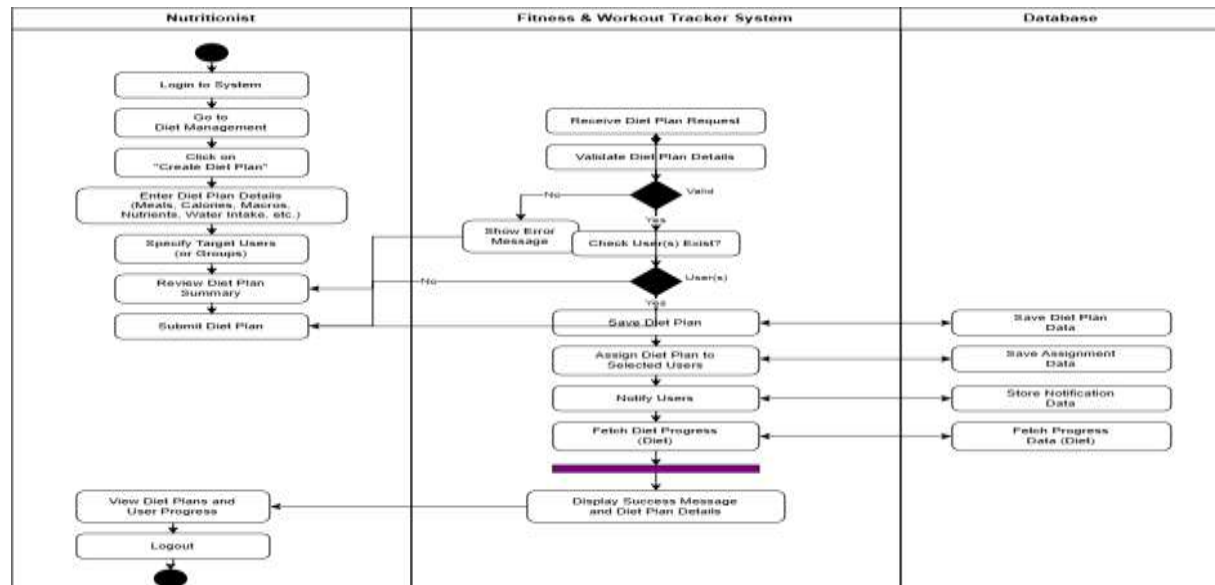


ACTIVITY DIAGRAM :

USER:

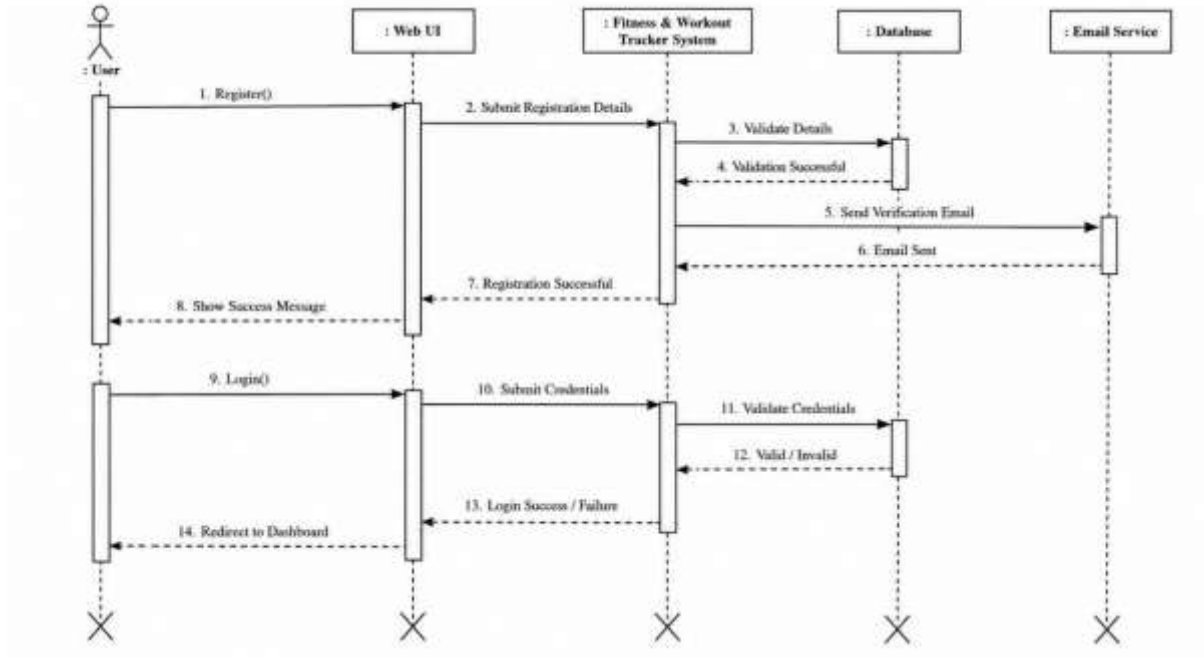


NURTITIONIST :

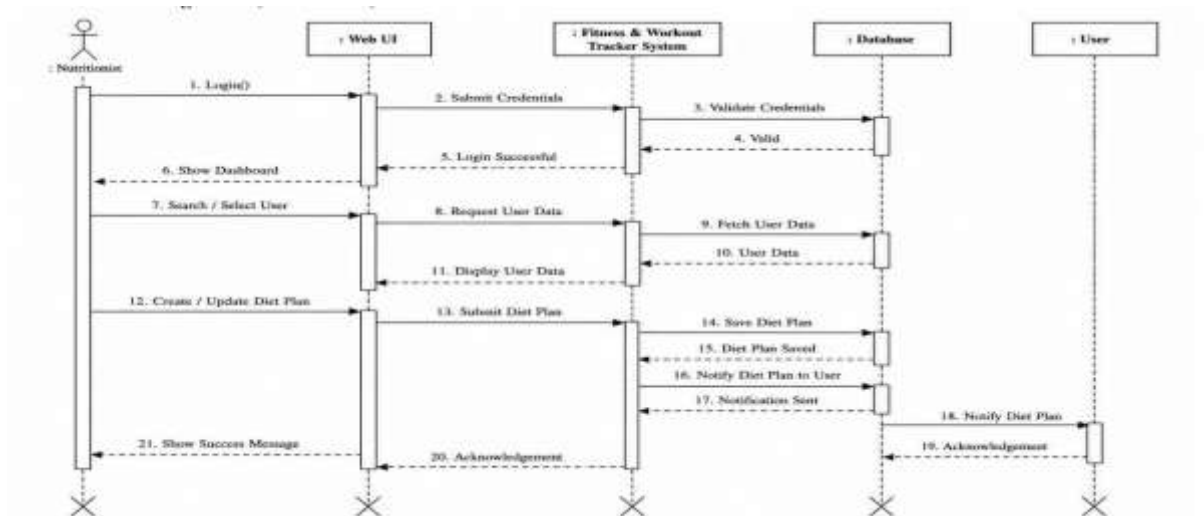


UML SEQUENCE DIAGRAM :

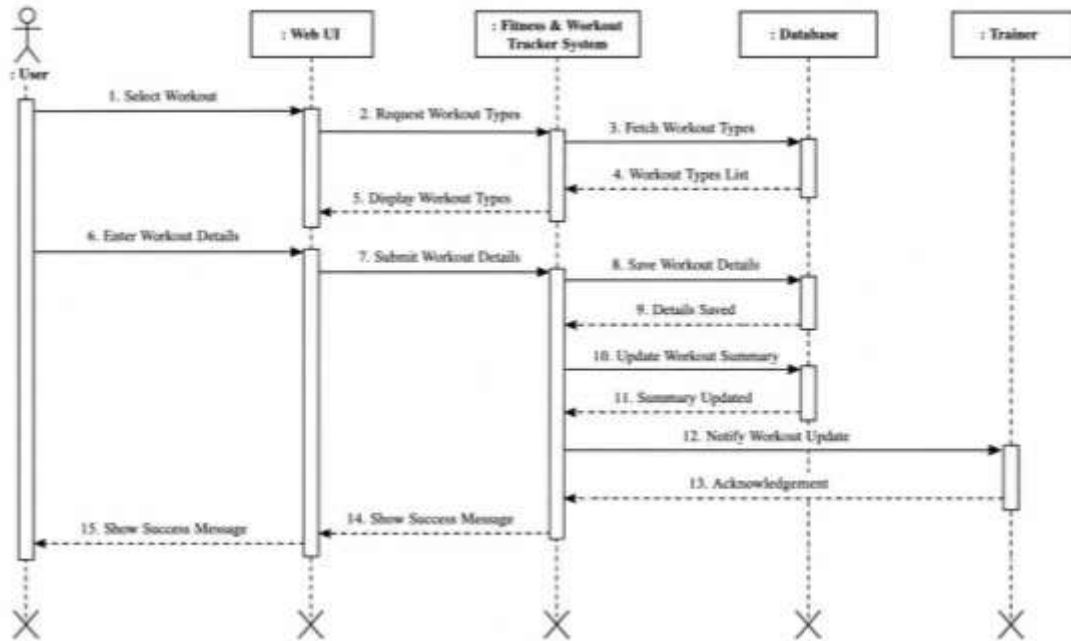
USER REGISTRATION AND LOGIN PAGE:



NUTRITIONIST:

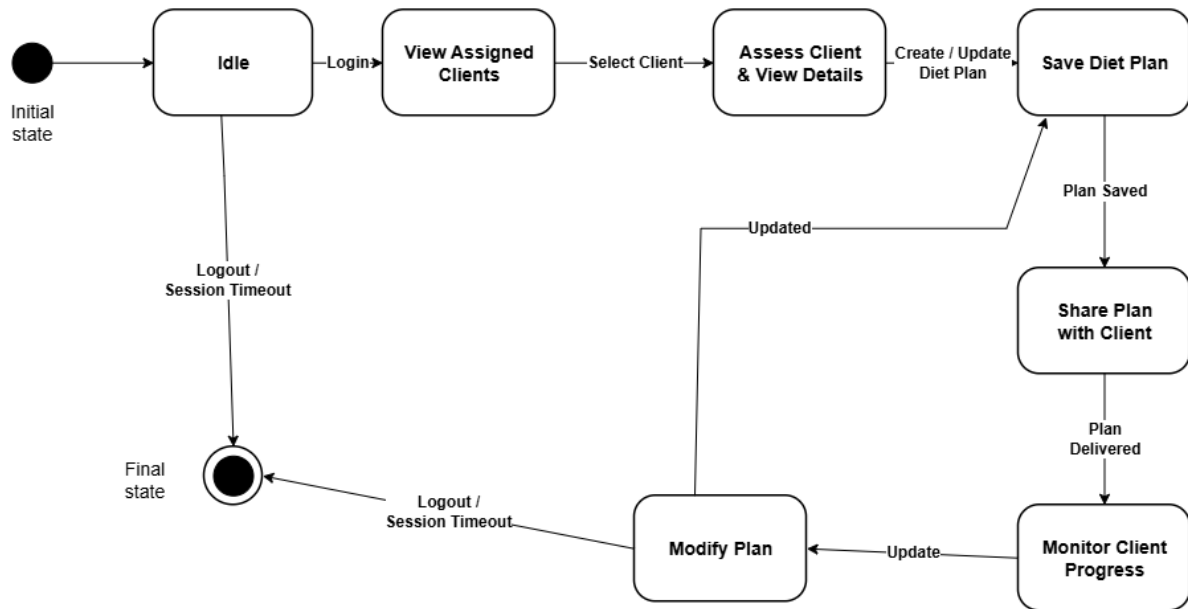


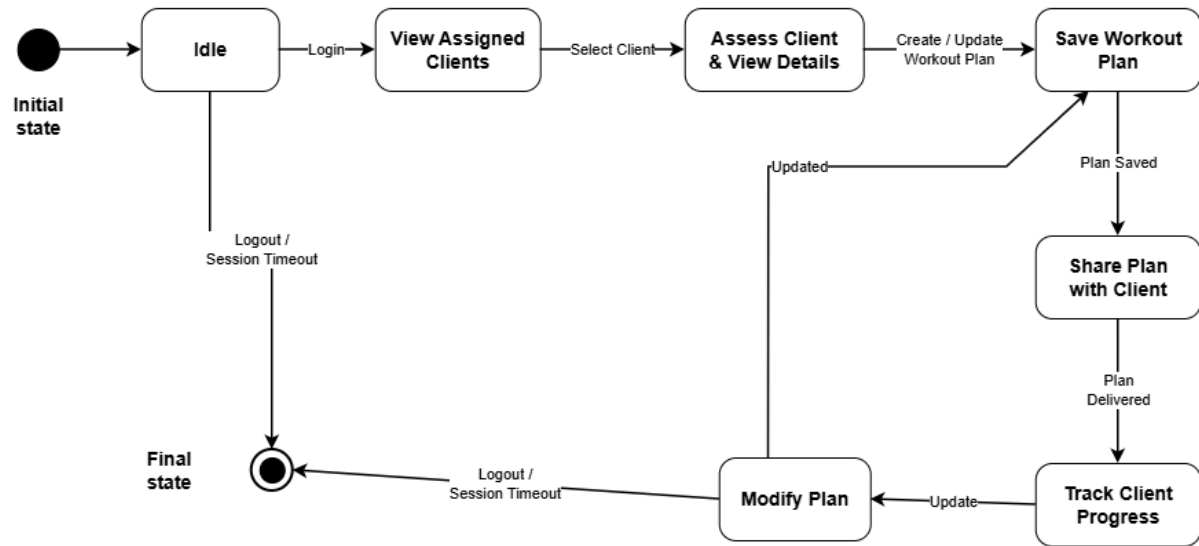
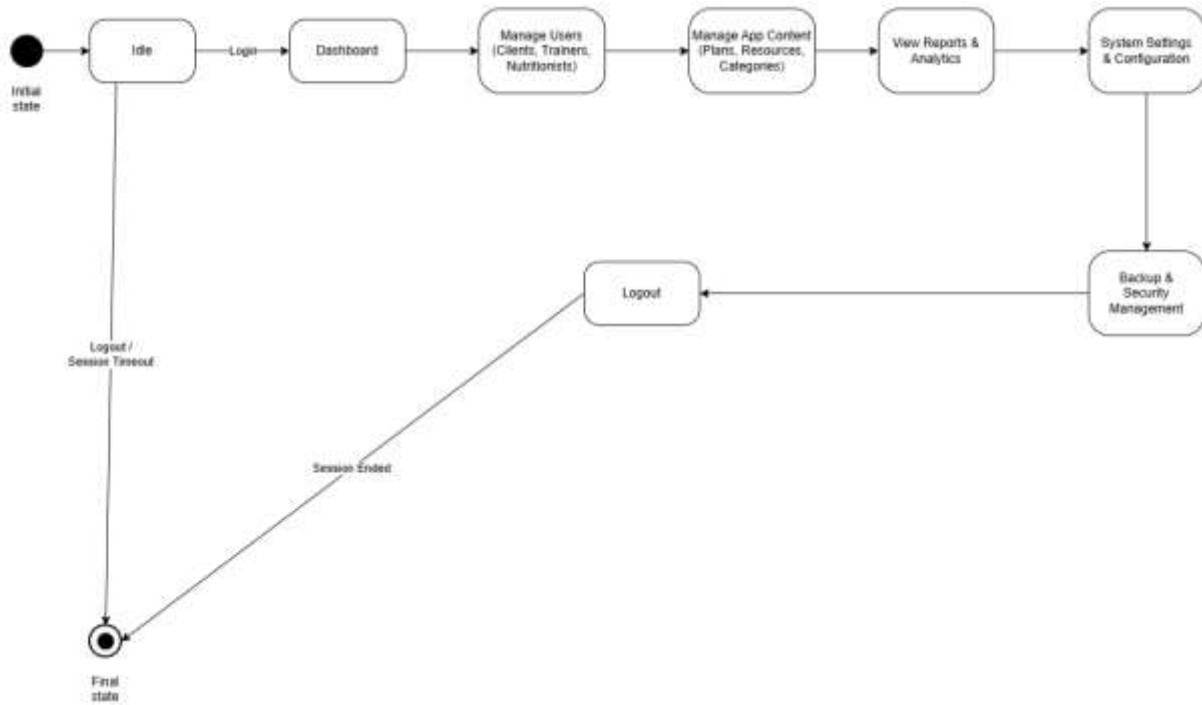
WORKOUT TRACKING USER :



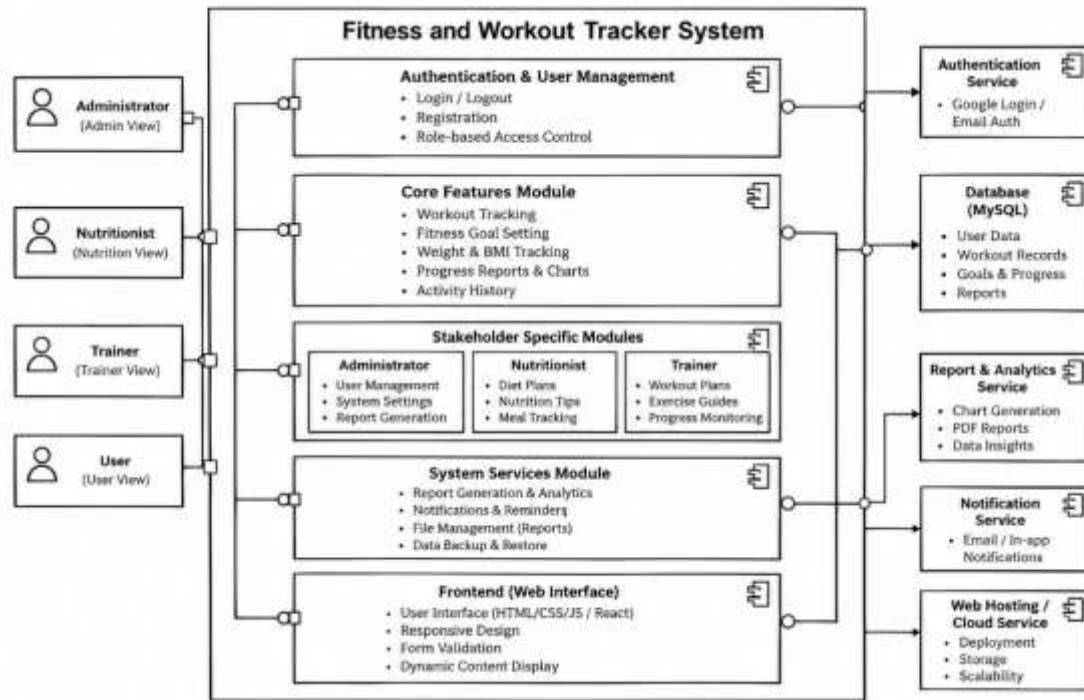
STATE CHART DIAGRAM :

NUTRITIONIST :

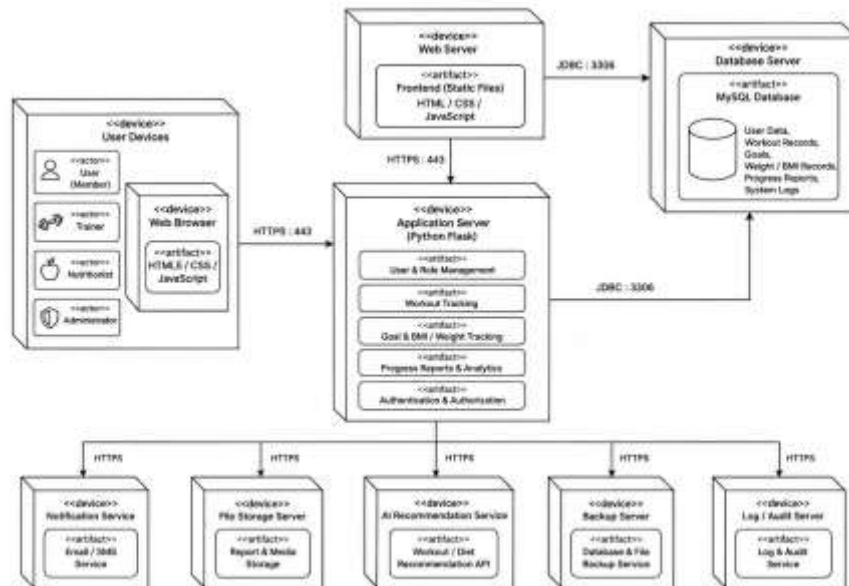


TRAINER:**ADMINISTRATOR :**

COMPONENT DIAGRAM:



DEPLOYMENT DIAGRAM :



SOURCE CODE:**__init__.py:**

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
from .user import User
from .workout import Workout
from .goal import Goal
from .weight import WeightRecord
from .recommendation import Recommendation
__all__ = ['db', 'User', 'Workout', 'Goal', 'WeightRecord',
'Recommendation']
```

Goal.py:

```
from datetime import datetime, date
from . import db
class Goal(db.Model):
    __tablename__ = 'goals'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer,
db.ForeignKey('users.id', ondelete='CASCADE'),
nullable=False, index=True)
    goal_type = db.Column(db.String(50), nullable=False)
    # Lose Weight, Gain Weight, Run Distance, Workout
    Frequency, Improve Strength, Burn Calories
    target_value = db.Column(db.String(100),
nullable=False) # e.g., '70 kg', '10 km', '5 workouts/week'
    current_value = db.Column(db.String(100),
nullable=True) # e.g., '75 kg', '6 km', '3 workouts'
    start_date = db.Column(db.Date, nullable=False,
default=date.today)
    target_date = db.Column(db.Date, nullable=False)
    status = db.Column(db.String(20), nullable=False,
default='ACTIVE') # 'ACTIVE', 'COMPLETED',
'EXPIRED'
    description = db.Column(db.Text, nullable=True)
    created_at = db.Column(db.DateTime,
default=datetime.utcnow, nullable=False)
    def __init__(self, user_id=None, goal_type=None,
target_value=None, current_value=None,
start_date=None, target_date=None, status='ACTIVE',
description=None, **kwargs):
        self.user_id = user_id
        self.goal_type = goal_type
        self.target_value = target_value
        self.current_value = current_value
        self.start_date = start_date
        self.target_date = target_date
        self.status = status
        self.description = description
        for k, v in kwargs.items():
            setattr(self, k, v)
    @property
    def is_expired(self):
        """Check if active goal passed target date."""
        if self.status == 'ACTIVE' and self.target_date <
date.today():
            return True
        return False
    @property
    def badge_color(self):
        if self.status == 'COMPLETED':
            return 'success'
        elif self.is_expired:
            return 'danger'
        return 'primary'
    def to_dict(self):
        return {
```

```
    'id': self.id,
    'user_id': self.user_id,
    'goal_type': self.goal_type,
    'target_value': self.target_value,
    'current_value': self.current_value,
    'start_date': self.start_date.strftime('%Y-%m-%d'),
    'target_date': self.target_date.strftime('%Y-%m-%d'),
    'status': self.status,
    'is_expired': self.is_expired,
    'description': self.description,
    'created_at': self.created_at.strftime('%Y-%m-%d
%H:%M:%S')
    }
    def __repr__(self):
        return f'<Goal {self.goal_type} ({self.status}) for
user {self.user_id}>'
recommendation.py:
from datetime import datetime
from . import db
class Recommendation(db.Model):
    __tablename__ = 'recommendations'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer,
db.ForeignKey('users.id', ondelete='CASCADE'),
nullable=False, index=True)
    author_id = db.Column(db.Integer,
db.ForeignKey('users.id', ondelete='CASCADE'),
nullable=False, index=True)
    note_type = db.Column(db.String(20), nullable=False)
    # 'WORKOUT' or 'NUTRITION'
    title = db.Column(db.String(150), nullable=False)
    content = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime,
default=datetime.utcnow, nullable=False)
    def __init__(self, user_id=None, author_id=None,
note_type=None, title=None, content=None, **kwargs):
        self.user_id = user_id
        self.author_id = author_id
        self.note_type = note_type
        self.title = title
        self.content = content
        for k, v in kwargs.items():
            setattr(self, k, v)
    @property
    def badge_color(self):
        return 'primary' if self.note_type == 'WORKOUT'
    else 'success'
    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'author_id': self.author_id,
            'author_name': self.author.name if self.author else
'Specialist',
            'note_type': self.note_type,
            'title': self.title,
            'content': self.content,
            'created_at': self.created_at.strftime('%Y-%m-%d
%H:%M')
        }
    def __repr__(self):
        return f'<Recommendation {self.note_type} by
{self.author_id} for {self.user_id}>'
user.py:
from datetime import datetime
```

```

from werkzeug.security import generate_password_hash,
check_password_hash
from . import db
class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(255),
    nullable=False)
    role = db.Column(db.String(20), nullable=False,
    default='user') # 'user', 'trainer', 'nutritionist', 'admin'
    age = db.Column(db.Integer, nullable=True)
    gender = db.Column(db.String(20), nullable=True) #
'Male', 'Female', 'Other', 'Prefer not to say'
    height = db.Column(db.Float, nullable=True) # in cm
    weight = db.Column(db.Float, nullable=True) # in kg
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, nullable=False)
    # Relationships
    workouts = db.relationship('Workout', backref='user',
    lazy=True, cascade='all, delete-orphan')
    goals = db.relationship('Goal', backref='user',
    lazy=True, cascade='all, delete-orphan')
    weight_records = db.relationship('WeightRecord',
    backref='user', lazy=True, cascade='all, delete-orphan',
    order_by='WeightRecord.date.desc()')
    recommendations_received = db.relationship(
    'Recommendation',
    foreign_keys='Recommendation.user_id',
    backref='client',
    lazy=True,
    cascade='all, delete-orphan')
    )
    recommendations_given = db.relationship(
    'Recommendation',
    foreign_keys='Recommendation.author_id',
    backref='author',
    lazy=True
    )
    def __init__(self, name=None, email=None, role='user',
    age=None, gender=None, height=None, weight=None,
    password_hash=None, **kwargs):
        self.name = name
        self.email = email
        self.role = role
        self.age = age
        self.gender = gender
        self.height = height
        self.weight = weight
        if password_hash:
            self.password_hash = password_hash
        for k, v in kwargs.items():
            setattr(self, k, v)
    def set_password(self, password):
        """Hash and set the user's password."""
        self.password_hash =
        generate_password_hash(password)
    def check_password(self, password):
        """Verify the user's password."""
        return check_password_hash(self.password_hash,
        password)
    @property
    def bmi(self):
        """Calculate BMI = weight (kg) / (height (m))^2."""
        if self.height and self.weight and self.height > 0 and
        self.weight > 0:
            height_in_meters = self.height / 100.0

```

```

        return round(self.weight / (height_in_meters ** 2),
1)
        return None
    @property
    def bmi_category(self):
        """Determine BMI standard WHO category."""
        bmi_val = self.bmi
        if bmi_val is None:
            return 'Not Recorded'
        if bmi_val < 18.5:
            return 'Underweight'
        elif 18.5 <= bmi_val < 25.0:
            return 'Normal weight'
        elif 25.0 <= bmi_val < 30.0:
            return 'Overweight'
        else:
            return 'Obesity'
    @property
    def bmi_badge_color(self):
        """Bootstrap color indicator for BMI category."""
        cat = self.bmi_category
        if cat == 'Normal weight':
            return 'success'
        elif cat == 'Underweight':
            return 'info'
        elif cat == 'Overweight':
            return 'warning'
        elif cat == 'Obesity':
            return 'danger'
        return 'secondary'
    def to_dict(self):
        return {
            'id': self.id,
            'name': self.name,
            'email': self.email,
            'role': self.role,
            'age': self.age,
            'gender': self.gender,
            'height': self.height,
            'weight': self.weight,
            'bmi': self.bmi,
            'bmi_category': self.bmi_category,
            'created_at': self.created_at.strftime('%Y-%m-%d
%H:%M:%S')
        }
    def __repr__(self):
        return f'<User {self.email} ({self.role})>'
weight.py:
from datetime import datetime, date
from . import db
class WeightRecord(db.Model):
    __tablename__ = 'weight_records'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer,
    db.ForeignKey('users.id', ondelete='CASCADE'),
    nullable=False, index=True)
    weight = db.Column(db.Float, nullable=False) # in kg
    date = db.Column(db.Date, nullable=False,
    default=date.today, index=True)
    notes = db.Column(db.String(255), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, nullable=False)
    def __init__(self, user_id=None, weight=None,
    date=None, notes=None, **kwargs):
        self.user_id = user_id
        self.weight = weight
        self.date = date
        self.notes = notes
        for k, v in kwargs.items():

```

```

        setattr(self, k, v)
    def bmi_for_height(self, height_cm):
        """Calculate BMI given height in cm for this weight record."""
        if height_cm and height_cm > 0 and self.weight > 0:
            height_m = height_cm / 100.0
            return round(self.weight / (height_m ** 2), 1)
        return None
    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'weight': self.weight,
            'date': self.date.strftime('%Y-%m-%d'),
            'notes': self.notes,
            'created_at': self.created_at.strftime('%Y-%m-%d %H:%M:%S')
        }
    def __repr__(self):
        return f'<WeightRecord {self.weight} kg on {self.date} for user {self.user_id}>'
weight.py:
from datetime import datetime, date
from . import db
class Workout(db.Model):
    __tablename__ = 'workouts'
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer,
db.ForeignKey('users.id', ondelete='CASCADE'),
    nullable=False, index=True)
    workout_type = db.Column(db.String(50),
    nullable=False) # e.g., Running, Strength Training,
    Cycling, Yoga, Cardio, Swimming, Walking, HIIT
    exercise_name = db.Column(db.String(100),
    nullable=False)
    duration = db.Column(db.Integer, nullable=False) # in
    minutes
    sets = db.Column(db.Integer, nullable=True, default=0)
    repetitions = db.Column(db.Integer, nullable=True,
    default=0)
    calories_burned = db.Column(db.Float, nullable=False,
    default=0.0)
    workout_date = db.Column(db.Date, nullable=False,
    default=date.today, index=True)
    notes = db.Column(db.Text, nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, nullable=False)
    def __init__(self, user_id=None, workout_type=None,
    exercise_name=None, duration=None, sets=0,
    repetitions=0, calories_burned=0.0, workout_date=None,
    notes=None, **kwargs):
        self.user_id = user_id
        self.workout_type = workout_type
        self.exercise_name = exercise_name
        self.duration = duration
        self.sets = sets
        self.repetitions = repetitions
        self.calories_burned = calories_burned
        self.workout_date = workout_date
        self.notes = notes
        for k, v in kwargs.items():
            setattr(self, k, v)
    @property
    def icon(self):
        """Return a suitable Bootstrap icon name for
        workout type."""
        icons = {
            'Running': 'bi-person-walking',
            'Walking': 'bi-person-walking',

```

```

            'Cycling': 'bi-bicycle',
            'Strength Training': 'bi-lightning-charge-fill',
            'Yoga': 'bi-flower1',
            'Cardio': 'bi-heart-pulse-fill',
            'Swimming': 'bi-water',
            'HIIT': 'bi-fire',
            'Pilates': 'bi-activity'
        }
        return icons.get(self.workout_type, 'bi-activity')
    def to_dict(self):
        return {
            'id': self.id,
            'user_id': self.user_id,
            'workout_type': self.workout_type,
            'exercise_name': self.exercise_name,
            'duration': self.duration,
            'sets': self.sets,
            'repetitions': self.repetitions,
            'calories_burned': self.calories_burned,
            'workout_date': self.workout_date.strftime('%Y-%m-%d'),
            'notes': self.notes,
            'created_at': self.created_at.strftime('%Y-%m-%d %H:%M:%S')
        }
    def __repr__(self):
        return f'<Workout {self.exercise_name} ({self.workout_type}) on {self.workout_date}>'
__init__.py:
from functools import wraps
from flask import session, redirect, url_for, flash
from models import db, User
def get_current_user():
    """Retrieve currently logged in User from DB."""
    user_id = session.get('user_id')
    if user_id:
        return db.session.get(User, user_id)
    return None
def login_required(f):
    """Decorator to enforce authenticated session."""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash('Please log in to access this page.', 'warning')
            return redirect(url_for('auth.login'))
        return f(*args, **kwargs)
    return decorated_function
def role_required(*allowed_roles):
    """Decorator to enforce role-based access control (RBAC)."""
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                flash('Please log in to access this page.',
                'warning')
            return redirect(url_for('auth.login'))

            user_role = session.get('user_role')
            if user_role not in allowed_roles and user_role !=
            'admin':
                flash('You do not have permission to access this
                resource.', 'danger')
                # Redirect to appropriate dashboard based on
                user's actual role
                if user_role == 'admin':
                    return redirect(url_for('admin.dashboard'))
                elif user_role == 'trainer':
                    return redirect(url_for('trainer.dashboard'))

```

```

        elif user_role == 'nutritionist':
            return
        redirect(url_for('nutritionist.dashboard'))
    else:
        return
    redirect(url_for('dashboard.user_dashboard'))
    return f(*args, **kwargs)
    return decorated_function
    return decorator
def get_role_dashboard(role):
    """Return dashboard route name based on role."""
    if role == 'admin':
        return 'admin.dashboard'
    elif role == 'trainer':
        return 'trainer.dashboard'
    elif role == 'nutritionist':
        return 'nutritionist.dashboard'
    return 'dashboard.user_dashboard'
adimin.py:
import re
from flask import Blueprint, render_template, request,
redirect, url_for, flash
from models import db, User, Workout, Goal,
WeightRecord, Recommendation
from . import login_required, role_required,
get_current_user
admin_bp = Blueprint('admin', __name__,
url_prefix='/admin')
EMAIL_REGEX = r'^[a-zA-Z0-9_.-]+@[a-zA-Z0-9-
]+\.[a-zA-Z0-9-]+\.$'
@admin_bp.route('/dashboard')
@login_required
@role_required('admin')
def dashboard():
    """Admin dashboard overview."""
    admin = get_current_user()
    total_users = User.query.count()
    users_by_role = {
        'user': User.query.filter_by(role='user').count(),
        'trainer': User.query.filter_by(role='trainer').count(),
        'nutritionist':
    User.query.filter_by(role='nutritionist').count(),
        'admin': User.query.filter_by(role='admin').count()
    }
    total_workouts = Workout.query.count()
    total_goals = Goal.query.count()
    total_weights = WeightRecord.query.count()
    total_recommendations =
Recommendation.query.count()
    recent_users =
    User.query.order_by(User.created_at.desc()).limit(6).all()
    recent_workouts =
    Workout.query.order_by(Workout.created_at.desc()).limit(
6).all()
    return render_template(
        'admin/dashboard.html',
        admin=admin,
        total_users=total_users,
        users_by_role=users_by_role,
        total_workouts=total_workouts,
        total_goals=total_goals,
        total_weights=total_weights,
        total_recommendations=total_recommendations,
        recent_users=recent_users,
        recent_workouts=recent_workouts
    )
@admin_bp.route('/users')
@login_required
@role_required('admin')

```

```

def list_users():
    """Manage all users."""
    role_filter = request.args.get('role', "").strip()
    search = request.args.get('search', "").strip()
    query = User.query
    if role_filter:
        query = query.filter_by(role=role_filter)
    if search:
        query = query.filter(
            (User.name.ilike(f'%{search}%')) |
            (User.email.ilike(f'%{search}%'))
        )
    users = query.order_by(User.id.asc()).all()
    return render_template('admin/users.html', users=users,
role_filter=role_filter, search=search)
@admin_bp.route('/users/create', methods=['GET',
'POST'])
@login_required
@role_required('admin')
def create_user():
    """Admin create user with any role."""
    if request.method == 'POST':
        name = request.form.get('name', "").strip()
        email = request.form.get('email', "").strip().lower()
        password = request.form.get('password', "")
        role = request.form.get('role', 'user').strip()
        gender = request.form.get('gender', "").strip()
        age = request.form.get('age', "").strip()
        height = request.form.get('height', "").strip()
        weight = request.form.get('weight', "").strip()
        errors = []
        if not name or len(name) < 2:
            errors.append('Name must be at least 2
characters.')
        if not email or not re.match(EMAIL_REGEX,
email):
            errors.append('Invalid email address.')
        elif User.query.filter_by(email=email).first():
            errors.append('Email already in use.')
        if not password or len(password) < 6:
            errors.append('Password must be at least 6
characters.')
        if role not in ['user', 'trainer', 'nutritionist', 'admin']:
            errors.append('Invalid role specified.')
        parsed_age = int(age) if age and age.isdigit() else
None
        parsed_height = float(height) if height and
height.replace('.', "", 1).isdigit() else None
        parsed_weight = float(weight) if weight and
weight.replace('.', "", 1).isdigit() else None
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('admin/user_create.html',
name=name, email=email, role=role)
        new_user = User(
            name=name,
            email=email,
            role=role,
            gender=gender if gender else None,
            age=parsed_age,
            height=parsed_height,
            weight=parsed_weight
        )
        new_user.set_password(password)
        db.session.add(new_user)
        db.session.commit()
        flash(f'User "{name}" ({role}) created successfully!',
'success')

```

```

        return redirect(url_for('admin.list_users'))
    return render_template('admin/user_create.html')
@admin_bp.route('/users/<int:user_id>/edit',
methods=['GET', 'POST'])
@login_required
@role_required('admin')
def edit_user(user_id):
    """Admin edit user details and role."""
    user = db.get_or_404(User, user_id)
    current_admin = get_current_user()
    if request.method == 'POST':
        name = request.form.get('name', "").strip()
        email = request.form.get('email', "").strip().lower()
        role = request.form.get('role', user.role).strip()
        new_password = request.form.get('new_password',
        "").strip()
        gender = request.form.get('gender', "").strip()
        age = request.form.get('age', "").strip()
        height = request.form.get('height', "").strip()
        weight = request.form.get('weight', "").strip()
        errors = []
        if not name or len(name) < 2:
            errors.append('Name must be at least 2
characters.')
        if not email or not re.match(EMAIL_REGEX,
email):
            errors.append('Invalid email address.')

        # Check duplicate email if changed
        existing = User.query.filter_by(email=email).first()
        if existing and existing.id != user.id:
            errors.append('Email is already taken by another
account.')
```

```

        # Protect against removing the last administrator
        if user.id == current_admin.id and role != 'admin':
            admin_count =
User.query.filter_by(role='admin').count()
            if admin_count <= 1:
                errors.append('Cannot demote the only
administrator account.')
```

```

        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('admin/user_edit.html',
user=user)
        user.name = name
        user.email = email
        user.role = role
        user.gender = gender if gender else None
        user.age = int(age) if age and age.isdigit() else None
        user.height = float(height) if height and
height.replace('.', '', 1).isdigit() else None
        user.weight = float(weight) if weight and
weight.replace('.', '', 1).isdigit() else None
        if new_password:
            if len(new_password) < 6:
                flash('Password must be at least 6 characters.',
'warning')
            else:
                user.set_password(new_password)
                db.session.commit()
                flash(f'User "{user.name}" updated successfully!',
'success')
                return redirect(url_for('admin.list_users'))
        return render_template('admin/user_edit.html',
user=user)
@admin_bp.route('/users/<int:user_id>/delete',
methods=['POST'])
@login_required
```

```

@role_required('admin')
def delete_user(user_id):
    """Admin delete user account."""
    user = db.get_or_404(User, user_id)
    current_admin = get_current_user()
    if user.id == current_admin.id:
        flash('You cannot delete your own administrator
account while logged in.', 'danger')
        return redirect(url_for('admin.list_users'))
    user_name = user.name
    db.session.delete(user)
    db.session.commit()
    flash(f'User "{user.name}" and all associated data
deleted successfully.', 'success')
    return redirect(url_for('admin.list_users'))
@admin_bp.route('/workouts')
@login_required
@role_required('admin')
def list_workouts():
    """Admin global workout management."""
    workout_type = request.args.get('type', "").strip()
    query = Workout.query
    if workout_type:
        query =
query.filter_by(workout_type=workout_type)
    workouts =
query.order_by(Workout.workout_date.desc(),
Workout.id.desc()).all()
    return render_template('admin/workouts.html',
workouts=workouts, workout_type=workout_type)
@admin_bp.route('/workouts/<int:workout_id>/delete',
methods=['POST'])
@login_required
@role_required('admin')
def delete_workout(workout_id):
    """Admin delete any workout."""
    workout = db.get_or_404(Workout, workout_id)
    db.session.delete(workout)
    db.session.commit()
    flash('Workout record deleted.', 'success')
    return redirect(url_for('admin.list_workouts'))
@admin_bp.route('/goals')
@login_required
@role_required('admin')
def list_goals():
    """Admin global goals management."""
    goals =
Goal.query.order_by(Goal.created_at.desc()).all()
    return render_template('admin/goals.html', goals=goals)
@admin_bp.route('/goals/<int:goal_id>/delete',
methods=['POST'])
@login_required
@role_required('admin')
def delete_goal(goal_id):
    """Admin delete any goal."""
    goal = db.get_or_404(Goal, goal_id)
    db.session.delete(goal)
    db.session.commit()
    flash('Goal record deleted.', 'success')
    return redirect(url_for('admin.list_goals'))
@admin_bp.route('/weights')
@login_required
@role_required('admin')
def list_weights():
    """Admin global weights management."""
    weights =
WeightRecord.query.order_by(WeightRecord.date.desc(),
WeightRecord.id.desc()).all()
```

```

    return render_template('admin/weights.html',
weights=weights)
@admin_bp.route('/weights/<int:record_id>/delete',
methods=['POST'])
@login_required
@role_required('admin')
def delete_weight(record_id):
    """Admin delete any weight log."""
    weight = db.get_or_404(WeightRecord, record_id)
    db.session.delete(weight)
    db.session.commit()
    flash('Weight record deleted.', 'success')
    return redirect(url_for('admin.list_weights'))
auth.py:
import re
from datetime import date
from flask import Blueprint, render_template, request,
redirect, url_for, flash, session
from models import db, User, WeightRecord
from . import get_role_dashboard
auth_bp = Blueprint('auth', __name__)
EMAIL_REGEX = r'^[a-zA-Z0-9_+]+@[a-zA-Z0-9-
]+\.[a-zA-Z0-9-]+\.$'
@auth_bp.route('/register', methods=['GET', 'POST'])
def register():
    """Register a new user account."""
    if 'user_id' in session:
        return
    redirect(url_for(get_role_dashboard(session.get('user_role'
))))
    if request.method == 'POST':
        name = request.form.get('name', "").strip()
        email = request.form.get('email', "").strip().lower()
        password = request.form.get('password', "")
        confirm_password =
request.form.get('confirm_password', "")
        role = request.form.get('role', 'user').strip()
        gender = request.form.get('gender', "").strip()
        age = request.form.get('age', "").strip()
        height = request.form.get('height', "").strip()
        weight = request.form.get('weight', "").strip()
        # Sanitize role (prevent registering directly as admin
for security)
        if role not in ['user', 'trainer', 'nutritionist']:
            role = 'user'
        errors = []
        # Validations
        if not name or len(name) < 2:
            errors.append('Please provide a valid full name
(minimum 2 characters).')
        if not email or not re.match(EMAIL_REGEX,
email):
            errors.append('Please provide a valid email
address.')
        elif User.query.filter_by(email=email).first():
            errors.append('An account with this email address
already exists.')
        if not password or len(password) < 6:
            errors.append('Password must be at least 6
characters long.')
        elif password != confirm_password:
            errors.append('Passwords do not match.')
        # Numerical validations
        parsed_age = None
        if age:
            try:
                parsed_age = int(age)
                if parsed_age < 5 or parsed_age > 120:

```

```

                errors.append('Age must be between 5 and
120 years.')
            except ValueError:
                errors.append('Age must be a valid integer.')
        parsed_height = None
        if height:
            try:
                parsed_height = float(height)
                if parsed_height < 50 or parsed_height > 260:
                    errors.append('Height must be between 50 cm
and 260 cm.')
            except ValueError:
                errors.append('Height must be a valid number.')
        parsed_weight = None
        if weight:
            try:
                parsed_weight = float(weight)
                if parsed_weight < 20 or parsed_weight > 400:
                    errors.append('Weight must be between 20 kg
and 400 kg.')
            except ValueError:
                errors.append('Weight must be a valid number.')
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template(
                'register.html',
                name=name,
                email=email,
                role=role,
                gender=gender,
                age=age,
                height=height,
                weight=weight
            )
        # Create user
        new_user = User(
            name=name,
            email=email,
            role=role,
            gender=gender if gender else None,
            age=parsed_age,
            height=parsed_height,
            weight=parsed_weight
        )
        new_user.set_password(password)
        db.session.add(new_user)
        db.session.flush() # to obtain new_user.id
        # If weight provided, log initial weight record
        if parsed_weight:
            initial_weight_log = WeightRecord(
                user_id=new_user.id,
                weight=parsed_weight,
                date=date.today(),
                notes='Initial weight during registration'
            )
            db.session.add(initial_weight_log)
            db.session.commit()
            flash('Account registered successfully! You can now
log in.', 'success')
            return redirect(url_for('auth.login'))
            return render_template('register.html')
@auth_bp.route('/login', methods=['GET', 'POST'])
def login():
    """Authenticate user and initialize session."""
    if 'user_id' in session:
        return
    redirect(url_for(get_role_dashboard(session.get('user_role'
))))

```

```

if request.method == 'POST':
    email = request.form.get('email', "").strip().lower()
    password = request.form.get('password', "")
    if not email or not password:
        flash('Please provide both email and password.',
'warning')
    return render_template('login.html', email=email)
    user = User.query.filter_by(email=email).first()
    if not user or not user.check_password(password):
        flash('Invalid email address or password.', 'danger')
    return render_template('login.html', email=email)
# Establish session
session.clear()
session['user_id'] = user.id
session['user_name'] = user.name
session['user_email'] = user.email
session['user_role'] = user.role
flash(f'Welcome back, {user.name}!', 'success')
return
redirect(url_for(get_role_dashboard(user.role)))
return render_template('login.html')
@auth_bp.route('/logout')
def logout():
    """Destroy user session and log out."""
    session.clear()
    flash('You have been logged out successfully.', 'info')
    return redirect(url_for('auth.login'))

bmi.py:
from flask import Blueprint, render_template, request,
jsonify
from models import WeightRecord
from . import get_current_user
bmi_bp = Blueprint('bmi', __name__)
def calculate_bmi_details(weight_kg, height_cm):
    """Compute BMI, category, healthy weight range, and
color."""
    if not weight_kg or not height_cm or weight_kg <= 0 or
height_cm <= 0:
        return None
    height_m = height_cm / 100.0
    bmi = round(weight_kg / (height_m ** 2), 1)
    # Standard WHO categories
    if bmi < 18.5:
        category = 'Underweight'
        color = 'info'
        advice = 'You are below the normal weight range.
Consider consulting a nutritionist for healthy weight-gain
nutrition strategies.'
    elif 18.5 <= bmi < 25.0:
        category = 'Normal weight'
        color = 'success'
        advice = 'Congratulations! Your BMI is within the
healthy range. Maintain your active lifestyle and balanced
diet.'
    elif 25.0 <= bmi < 30.0:
        category = 'Overweight'
        color = 'warning'
        advice = 'You are slightly above the standard healthy
range. Regular cardio and strength workouts alongside
calorie awareness can help achieve optimal balance.'
    else:
        category = 'Obesity'
        color = 'danger'
        advice = 'Your BMI is in the obesity category. It is
recommended to consult a healthcare provider or
nutritionist to design a safe, gradual fitness and meal plan.'
    # Healthy weight range (BMI 18.5 to 24.9)
    min_healthy_weight = round(18.5 * (height_m ** 2), 1)

```

```

max_healthy_weight = round(24.9 * (height_m ** 2),
1)
return {
    'bmi': bmi,
    'category': category,
    'color': color,
    'advice': advice,
    'height_cm': height_cm,
    'weight_kg': weight_kg,
    'min_healthy_weight': min_healthy_weight,
    'max_healthy_weight': max_healthy_weight
}
}
@bmi_bp.route('/bmi')
def view_calculator():
    """Render BMI calculator page with user's saved data if
available."""
    user = get_current_user()
    initial_height = user.height if user and user.height else
170.0
    initial_weight = user.weight if user and user.weight else
70.0
    bmi_details = calculate_bmi_details(initial_weight,
initial_height)
    # User's recent BMI history
    bmi_history = []
    if user and user.height:
        records =
WeightRecord.query.filter_by(user_id=user.id).order_by(
WeightRecord.date.desc()).limit(8).all()
        for r in records:
            bmi_history.append({
                'date': r.date.strftime('%Y-%m-%d'),
                'weight': r.weight,
                'bmi': r.bmi_for_height(user.height)
            })
    return render_template(
        'bmi.html',
        user=user,
        initial_height=initial_height,
        initial_weight=initial_weight,
        bmi_details=bmi_details,
        bmi_history=bmi_history
    )
@bmi_bp.route('/api/bmi/calculate', methods=['POST'])
def api_calculate():
    """JSON API endpoint for BMI calculation."""
    data = request.get_json() or {}
    try:
        height = float(data.get('height', 0))
        weight = float(data.get('weight', 0))
    except (ValueError, TypeError):
        return jsonify({'error': 'Invalid numeric values'}), 400
    details = calculate_bmi_details(weight, height)
    if not details:
        return jsonify({'error': 'Height and weight must be
positive numbers'}), 400
    return jsonify(details)

dashboard.py:
from datetime import timedelta, date
from flask import Blueprint, render_template, session,
redirect, url_for
from models import db, Workout, Goal, WeightRecord,
Recommendation
from . import login_required, role_required,
get_current_user
dashboard_bp = Blueprint('dashboard', __name__)
@dashboard_bp.route('/')
def index():
    """Public home / landing page."""

```

```

if 'user_id' in session:
    role = session.get('user_role')
    if role == 'admin':
        return redirect(url_for('admin.dashboard'))
    elif role == 'trainer':
        return redirect(url_for('trainer.dashboard'))
    elif role == 'nutritionist':
        return redirect(url_for('nutritionist.dashboard'))
    return redirect(url_for('dashboard.user_dashboard'))
return render_template('index.html')
@dashboard_bp.route('/dashboard')
@login_required
@role_required('user')
def user_dashboard():
    """Main dashboard for regular user."""
    user = get_current_user()
    if not user:
        session.clear()
        return redirect(url_for('auth.login'))
    # Metrics calculation
    total_workouts =
    Workout.query.filter_by(user_id=user.id).count()

    # Workouts in the last 7 days
    seven_days_ago = date.today() - timedelta(days=7)
    recent_workouts = Workout.query.filter(
        Workout.user_id == user.id,
        Workout.workout_date >= seven_days_ago
    ).all()

    weekly_workouts_count = len(recent_workouts)
    weekly_calories = sum(w.calories_burned for w in
recent_workouts)
    total_calories =
    db.session.query(db.func.sum(Workout.calories_burned)).
filter_by(user_id=user.id).scalar() or 0.0
    total_minutes =
    db.session.query(db.func.sum(Workout.duration)).filter_b
y(user_id=user.id).scalar() or 0
    # Recent workout logs (top 5)
    latest_workouts =
    Workout.query.filter_by(user_id=user.id).order_by(Work
out.workout_date.desc(), Workout.id.desc()).limit(5).all()
    # Goals summary
    active_goals = Goal.query.filter_by(user_id=user.id,
status='ACTIVE').order_by(Goal.target_date.asc()).all()
    completed_goals_count =
    Goal.query.filter_by(user_id=user.id,
status='COMPLETED').count()
    # Weight history (last 5 records)
    recent_weights =
    WeightRecord.query.filter_by(user_id=user.id).order_by(
WeightRecord.date.desc(),
WeightRecord.id.desc()).limit(5).all()
    # Guidance recommendations received from trainer or
nutritionist
    recommendations =
    Recommendation.query.filter_by(user_id=user.id).order_
by(Recommendation.created_at.desc()).limit(4).all()
    # BMI status
    bmi_val = user.bmi
    bmi_category = user.bmi_category
    bmi_color = user.bmi_badge_color
    return render_template(
        'dashboard.html',
        user=user,
        total_workouts=total_workouts,
        weekly_workouts_count=weekly_workouts_count,
        weekly_calories=round(weekly_calories, 1),

```

```

total_calories=round(total_calories, 1),
total_minutes=total_minutes,
latest_workouts=latest_workouts,
active_goals=active_goals,
completed_goals_count=completed_goals_count,
recent_weights=recent_weights,
recommendations=recommendations,
bmi=bmi_val,
bmi_category=bmi_category,
bmi_color=bmi_color
)
goal.py:
from datetime import datetime, date
from flask import Blueprint, render_template, request,
redirect, url_for, flash
from models import db, Goal
from . import login_required, get_current_user
goal_bp = Blueprint('goal', __name__)
GOAL_TYPES = [
    'Lose Weight',
    'Gain Weight',
    'Run Distance',
    'Workout Frequency',
    'Improve Strength',
    'Burn Calories',
    'Build Muscle',
    'General Fitness'
]
@goal_bp.route('/goals')
@login_required
def view_goals():
    """View user goals with status filtering."""
    user = get_current_user()
    status_filter = request.args.get('status', 'ALL').upper()
    query = Goal.query.filter_by(user_id=user.id)
    if status_filter in ['ACTIVE', 'COMPLETED',
'EXPIRED']:
        query = query.filter_by(status=status_filter)
    goals = query.order_by(Goal.status.asc(),
Goal.target_date.asc()).all()
    total_goals =
    Goal.query.filter_by(user_id=user.id).count()
    active_count = Goal.query.filter_by(user_id=user.id,
status='ACTIVE').count()
    completed_count =
    Goal.query.filter_by(user_id=user.id,
status='COMPLETED').count()
    return render_template(
        'goals/view.html',
        goals=goals,
        current_status=status_filter,
        total_goals=total_goals,
        active_count=active_count,
        completed_count=completed_count
    )
@goal_bp.route('/goals/add', methods=['GET', 'POST'])
@login_required
def add():
    """Add a new fitness goal."""
    user = get_current_user()
    if request.method == 'POST':
        goal_type = request.form.get('goal_type', "").strip()
        target_value = request.form.get('target_value',
        "").strip()
        current_value = request.form.get('current_value',
        "").strip()
        start_date_str = request.form.get('start_date', "").strip()
        target_date_str = request.form.get('target_date',
        "").strip()

```

```

description = request.form.get('description', "").strip()
errors = []
if not goal_type:
    errors.append('Please select a goal type.')
if not target_value:
    errors.append('Please provide a target value (e.g.,
"70 kg" or "5 days/week").')
start_date = date.today()
if start_date_str:
    try:
        start_date = datetime.strptime(start_date_str,
"%Y-%m-%d").date()
    except ValueError:
        errors.append('Invalid start date format.')
if not target_date_str:
    errors.append('Please select a target completion
date.')
else:
    try:
        target_date = datetime.strptime(target_date_str,
"%Y-%m-%d").date()
        if target_date < start_date:
            errors.append('Target date cannot be earlier
than the start date.')
    except ValueError:
        errors.append('Invalid target date format.')
if errors:
    for err in errors:
        flash(err, 'danger')
    return render_template(
        'goals/add.html',
        goal_types=GOAL_TYPES,
        goal_type=goal_type,
        target_value=target_value,
        current_value=current_value,
        start_date=start_date_str,
        target_date=target_date_str,
        description=description
    )
new_goal = Goal(
    user_id=user.id,
    goal_type=goal_type,
    target_value=target_value,
    current_value=current_value if current_value else
None,
    start_date=start_date,
    target_date=target_date,
    status='ACTIVE',
    description=description if description else None
)
db.session.add(new_goal)
db.session.commit()
flash(f'Goal "{goal_type}" added successfully!',
'success')
return redirect(url_for('goal.view_goals'))
return render_template(
    'goals/add.html',
    goal_types=GOAL_TYPES,
    today_date=date.today().strftime("%Y-%m-%d")
)
@ggoal_bp.route('/goals/<int:goal_id>/edit',
methods=['GET', 'POST'])
@login_required
def edit(goal_id):
    """Edit an existing goal."""
    goal = db.get_or_404(Goal, goal_id)
    user = get_current_user()
    if goal.user_id != user.id and user.role != 'admin':

```

```

        flash('You are not authorized to edit this goal.',
'danger')
        return redirect(url_for('goal.view_goals'))
    if request.method == 'POST':
        goal_type = request.form.get('goal_type', "").strip()
        target_value = request.form.get('target_value',
        "").strip()
        current_value = request.form.get('current_value',
        "").strip()
        start_date_str = request.form.get('start_date', "").strip()
        target_date_str = request.form.get('target_date',
        "").strip()
        status = request.form.get('status', 'ACTIVE').strip()
        description = request.form.get('description', "").strip()
        errors = []
        if not goal_type:
            errors.append('Please select a goal type.')
        if not target_value:
            errors.append('Please provide a target value.')
        try:
            start_date = datetime.strptime(start_date_str, "%Y-
            %m-%d").date() if start_date_str else goal.start_date
            target_date = datetime.strptime(target_date_str,
            "%Y-%m-%d").date() if target_date_str else
            goal.target_date
            if target_date < start_date:
                errors.append('Target date cannot be earlier than
start date.')
        except ValueError:
            errors.append('Invalid date format.')
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('goals/edit.html',
            goal=goal, goal_types=GOAL_TYPES)
        goal.goal_type = goal_type
        goal.target_value = target_value
        goal.current_value = current_value if current_value
        else None
        goal.start_date = start_date
        goal.target_date = target_date
        goal.status = status
        goal.description = description if description else
        None
        db.session.commit()
        flash('Goal updated successfully!', 'success')
        return redirect(url_for('goal.view_goals'))
    return render_template('goals/edit.html', goal=goal,
    goal_types=GOAL_TYPES)
@ggoal_bp.route('/goals/<int:goal_id>/status',
methods=['POST'])
@login_required
def toggle_status(goal_id):
    """Toggle goal status (ACTIVE / COMPLETED)."""
    goal = db.get_or_404(Goal, goal_id)
    user = get_current_user()
    if goal.user_id != user.id and user.role != 'admin':
        flash('Unauthorized action.', 'danger')
        return redirect(url_for('goal.view_goals'))
    if goal.status == 'ACTIVE':
        goal.status = 'COMPLETED'
        flash(f'Congratulations! Goal "{goal.goal_type}"
        marked as Completed! 🎉', 'success')
    else:
        goal.status = 'ACTIVE'
        flash(f'Goal "{goal.goal_type}" set to Active.', 'info')
    db.session.commit()
    return redirect(url_for('goal.view_goals'))

```

```

@goal_bp.route('/goals/<int:goal_id>/delete',
methods=['POST'])
@login_required
def delete(goal_id):
    """Delete a goal."""
    goal = db.get_or_404(Goal, goal_id)
    user = get_current_user()
    if goal.user_id != user.id and user.role != 'admin':
        flash('Unauthorized action.', 'danger')
        return redirect(url_for('goal.view_goals'))
    db.session.delete(goal)
    db.session.commit()
    flash('Goal deleted successfully.', 'success')
    return redirect(url_for('goal.view_goals'))

nutritionist.py:
from flask import Blueprint, render_template, request,
redirect, url_for, flash
from models import db, User, Goal, WeightRecord,
Recommendation
from . import login_required, role_required,
get_current_user
nutritionist_bp = Blueprint('nutritionist', __name__,
url_prefix='/nutritionist')
@nutritionist_bp.route('/dashboard')
@login_required
@role_required('nutritionist', 'admin')
def dashboard():
    """Nutritionist dashboard overview."""
    clients =
    User.query.filter_by(role='user').order_by(User.name.asc(
)).all()
    total_clients = len(clients)
    # Calculate BMI distributions
    bmi_dist = {'Underweight': 0, 'Normal weight': 0,
'Overweight': 0, 'Obesity': 0, 'Not Recorded': 0}
    for c in clients:
        bmi_dist[c.bmi_category] =
bmi_dist.get(c.bmi_category, 0) + 1
    nutritionist = get_current_user()
    my_notes =
Recommendation.query.filter_by(author_id=nutritionist.id
,
note_type='NUTRITION').order_by(Recommendation.cre
ated_at.desc()).limit(5).all()
    return render_template(
        'nutritionist/dashboard.html',
        nutritionist=nutritionist,
        clients=clients,
        total_clients=total_clients,
        bmi_dist=bmi_dist,
        my_notes=my_notes
    )
@nutritionist_bp.route('/users')
@login_required
@role_required('nutritionist', 'admin')
def list_users():
    """List all clients with their BMI and weight metrics for
nutritionist review."""
    search = request.args.get('search', "").strip()
    query = User.query.filter_by(role='user')
    if search:
        query = query.filter(
            (User.name.ilike(f'%{search}%')) |
            (User.email.ilike(f'%{search}%'))
        )
    clients = query.order_by(User.name.asc()).all()
    return render_template('nutritionist/users.html',
clients=clients, search=search)
@nutritionist_bp.route('/users/<int:user_id>')

```

```

@login_required
@role_required('nutritionist', 'admin')
def user_detail(user_id):
    """View client's weight logs, BMI trends, dietary goals,
and existing nutrition notes."""
    client = db.get_or_404(User, user_id)
    if client.role != 'user':
        flash('Requested user is not a client.', 'warning')
        return redirect(url_for('nutritionist.list_users'))
    weight_records =
WeightRecord.query.filter_by(user_id=client.id).order_by
(WeightRecord.date.desc()).all()
    goals =
Goal.query.filter_by(user_id=client.id).order_by(Goal.stat
us.asc(), Goal.target_date.asc()).all()
    recommendations =
Recommendation.query.filter_by(user_id=client.id,
note_type='NUTRITION').order_by(Recommendation.cre
ated_at.desc()).all()
    # Calculate initial vs current weight
    current_weight = client.weight
    initial_weight = weight_records[-1].weight if
weight_records else current_weight
    weight_change = round(current_weight - initial_weight,
1) if (current_weight and initial_weight) else 0.0
    return render_template(
        'nutritionist/user_detail.html',
        client=client,
        weight_records=weight_records,
        goals=goals,
        recommendations=recommendations,
        current_weight=current_weight,
        initial_weight=initial_weight,
        weight_change=weight_change
    )
@nutritionist_bp.route('/users/<int:user_id>/recommend',
methods=['POST'])
@login_required
@role_required('nutritionist', 'admin')
def add_recommendation(user_id):
    """Provide a nutrition/diet recommendation for a
client."""
    client = db.get_or_404(User, user_id)
    nutritionist = get_current_user()
    title = request.form.get('title', "").strip()
    content = request.form.get('content', "").strip()
    if not title or not content:
        flash('Title and advice content are required for
nutrition recommendations.', 'danger')
        return redirect(url_for('nutritionist.user_detail',
user_id=user_id))
    new_rec = Recommendation(
        user_id=client.id,
        author_id=nutritionist.id,
        note_type='NUTRITION',
        title=title,
        content=content
    )
    db.session.add(new_rec)
    db.session.commit()
    flash(f'Nutrition guidance sent to {client.name}
successfully!', 'success')
    return redirect(url_for('nutritionist.user_detail',
user_id=user_id))
@nutritionist_bp.route('/recommendations/<int:rec_id>/de
lete', methods=['POST'])
@login_required
@role_required('nutritionist', 'admin')
def delete_recommendation(rec_id):

```

```

"""Delete a nutrition recommendation."""
rec = db.get_or_404(Recommendation, rec_id)
nutritionist = get_current_user()
if rec.author_id != nutritionist.id and nutritionist.role !=
'admin':
    flash("Unauthorized action.", 'danger')
    return redirect(url_for('nutritionist.dashboard'))
user_id = rec.user_id
db.session.delete(rec)
db.session.commit()
flash("Nutrition recommendation removed.", 'success')
return redirect(url_for('nutritionist.user_detail',
user_id=user_id))

```

profile.py:

```

from datetime import date
from flask import Blueprint, render_template, request,
redirect, url_for, flash, session
from models import db, WeightRecord
from . import login_required, get_current_user
profile_bp = Blueprint('profile', __name__)
@profile_bp.route('/profile', methods=['GET', 'POST'])
@login_required
def view_profile():
    """View and update profile information."""
    user = get_current_user()
    if not user:
        return redirect(url_for('auth.login'))
    if request.method == 'POST':
        name = request.form.get('name', "").strip()
        age = request.form.get('age', "").strip()
        gender = request.form.get('gender', "").strip()
        height = request.form.get('height', "").strip()
        weight = request.form.get('weight', "").strip()
        errors = []
        if not name or len(name) < 2:
            errors.append('Name must be at least 2 characters
long.')
        parsed_age = None
        if age:
            try:
                parsed_age = int(age)
                if parsed_age < 5 or parsed_age > 120:
                    errors.append('Age must be between 5 and
120.')
            except ValueError:
                errors.append('Invalid age value.')
        parsed_height = None
        if height:
            try:
                parsed_height = float(height)
                if parsed_height < 50 or parsed_height > 260:
                    errors.append('Height must be between 50 cm
and 260 cm.')
            except ValueError:
                errors.append('Invalid height value.')
        parsed_weight = None
        if weight:
            try:
                parsed_weight = float(weight)
                if parsed_weight < 20 or parsed_weight > 400:
                    errors.append('Weight must be between 20 kg
and 400 kg.')
            except ValueError:
                errors.append('Invalid weight value.')
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('profile.html', user=user)
        user.name = name

```

```

user.age = parsed_age
user.gender = gender if gender else None
user.height = parsed_height
# If weight updated, also log a weight history entry
if parsed_weight is not None and parsed_weight !=
user.weight:
    user.weight = parsed_weight
    weight_entry = WeightRecord(
        user_id=user.id,
        weight=parsed_weight,
        date=date.today(),
        notes='Profile update'
    )
    db.session.add(weight_entry)
    db.session.commit()
    session['user_name'] = user.name
    flash('Profile updated successfully!', 'success')
    return redirect(url_for('profile.view_profile'))
    return render_template('profile.html', user=user)

```

progress.py:

```

from datetime import date, timedelta
from flask import Blueprint, render_template, jsonify,
request
from models import db, Workout, Goal, WeightRecord
from . import login_required, get_current_user
progress_bp = Blueprint('progress', __name__)
@progress_bp.route('/progress')
@login_required
def view_progress():
    """Render progress visualization analytics page."""
    user = get_current_user()
    total_workouts =
Workout.query.filter_by(user_id=user.id).count()
    total_weights =
WeightRecord.query.filter_by(user_id=user.id).count()
    total_goals =
Goal.query.filter_by(user_id=user.id).count()
    return render_template(
        'progress.html',
        user=user,
        total_workouts=total_workouts,
        total_weights=total_weights,
        total_goals=total_goals
    )
@progress_bp.route('/api/progress-data')
@login_required
def get_progress_data():
    """Aggregate real database data for Chart.js
visualization."""
    user = get_current_user()
    days_param = request.args.get('days', '30')
    # Date filter cutoff
    cutoff_date = None
    if days_param != 'all':
        try:
            days_count = int(days_param)
            cutoff_date = date.today() -
timedelta(days=days_count)
        except ValueError:
            cutoff_date = date.today() - timedelta(days=30)
    # 1. Weight history over time (ascending for charts)
    weight_query =
WeightRecord.query.filter_by(user_id=user.id)
    if cutoff_date:
        weight_query =
weight_query.filter(WeightRecord.date >= cutoff_date)
    weight_records =
weight_query.order_by(WeightRecord.date.asc()).all()

```

```

weight_labels = [r.date.strftime('%b %d') for r in
weight_records]
weight_data = [r.weight for r in weight_records]
# 2. Workouts over time (duration and calories)
workout_query =
Workout.query.filter_by(user_id=user.id)
if cutoff_date:
    workout_query =
workout_query.filter(Workout.workout_date >=
cutoff_date)
workouts =
workout_query.order_by(Workout.workout_date.asc()).all()
# Aggregate by date
workout_by_date = {}
for w in workouts:
    d_str = w.workout_date.strftime('%b %d')
    if d_str not in workout_by_date:
        workout_by_date[d_str] = {'duration': 0, 'calories':
0.0, 'count': 0}
    workout_by_date[d_str]['duration'] += w.duration
    workout_by_date[d_str]['calories'] +=
w.calories_burned
    workout_by_date[d_str]['count'] += 1
workout_labels = list(workout_by_date.keys())
workout_durations = [workout_by_date[d]['duration']
for d in workout_labels]
workout_calories = [workout_by_date[d]['calories'] for
d in workout_labels]
workout_counts = [workout_by_date[d]['count'] for d in
workout_labels]
# 3. Workout Breakdown by Type (all time or filtered)
type_counts = db.session.query(
    Workout.workout_type,
    db.func.count(Workout.id),
    db.func.sum(Workout.duration)
).filter(Workout.user_id == user.id)
if cutoff_date:
    type_counts =
type_counts.filter(Workout.workout_date >= cutoff_date)
type_counts =
type_counts.group_by(Workout.workout_type).all()
type_labels = [tc[0] for tc in type_counts]
type_data_counts = [tc[1] for tc in type_counts]
type_data_durations = [tc[2] or 0 for tc in type_counts]
# 4. Goals Status Breakdown
active_goals = Goal.query.filter_by(user_id=user.id,
status='ACTIVE').count()
completed_goals =
Goal.query.filter_by(user_id=user.id,
status='COMPLETED').count()
expired_goals = Goal.query.filter_by(user_id=user.id,
status='EXPIRED').count()
return jsonify({
    'weight': {
        'labels': weight_labels,
        'data': weight_data,
        'has_data': len(weight_data) > 0
    },
    'workouts': {
        'labels': workout_labels,
        'durations': workout_durations,
        'calories': workout_calories,
        'counts': workout_counts,
        'has_data': len(workout_labels) > 0
    },
    'types': {
        'labels': type_labels,
        'counts': type_data_counts,

```

```

'durations': type_data_durations,
'has_data': len(type_labels) > 0
},
'goals': {
    'labels': ['Active', 'Completed', 'Expired'],
    'data': [active_goals, completed_goals,
expired_goals],
    'has_data': (active_goals + completed_goals +
expired_goals) > 0
}
})
trainer.py:
from flask import Blueprint, render_template, request,
redirect, url_for, flash
from models import db, User, Workout, Goal,
WeightRecord, Recommendation
from . import login_required, role_required,
get_current_user
trainer_bp = Blueprint('trainer', __name__,
url_prefix='/trainer')
@trainer_bp.route('/dashboard')
@login_required
@role_required('trainer', 'admin')
def dashboard():
    """Trainer dashboard overview."""
    clients =
User.query.filter_by(role='user').order_by(User.name.asc()
).all()
    total_clients = len(clients)

    # Total workouts logged by all clients
    total_client_workouts =
Workout.query.join(User).filter(User.role ==
'user').count()
    active_client_goals =
Goal.query.join(User).filter(User.role == 'user',
Goal.status == 'ACTIVE').count()

    # Recommendations given by this trainer
    trainer = get_current_user()
    my_notes =
Recommendation.query.filter_by(author_id=trainer.id,
note_type='WORKOUT').order_by(Recommendation.crea
ted_at.desc()).limit(5).all()
    return render_template(
        'trainer/dashboard.html',
        trainer=trainer,
        clients=clients,
        total_clients=total_clients,
        total_client_workouts=total_client_workouts,
        active_client_goals=active_client_goals,
        my_notes=my_notes
    )
@trainer_bp.route('/users')
@login_required
@role_required('trainer', 'admin')
def list_users():
    """List all client users for trainer review."""
    search = request.args.get('search', '').strip()
    query = User.query.filter_by(role='user')
    if search:
        query = query.filter(
            (User.name.ilike(f'%{search}%')) |
            (User.email.ilike(f'%{search}%'))
        )
    clients = query.order_by(User.name.asc()).all()
    return render_template('trainer/users.html',
clients=clients, search=search)
@trainer_bp.route('/users/<int:user_id>')

```

```

@login_required
@role_required('trainer', 'admin')
def user_detail(user_id):
    """View client's fitness details, workouts, goals, and
    existing trainer notes."""
    client = db.get_or_404(User, user_id)
    if client.role != 'user':
        flash('Requested user is not a client.', 'warning')
        return redirect(url_for('trainer.list_users'))
    workouts =
    Workout.query.filter_by(user_id=client.id).order_by(Wor
    kout.workout_date.desc()).all()
    goals =
    Goal.query.filter_by(user_id=client.id).order_by(Goal.stat
    us.asc(), Goal.target_date.asc()).all()
    weight_records =
    WeightRecord.query.filter_by(user_id=client.id).order_by
    (WeightRecord.date.desc()).limit(10).all()
    recommendations =
    Recommendation.query.filter_by(user_id=client.id,
    note_type='WORKOUT').order_by(Recommendation.crea
    ted_at.desc()).all()
    total_calories = sum(w.calories_burned for w in
    workouts)
    total_minutes = sum(w.duration for w in workouts)
    return render_template(
        'trainer/user_detail.html',
        client=client,
        workouts=workouts,
        goals=goals,
        weight_records=weight_records,
        recommendations=recommendations,
        total_calories=round(total_calories, 1),
        total_minutes=total_minutes
    )
@trainer_bp.route('/users/<int:user_id>/recommend',
methods=['POST'])
@login_required
@role_required('trainer', 'admin')
def add_recommendation(user_id):
    """Provide a workout recommendation or guidance
    note for a client."""
    client = db.get_or_404(User, user_id)
    trainer = get_current_user()
    title = request.form.get('title', '').strip()
    content = request.form.get('content', '').strip()
    if not title or not content:
        flash('Title and content are required for workout
        recommendations.', 'danger')
        return redirect(url_for('trainer.user_detail',
        user_id=user_id))
    new_rec = Recommendation(
        user_id=client.id,
        author_id=trainer.id,
        note_type='WORKOUT',
        title=title,
        content=content
    )
    db.session.add(new_rec)
    db.session.commit()
    flash(f'Workout recommendation provided to
    {client.name} successfully!', 'success')
    return redirect(url_for('trainer.user_detail',
    user_id=user_id))
@trainer_bp.route('/recommendations/<int:rec_id>/delete'
, methods=['POST'])
@login_required
@role_required('trainer', 'admin')
def delete_recommendation(rec_id):

```

```

        """Delete a workout recommendation."""
        rec = db.get_or_404(Recommendation, rec_id)
        trainer = get_current_user()
        if rec.author_id != trainer.id and trainer.role != 'admin':
            flash('Unauthorized action.', 'danger')
            return redirect(url_for('trainer.dashboard'))
        user_id = rec.user_id
        db.session.delete(rec)
        db.session.commit()
        flash('Recommendation removed.', 'success')
        return redirect(url_for('trainer.user_detail',
        user_id=user_id))
weight.py:
from datetime import datetime, date
from flask import Blueprint, render_template, request,
redirect, url_for, flash
from models import db, WeightRecord
from . import login_required, get_current_user
weight_bp = Blueprint('weight', __name__)
@weight_bp.route('/weight')
@login_required
def history():
    """View weight history logs with BMI and change
    indications."""
    user = get_current_user()
    records =
    WeightRecord.query.filter_by(user_id=user.id).order_by(
    WeightRecord.date.desc(), WeightRecord.id.desc()).all()
    # Enrich records with BMI and delta calculation
    enriched_records = []
    for i, r in enumerate(records):
        bmi_val = r.bmi_for_height(user.height)
        delta = None
        # delta compared to the chronologically prior entry
        (which is at index i+1 in descending list)
        if i + 1 < len(records):
            delta = round(r.weight - records[i + 1].weight, 1)
        enriched_records.append({
            'record': r,
            'bmi': bmi_val,
            'delta': delta
        })
    # Stats
    current_weight = user.weight
    initial_weight = records[-1].weight if records else
    current_weight
    total_change = round(current_weight - initial_weight,
    1) if (current_weight and initial_weight) else 0.0
    return render_template(
        'weight/history.html',
        records=enriched_records,
        current_weight=current_weight,
        initial_weight=initial_weight,
        total_change=total_change,
        user=user
    )
@weight_bp.route('/weight/add', methods=['GET',
'POST'])
@login_required
def add():
    """Add a new weight entry."""
    user = get_current_user()
    if request.method == 'POST':
        weight_str = request.form.get('weight', '').strip()
        date_str = request.form.get('date', '').strip()
        notes = request.form.get('notes', '').strip()
        errors = []
        if not weight_str:
            errors.append('Please provide a weight value.')

```

```

        parsed_weight = 0.0
    try:
        parsed_weight = float(weight_str)
        if parsed_weight < 20 or parsed_weight > 400:
            errors.append('Weight must be between 20 kg
and 400 kg.')
    except ValueError:
        errors.append('Weight must be a valid number.')
    parsed_date = date.today()
    if date_str:
        try:
            parsed_date = datetime.strptime(date_str, '%Y-
%m-%d').date()
        except ValueError:
            errors.append('Invalid date format.')
    if errors:
        for err in errors:
            flash(err, 'danger')
        return render_template(
            'weight/add.html',
            weight=parsed_weight,
            date=date_str,
            notes=notes
        )
    new_record = WeightRecord(
        user_id=user.id,
        weight=parsed_weight,
        date=parsed_date,
        notes=notes if notes else None
    )
    db.session.add(new_record)
    # Update user's current weight if this record is today
    or most recent
    latest_record =
WeightRecord.query.filter_by(user_id=user.id).order_by(
WeightRecord.date.desc()).first()
    if not latest_record or parsed_date >=
latest_record.date:
        user.weight = parsed_weight
        db.session.commit()
        flash(f'Weight entry of {parsed_weight} kg logged
successfully!', 'success')
        return redirect(url_for('weight.history'))
    return render_template(
        'weight/add.html',
        today_date=date.today().strftime('%Y-%m-%d'),
        user=user
    )
@weight_bp.route('/weight/<int:record_id>/edit',
methods=['GET', 'POST'])
@login_required
def edit(record_id):
    """Edit a weight record."""
    record = db.get_or_404(WeightRecord, record_id)
    user = get_current_user()
    if record.user_id != user.id and user.role != 'admin':
        flash('Unauthorized action.', 'danger')
        return redirect(url_for('weight.history'))
    if request.method == 'POST':
        weight_str = request.form.get('weight', '').strip()
        date_str = request.form.get('date', '').strip()
        notes = request.form.get('notes', '').strip()
        errors = []
        parsed_weight = record.weight
        try:
            parsed_weight = float(weight_str)
            if parsed_weight < 20 or parsed_weight > 400:
                errors.append('Weight must be between 20 kg
and 400 kg.')

```

```

        except ValueError:
            errors.append('Weight must be a valid number.')
        parsed_date = record.date
        if date_str:
            try:
                parsed_date = datetime.strptime(date_str, '%Y-
%m-%d').date()
            except ValueError:
                errors.append('Invalid date format.')
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('weight/edit.html',
record=record)
        record.weight = parsed_weight
        record.date = parsed_date
        record.notes = notes if notes else None
        # Re-sync current weight from latest date
        latest =
WeightRecord.query.filter_by(user_id=user.id).order_by(
WeightRecord.date.desc(), WeightRecord.id.desc()).first()
        if latest:
            user.weight = latest.weight
            db.session.commit()
            flash('Weight record updated successfully!', 'success')
            return redirect(url_for('weight.history'))
        return render_template('weight/edit.html',
record=record)
@weight_bp.route('/weight/<int:record_id>/delete',
methods=['POST'])
@login_required
def delete(record_id):
    """Delete a weight record."""
    record = db.get_or_404(WeightRecord, record_id)
    user = get_current_user()
    if record.user_id != user.id and user.role != 'admin':
        flash('Unauthorized action.', 'danger')
        return redirect(url_for('weight.history'))
    db.session.delete(record)
    db.session.commit()
    # Re-sync user's current weight from remaining latest
    record
    latest =
WeightRecord.query.filter_by(user_id=user.id).order_by(
WeightRecord.date.desc(), WeightRecord.id.desc()).first()
    if latest:
        user.weight = latest.weight
    else:
        user.weight = None
    db.session.commit()
    flash('Weight record deleted.', 'success')
    return redirect(url_for('weight.history'))

workout.py:
from datetime import datetime, date
from flask import Blueprint, render_template, request,
redirect, url_for, flash
from models import db, Workout
from . import login_required, get_current_user
workout_bp = Blueprint('workout', __name__)
WORKOUT_TYPES = [
    'Running',
    'Cycling',
    'Walking',
    'Strength Training',
    'Yoga',
    'Cardio',
    'Swimming',
    'HIIT',
    'Pilates',

```

```

    'Other'
]
@workout_bp.route('/workouts')
@login_required
def history():
    """View workout history with filtering."""
    user = get_current_user()
    workout_type_filter = request.args.get('type', '').strip()
    search_query = request.args.get('search', '').strip()
    query = Workout.query.filter_by(user_id=user.id)
    if workout_type_filter:
        query = query.filter_by(workout_type=workout_type_filter)
    if search_query:
        query = query.filter(
            (Workout.exercise_name.ilike(f'%{search_query}%')) |
            (Workout.notes.ilike(f'%{search_query}%'))
        )
    workouts = query.order_by(Workout.workout_date.desc(),
                               Workout.id.desc()).all()
    # Aggregate stats for this user
    total_workouts = len(workouts)
    total_duration = sum(w.duration for w in workouts)
    total_calories = sum(w.calories_burned for w in workouts)
    return render_template(
        'workouts/history.html',
        workouts=workouts,
        workout_types=WORKOUT_TYPES,
        selected_type=workout_type_filter,
        search_query=search_query,
        total_workouts=total_workouts,
        total_duration=total_duration,
        total_calories=round(total_calories, 1)
    )
@workout_bp.route('/workouts/add', methods=['GET', 'POST'])
@login_required
def add():
    """Record a new workout entry."""
    user = get_current_user()
    if request.method == 'POST':
        workout_type = request.form.get('workout_type',
                                         "").strip()
        exercise_name = request.form.get('exercise_name',
                                         "").strip()
        duration = request.form.get('duration', "").strip()
        sets = request.form.get('sets', '0').strip()
        repetitions = request.form.get('repetitions', '0').strip()
        calories_burned = request.form.get('calories_burned',
                                           '0').strip()
        workout_date_str = request.form.get('workout_date',
                                             "").strip()
        notes = request.form.get('notes', "").strip()
        errors = []
        if not workout_type:
            errors.append('Please select a workout type.')
        if not exercise_name or len(exercise_name) < 2:
            errors.append('Please provide an exercise or workout name.')
        parsed_duration = 0
        try:
            parsed_duration = int(duration)
            if parsed_duration <= 0 or parsed_duration >
1440:
                errors.append('Duration must be between 1 and
1440 minutes.')

```

```

        except ValueError:
            errors.append('Duration must be a valid integer in
minutes.')
        parsed_sets = 0
        if sets:
            try:
                parsed_sets = int(sets)
                if parsed_sets < 0:
                    errors.append('Sets cannot be negative.')
            except ValueError:
                errors.append('Sets must be an integer.')
        parsed_reps = 0
        if repetitions:
            try:
                parsed_reps = int(repetitions)
                if parsed_reps < 0:
                    errors.append('Repetitions cannot be
negative.')
            except ValueError:
                errors.append('Repetitions must be an integer.')
        parsed_calories = 0.0
        if calories_burned:
            try:
                parsed_calories = float(calories_burned)
                if parsed_calories < 0:
                    errors.append('Calories burned cannot be
negative.')
            except ValueError:
                errors.append('Calories burned must be a valid
number.')
        parsed_date = date.today()
        if workout_date_str:
            try:
                parsed_date =
datetime.strptime(workout_date_str, '%Y-%m-%d').date()
            except ValueError:
                errors.append('Invalid workout date format. Use
YYYY-MM-DD.')
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template(
                'workouts/add.html',
                workout_types=WORKOUT_TYPES,
                workout_type=workout_type,
                exercise_name=exercise_name,
                duration=duration,
                sets=sets,
                repetitions=repetitions,
                calories_burned=calories_burned,
                workout_date=workout_date_str,
                notes=notes
            )
        new_workout = Workout(
            user_id=user.id,
            workout_type=workout_type,
            exercise_name=exercise_name,
            duration=parsed_duration,
            sets=parsed_sets,
            repetitions=parsed_reps,
            calories_burned=parsed_calories,
            workout_date=parsed_date,
            notes=notes if notes else None
        )
        db.session.add(new_workout)
        db.session.commit()
        flash(f'Workout "{exercise_name}" logged
successfully!', 'success')
        return redirect(url_for('workout.history'))

```

```

return render_template(
    'workouts/add.html',
    workout_types=WORKOUT_TYPES,
    today_date=date.today().strftime('%Y-%m-%d')
)
@workout_bp.route('/workouts/<int:workout_id>/edit',
methods=['GET', 'POST'])
@login_required
def edit(workout_id):
    """Edit an existing workout entry."""
    workout = db.get_or_404(Workout, workout_id)
    user = get_current_user()
    # RBAC: Only owner or admin can edit
    if workout.user_id != user.id and user.role != 'admin':
        flash('You are not authorized to edit this workout.',
'danger')
    return redirect(url_for('workout.history'))
    if request.method == 'POST':
        workout_type = request.form.get('workout_type',
'').strip()
        exercise_name = request.form.get('exercise_name',
'').strip()
        duration = request.form.get('duration', '').strip()
        sets = request.form.get('sets', '0').strip()
        repetitions = request.form.get('repetitions', '0').strip()
        calories_burned = request.form.get('calories_burned',
'0').strip()
        workout_date_str = request.form.get('workout_date',
'').strip()
        notes = request.form.get('notes', '').strip()
        errors = []
        if not workout_type:
            errors.append('Please select a workout type.')
        if not exercise_name or len(exercise_name) < 2:
            errors.append('Please provide an exercise name.')
        parsed_duration = 0
        try:
            parsed_duration = int(duration)
            if parsed_duration <= 0 or parsed_duration >
1440:
                errors.append('Duration must be between 1 and
1440 minutes.')
        except ValueError:
            errors.append('Duration must be a valid integer.')
        parsed_sets = 0
        if sets:
            try:
                parsed_sets = int(sets)
                if parsed_sets < 0:
                    errors.append('Sets cannot be negative.')
            except ValueError:
                errors.append('Sets must be an integer.')
        parsed_reps = 0
        if repetitions:
            try:
                parsed_reps = int(repetitions)
                if parsed_reps < 0:
                    errors.append('Repetitions cannot be
negative.')
            except ValueError:
                errors.append('Repetitions must be an integer.')
        parsed_calories = 0.0
        if calories_burned:
            try:
                parsed_calories = float(calories_burned)
                if parsed_calories < 0:
                    errors.append('Calories cannot be negative.')
            except ValueError:
                errors.append('Calories must be a number.')

```

```

        parsed_date = workout.workout_date
        if workout_date_str:
            try:
                parsed_date =
datetime.strptime(workout_date_str, '%Y-%m-%d').date()
            except ValueError:
                errors.append('Invalid date format.')
        if errors:
            for err in errors:
                flash(err, 'danger')
            return render_template('workouts/edit.html',
workout=workout, workout_types=WORKOUT_TYPES)
        workout.workout_type = workout_type
        workout.exercise_name = exercise_name
        workout.duration = parsed_duration
        workout.sets = parsed_sets
        workout.repetitions = parsed_reps
        workout.calories_burned = parsed_calories
        workout.workout_date = parsed_date
        workout.notes = notes if notes else None
        db.session.commit()
        flash('Workout updated successfully!', 'success')
        return redirect(url_for('workout.history'))
    return render_template('workouts/edit.html',
workout=workout, workout_types=WORKOUT_TYPES)
@workout_bp.route('/workouts/<int:workout_id>/delete',
methods=['POST'])
@login_required
def delete(workout_id):
    """Delete a workout entry."""
    workout = db.get_or_404(Workout, workout_id)
    user = get_current_user()
    if workout.user_id != user.id and user.role != 'admin':
        flash('You are not authorized to delete this workout.',
'danger')
    return redirect(url_for('workout.history'))
    db.session.delete(workout)
    db.session.commit()
    flash('Workout record deleted successfully.', 'success')
    return redirect(url_for('workout.history'))
style.css:
@import
url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+
a+Sans:wght@300;400;500;600;700;800&display=swap')
;
:root {
    --bg-primary: #0b1120;
    --bg-secondary: #0f172a;
    --bg-card: #1e293b;
    --bg-card-hover: #273549;
    --bg-glass: rgba(30, 41, 59, 0.75);

    --accent-emerald: #10b981;
    --accent-cyan: #06b6d4;
    --accent-blue: #3b82f6;
    --accent-indigo: #6366f1;
    --accent-violet: #8b5cf6;
    --accent-amber: #f59e0b;
    --accent-rose: #f43f5e;

    --text-primary: #f8fafc;
    --text-secondary: #94a3b8;
    --text-muted: #64748b;

    --border-color: rgba(255, 255, 255, 0.08);
    --border-highlight: rgba(16, 185, 129, 0.3);

    --gradient-emerald: linear-gradient(135deg, #10b981
0%, #06b6d4 100%);

```

```

--gradient-violet: linear-gradient(135deg, #6366f1 0%,
#a855f7 100%);
--gradient-amber: linear-gradient(135deg, #f59e0b 0%,
#ea580c 100%);
--gradient-rose: linear-gradient(135deg, #f43f5e 0%,
#e11d48 100%);
--gradient-dark: linear-gradient(180deg, #0f172a 0%,
#0b1120 100%);
--shadow-sm: 0 2px 4px rgba(0, 0, 0, 0.2);
--shadow-md: 0 4px 12px rgba(0, 0, 0, 0.3);
--shadow-lg: 0 10px 25px rgba(0, 0, 0, 0.4);
--shadow-glow: 0 0 20px rgba(16, 185, 129, 0.25);
--radius-sm: 8px;
--radius-md: 14px;
--radius-lg: 20px;
}
* {
  box-sizing: border-box;
}
body {
  font-family: 'Plus Jakarta Sans', -apple-system,
BlinkMacSystemFont, sans-serif;
background-color: var(--bg-primary);
color: var(--text-primary);
min-height: 100vh;
display: flex;
flex-direction: column;
overflow-x: hidden;
}
.navbar-custom {
  background-color: rgba(15, 23, 42, 0.85);
backdrop-filter: blur(16px);
-webkit-backdrop-filter: blur(16px);
border-bottom: 1px solid var(--border-color);
padding: 0.85rem 1.5rem;
position: sticky;
top: 0;
z-index: 1030;
transition: all 0.3s ease;
}
.brand-logo {
  font-weight: 800;
  font-size: 1.45rem;
  letter-spacing: -0.5px;
  background: var(--gradient-emerald);
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
display: flex;
align-items: center;
gap: 0.5rem;
text-decoration: none;
}
.brand-logo i {
  -webkit-text-fill-color: #10b981;
}
.nav-link-custom {
  color: var(--text-secondary);
  font-weight: 500;
  font-size: 0.95rem;
  padding: 0.5rem 0.9rem !important;
border-radius: var(--radius-sm);
transition: all 0.2s ease;
}
.nav-link-custom:hover,
.nav-link-custom.active {
  color: #fff;
  background-color: rgba(255, 255, 255, 0.06);
}
.nav-link-custom.active {
  color: var(--accent-emerald);
  font-weight: 600;
}
}
.role-badge {
  font-size: 0.72rem;
  text-transform: uppercase;
  font-weight: 700;
  letter-spacing: 0.8px;
  padding: 0.25rem 0.6rem;
border-radius: 20px;
}
}
.card-dark {
  background-color: var(--bg-card);
border: 1px solid var(--border-color);
border-radius: var(--radius-md);
box-shadow: var(--shadow-md);
color: var(--text-primary);
transition: transform 0.2s ease, box-shadow 0.2s ease,
border-color 0.2s ease;
}
.card-dark:hover {
  border-color: rgba(255, 255, 255, 0.14);
}
}
.card-dark:hover:active {
  transform: translateY(-3px);
  box-shadow: var(--shadow-lg);
border-color: var(--border-highlight);
}
}
.card-header-dark {
  background-color: rgba(15, 23, 42, 0.4);
border-bottom: 1px solid var(--border-color);
padding: 1rem 1.25rem;
font-weight: 600;
}
}
.stat-card {
  padding: 1.25rem;
border-radius: var(--radius-md);
background-color: var(--bg-card);
border: 1px solid var(--border-color);
position: relative;
overflow: hidden;
transition: all 0.25s ease;
}
}
.stat-card:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-lg);
}
}
.stat-card::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 3px;
background: var(--gradient-emerald);
opacity: 0.8;
}
}
.stat-card.stat-violet::before {
  background: var(--gradient-violet);
}
}
.stat-card.stat-amber::before {
  background: var(--gradient-amber);
}
}
.stat-card.stat-rose::before {
  background: var(--gradient-rose);
}
}
.stat-icon {
  width: 48px;
  height: 48px;

```

```

border-radius: 12px;
display: flex;
align-items: center;
justify-content: center;
font-size: 1.4rem;
background: rgba(16, 185, 129, 0.12);
color: var(--accent-emerald);
}
.stat-icon.icon-violet {
background: rgba(99, 102, 241, 0.12);
color: var(--accent-indigo);
}
.stat-icon.icon-amber {
background: rgba(245, 158, 11, 0.12);
color: var(--accent-amber);
}
.stat-icon.icon-rose {
background: rgba(244, 63, 94, 0.12);
color: var(--accent-rose);
}
.stat-value {
font-size: 1.85rem;
font-weight: 800;
letter-spacing: -0.5px;
color: var(--text-primary);
line-height: 1.1;
}
.stat-label {
font-size: 0.85rem;
color: var(--text-secondary);
font-weight: 500;
}
.form-control-dark,
.form-select-dark {
background-color: rgba(15, 23, 42, 0.6) !important;
border: 1px solid rgba(255, 255, 255, 0.12) !important;
color: var(--text-primary) !important;
border-radius: var(--radius-sm) !important;
padding: 0.65rem 0.95rem;
font-size: 0.95rem;
transition: all 0.2s ease;
}
.form-control-dark:focus,
.form-select-dark:focus {
border-color: var(--accent-emerald) !important;
box-shadow: 0 0 3px rgba(16, 185, 129, 0.2) !important;
background-color: rgba(15, 23, 42, 0.8) !important;
}
.form-control-dark::placeholder {
color: var(--text-muted);
}
.form-label-dark {
font-weight: 600;
font-size: 0.88rem;
color: var(--text-secondary);
margin-bottom: 0.35rem;
}
.btn-emerald {
background: var(--gradient-emerald);
color: #fff;
font-weight: 600;
border: none;
border-radius: var(--radius-sm);
padding: 0.6rem 1.25rem;
transition: all 0.25s ease;
box-shadow: 0 2px 8px rgba(16, 185, 129, 0.25);
}
.btn-emerald:hover {
color: #fff;
transform: translateY(-1px);
box-shadow: 0 4px 14px rgba(16, 185, 129, 0.35);
}
.btn-emerald:active {
transform: translateY(0);
}
.btn-outline-custom {
color: var(--text-secondary);
border: 1px solid rgba(255, 255, 255, 0.15);
background: transparent;
border-radius: var(--radius-sm);
font-weight: 500;
padding: 0.55rem 1.15rem;
transition: all 0.2s ease;
}
.btn-outline-custom:hover {
color: #fff;
border-color: rgba(255, 255, 255, 0.3);
background: rgba(255, 255, 255, 0.05);
}
.table-dark-custom {
--bs-table-bg: transparent;
--bs-table-color: var(--text-primary);
border-color: var(--border-color);
margin-bottom: 0;
}
.table-dark-custom thead th {
background-color: rgba(15, 23, 42, 0.5);
color: var(--text-secondary);
font-weight: 600;
font-size: 0.82rem;
text-transform: uppercase;
letter-spacing: 0.6px;
border-bottom: 1px solid var(--border-color);
padding: 0.85rem 1rem;
}
.table-dark-custom tbody td {
padding: 0.85rem 1rem;
border-bottom: 1px solid var(--border-color);
vertical-align: middle;
font-size: 0.92rem;
}
.table-dark-custom tbody tr:hover {
background-color: rgba(255, 255, 255, 0.02);
}
.bmi-meter-bar {
height: 12px;
border-radius: 6px;
background: linear-gradient(90deg, #06b6d4 0%, #10b981 35%, #f59e0b 70%, #f43f5e 100%);
position: relative;
margin: 1.5rem 0 0.5rem 0;
}
.bmi-meter-indicator {
position: absolute;
top: -6px;
width: 24px;
height: 24px;
background-color: #fff;
border: 3px solid #0f172a;
border-radius: 50%;
transform: translateX(-50%);
box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
transition: left 0.4s cubic-bezier(0.34, 1.56, 0.64, 1);
}
.hero-section {
padding: 5rem 0 4rem 0;
position: relative;

```

```

    overflow: hidden;
  }
  .hero-title {
    font-size: 3.25rem;
    font-weight: 800;
    letter-spacing: -1px;
    line-height: 1.15;
    margin-bottom: 1.25rem;
  }
  .hero-subtitle {
    font-size: 1.2rem;
    color: var(--text-secondary);
    max-width: 600px;
    line-height: 1.6;
    margin-bottom: 2rem;
  }
  .feature-icon-box {
    width: 56px;
    height: 56px;
    border-radius: 16px;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.6rem;
    margin-bottom: 1.25rem;
  }
  .alert-custom {
    border-radius: var(--radius-sm);
    border: 1px solid transparent;
    padding: 0.9rem 1.2rem;
    font-size: 0.92rem;
    font-weight: 500;
  }
  .modal-content-dark {
    background-color: var(--bg-card);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    color: var(--text-primary);
  }
  .modal-header-dark {
    border-bottom: 1px solid var(--border-color);
  }
  .modal-footer-dark {
    border-top: 1px solid var(--border-color);
  }
  footer {
    background-color: var(--bg-secondary);
    border-top: 1px solid var(--border-color);
    margin-top: auto;
    padding: 1.5rem 0;
    color: var(--text-muted);
    font-size: 0.88rem;
  }
  @media (max-width: 768px) {
    .hero-title {
      font-size: 2.25rem;
    }
    .stat-value {
      font-size: 1.5rem;
    }
  }
  bmi.js:
  document.addEventListener('DOMContentLoaded', () =>
  {
    const heightInput =
    document.getElementById('bmiHeightInput');
    const heightSlider =
    document.getElementById('bmiHeightSlider');

```

```

    const weightInput =
    document.getElementById('bmiWeightInput');
    const weightSlider =
    document.getElementById('bmiWeightSlider');
    const bmiValueEl =
    document.getElementById('bmiDisplayValue');
    const bmiCategoryEl =
    document.getElementById('bmiDisplayCategory');
    const bmiAdviceEl =
    document.getElementById('bmiDisplayAdvice');
    const bmiIndicatorEl =
    document.getElementById('bmiIndicator');
    const minHealthyEl =
    document.getElementById('minHealthyWeight');
    const maxHealthyEl =
    document.getElementById('maxHealthyWeight');
    if (!heightInput || !weightInput) return;
    function calculateBMI(heightCm, weightKg) {
      if (heightCm <= 0 || weightKg <= 0) return null;
      const heightM = heightCm / 100.0;
      const bmi = (weightKg / (heightM * heightM));

      let category = 'Normal weight';
      let colorClass = 'bg-success';
      let advice = 'You are within a healthy weight range.
      Keep maintaining your active fitness routines!';

      if (bmi < 18.5) {
        category = 'Underweight';
        colorClass = 'bg-info text-dark';
        advice = 'Your BMI indicates you are
        underweight. Consider speaking with our nutritionist to
        build a healthy calorie-dense meal plan.';
      } else if (bmi >= 18.5 && bmi < 25.0) {
        category = 'Normal weight';
        colorClass = 'bg-success';
        advice = 'Excellent work! Your BMI is in the ideal
        healthy zone. Maintain your balanced diet and workout
        frequency.';
      } else if (bmi >= 25.0 && bmi < 30.0) {
        category = 'Overweight';
        colorClass = 'bg-warning text-dark';
        advice = 'Your BMI is in the overweight range.
        Regular cardio sessions and balanced calorie tracking can
        help you reach a healthy baseline.';
      } else {
        category = 'Obesity';
        colorClass = 'bg-danger';
        advice = 'Your BMI falls into the obesity category.
        A structured fitness routine and nutritional consultation
        are strongly recommended.';
      }
      const minHealthy = (18.5 * heightM *
      heightM).toFixed(1);
      const maxHealthy = (24.9 * heightM *
      heightM).toFixed(1);
      // Map BMI (15 to 40) to percentage (0% to 100%)
      for gauge indicator
      let percentage = ((bmi - 15) / (40 - 15)) * 100;
      if (percentage < 0) percentage = 0;
      if (percentage > 100) percentage = 100;
      return {
        bmi: bmi.toFixed(1),
        category: category,
        colorClass: colorClass,
        advice: advice,
        percentage: percentage,
        minHealthy: minHealthy,
        maxHealthy: maxHealthy
      }
    }
  }

```

```

    };
  }
  function updateUI() {
    const h = parseFloat(heightInput.value) || 0;
    const w = parseFloat(weightInput.value) || 0;
    const res = calculateBMI(h, w);
    if (!res) return;
    if (bmiValueEl) bmiValueEl.textContent = res.bmi;
    if (bmiCategoryEl) {
      bmiCategoryEl.textContent = res.category;
      bmiCategoryEl.className = `badge
    ${res.colorClass} fs-6 px-3 py-2`;
    }
    if (bmiAdviceEl) bmiAdviceEl.textContent =
    res.advice;
    if (bmiIndicatorEl) bmiIndicatorEl.style.left =
    `${res.percentage}%`;
    if (minHealthyEl) minHealthyEl.textContent =
    `${res.minHealthy} kg`;
    if (maxHealthyEl) maxHealthyEl.textContent =
    `${res.maxHealthy} kg`;
  }
  // Input-Slider two-way bindings
  heightInput.addEventListener('input', () => {
    if (heightSlider) heightSlider.value =
    heightInput.value;
    updateUI();
  });
  if (heightSlider) {
    heightSlider.addEventListener('input', () => {
      heightInput.value = heightSlider.value;
      updateUI();
    });
  }
  weightInput.addEventListener('input', () => {
    if (weightSlider) weightSlider.value =
    weightInput.value;
    updateUI();
  });
  if (weightSlider) {
    weightSlider.addEventListener('input', () => {
      weightInput.value = weightSlider.value;
      updateUI();
    });
  }
  // Initial run
  updateUI();
});
chart.js:
let weightChartInstance = null;
let workoutChartInstance = null;
let typeChartInstance = null;
let goalChartInstance = null;
if (window.Chart) {
  Chart.defaults.color = '#94a3b8';
  Chart.defaults.font.family = "'Plus Jakarta Sans', sans-
  serif";
  Chart.defaults.plugins.tooltip.backgroundColor =
  '#1e293b';
  Chart.defaults.plugins.tooltip.borderColor = 'rgba(255,
  255, 255, 0.1)';
  Chart.defaults.plugins.tooltip.borderWidth = 1;
  Chart.defaults.plugins.tooltip.padding = 10;
  Chart.defaults.plugins.tooltip.cornerRadius = 8;
  Chart.defaults.plugins.legend.labels.usePointStyle =
  true;
}
function loadProgressData(days = '30') {

```

```

  const loadingIndicators =
  document.querySelectorAll('.chart-spinner');
  loadingIndicators.forEach(el => el.classList.remove('d-
  none'));
  fetch(`/api/progress-data?days=${days}`)
    .then(res => res.json())
    .then(data => {
      loadingIndicators.forEach(el =>
    el.classList.add('d-none'));
      renderWeightChart(data.weight);
      renderWorkoutChart(data.workouts);
      renderTypeChart(data.types);
      renderGoalChart(data.goals);
    })
    .catch(err => {
      console.error('Error loading analytics data:', err);
      loadingIndicators.forEach(el =>
    el.classList.add('d-none'));
    });
}
function renderWeightChart(weightData) {
  const canvas =
  document.getElementById('weightProgressChart');
  const emptyNotice =
  document.getElementById('weightEmptyNotice');
  if (!canvas) return;
  if (!weightData || !weightData.has_data) {
    canvas.classList.add('d-none');
    if (emptyNotice) emptyNotice.classList.remove('d-
    none');
    return;
  }
  canvas.classList.remove('d-none');
  if (emptyNotice) emptyNotice.classList.add('d-none');
  const ctx = canvas.getContext('2d');
  if (weightChartInstance) weightChartInstance.destroy();
  const gradient = ctx.createLinearGradient(0, 0, 0, 300);
  gradient.addColorStop(0, 'rgba(16, 185, 129, 0.35)');
  gradient.addColorStop(1, 'rgba(16, 185, 129, 0.0)');
  weightChartInstance = new Chart(ctx, {
    type: 'line',
    data: {
      labels: weightData.labels,
      datasets: [{
        label: 'Weight (kg)',
        data: weightData.data,
        borderColor: '#10b981',
        backgroundColor: gradient,
        fill: true,
        tension: 0.35,
        borderWidth: 3,
        pointBackgroundColor: '#10b981',
        pointBorderColor: '#0f172a',
        pointBorderWidth: 2,
        pointRadius: 5,
        pointHoverRadius: 7
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      scales: {
        x: {
          grid: { color: 'rgba(255, 255, 255, 0.05)' },
          ticks: { color: '#94a3b8' }
        },
        y: {
          grid: { color: 'rgba(255, 255, 255, 0.05)' },
          ticks: {

```

```

        color: '#94a3b8',
        callback: val => `${val} kg`
      }
    },
    plugins: {
      legend: { display: false }
    }
  });
}
function renderWorkoutChart(workoutData) {
  const canvas =
document.getElementById('workoutProgressChart');
  const emptyNotice =
document.getElementById('workoutEmptyNotice');
  if (!canvas) return;
  if (!workoutData || !workoutData.has_data) {
    canvas.classList.add('d-none');
    if (emptyNotice) emptyNotice.classList.remove('d-none');
    return;
  }
  canvas.classList.remove('d-none');
  if (emptyNotice) emptyNotice.classList.add('d-none');
  const ctx = canvas.getContext('2d');
  if (workoutChartInstance)
workoutChartInstance.destroy();
  workoutChartInstance = new Chart(ctx, {
    type: 'bar',
    data: {
      labels: workoutData.labels,
      datasets: [
        {
          type: 'bar',
          label: 'Duration (mins)',
          data: workoutData.durations,
          backgroundColor: 'rgba(6, 182, 212, 0.7)',
          borderColor: '#06b6d4',
          borderWidth: 1,
          borderRadius: 6,
          yAxisID: 'y'
        },
        {
          type: 'line',
          label: 'Calories Burned (kcal)',
          data: workoutData.calories,
          borderColor: '#f59e0b',
          backgroundColor: 'transparent',
          borderWidth: 2.5,
          pointBackgroundColor: '#f59e0b',
          pointRadius: 4,
          yAxisID: 'y1'
        }
      ]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      scales: {
        x: {
          grid: { color: 'rgba(255, 255, 255, 0.05)' }
        },
        y: {
          type: 'linear',
          position: 'left',
          grid: { color: 'rgba(255, 255, 255, 0.05)' },
          title: { display: true, text: 'Minutes', color:
'#94a3b8' }
        }
      }
    }
  });
}
function renderTypeChart(typeData) {
  const canvas =
document.getElementById('workoutTypeChart');
  const emptyNotice =
document.getElementById('typeEmptyNotice');
  if (!canvas) return;
  if (!typeData || !typeData.has_data) {
    canvas.classList.add('d-none');
    if (emptyNotice) emptyNotice.classList.remove('d-none');
    return;
  }
  canvas.classList.remove('d-none');
  if (emptyNotice) emptyNotice.classList.add('d-none');
  const ctx = canvas.getContext('2d');
  if (typeChartInstance) typeChartInstance.destroy();
  const colors = ['#10b981', '#06b6d4', '#6366f1',
'#a855f7', '#f59e0b', '#f43f5e', '#ec4899', '#14b8a6'];
  typeChartInstance = new Chart(ctx, {
    type: 'doughnut',
    data: {
      labels: typeData.labels,
      datasets: [{
        data: typeData.counts,
        backgroundColor: colors.slice(0,
typeData.labels.length),
        borderColor: '#1e293b',
        borderWidth: 3
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: {
        legend: {
          position: 'bottom',
          labels: { boxWidth: 12, padding: 15 }
        }
      },
      cutout: '65%'
    }
  });
}
function renderGoalChart(goalData) {
  const canvas =
document.getElementById('goalStatusChart');
  const emptyNotice =
document.getElementById('goalEmptyNotice');
  if (!canvas) return;
  if (!goalData || !goalData.has_data) {
    canvas.classList.add('d-none');
    if (emptyNotice) emptyNotice.classList.remove('d-none');
    return;
  }
  canvas.classList.remove('d-none');
  if (emptyNotice) emptyNotice.classList.add('d-none');

```

```

    },
    y1: {
      type: 'linear',
      position: 'right',
      grid: { drawOnChartArea: false },
      title: { display: true, text: 'Calories (kcal)',
color: '#f59e0b' }
    }
  });
}
function renderTypeChart(typeData) {
  const canvas =
document.getElementById('workoutTypeChart');
  const emptyNotice =
document.getElementById('typeEmptyNotice');
  if (!canvas) return;
  if (!typeData || !typeData.has_data) {
    canvas.classList.add('d-none');
    if (emptyNotice) emptyNotice.classList.remove('d-
none');
    return;
  }
  canvas.classList.remove('d-none');
  if (emptyNotice) emptyNotice.classList.add('d-none');
  const ctx = canvas.getContext('2d');
  if (typeChartInstance) typeChartInstance.destroy();
  const colors = ['#10b981', '#06b6d4', '#6366f1',
'#a855f7', '#f59e0b', '#f43f5e', '#ec4899', '#14b8a6'];
  typeChartInstance = new Chart(ctx, {
    type: 'doughnut',
    data: {
      labels: typeData.labels,
      datasets: [{
        data: typeData.counts,
        backgroundColor: colors.slice(0,
typeData.labels.length),
        borderColor: '#1e293b',
        borderWidth: 3
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: {
        legend: {
          position: 'bottom',
          labels: { boxWidth: 12, padding: 15 }
        }
      },
      cutout: '65%'
    }
  });
}
function renderGoalChart(goalData) {
  const canvas =
document.getElementById('goalStatusChart');
  const emptyNotice =
document.getElementById('goalEmptyNotice');
  if (!canvas) return;
  if (!goalData || !goalData.has_data) {
    canvas.classList.add('d-none');
    if (emptyNotice) emptyNotice.classList.remove('d-
none');
    return;
  }
  canvas.classList.remove('d-none');
  if (emptyNotice) emptyNotice.classList.add('d-none');

```

```

const ctx = canvas.getContext('2d');
if (goalChartInstance) goalChartInstance.destroy();
goalChartInstance = new Chart(ctx, {
  type: 'pie',
  data: {
    labels: goalData.labels,
    datasets: [{
      data: goalData.data,
      backgroundColor: ['#3b82f6', '#10b981',
'ef4444'],
      borderColor: '#1e293b',
      borderWidth: 3
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: {
        position: 'bottom',
        labels: { boxWidth: 12, padding: 15 }
      }
    }
  }
});
document.addEventListener('DOMContentLoaded', () => {
  if
(document.getElementById('progressDashboardContainer')
)) {
    loadProgressData('30');
    const filterPills =
document.querySelectorAll('.progress-range-btn');
    filterPills.forEach(btn => {
      btn.addEventListener('click', (e) => {
        filterPills.forEach(b =>
b.classList.remove('active', 'btn-emerald'));
        filterPills.forEach(b => b.classList.add('btn-
outline-custom'));

        btn.classList.add('active', 'btn-emerald');
        btn.classList.remove('btn-outline-custom');
        const days = btn.getAttribute('data-days');
        loadProgressData(days);
      });
    });
  }
});
main.js:
document.addEventListener('DOMContentLoaded', () => {
  const togglePasswordBtns =
document.querySelectorAll('.toggle-password-btn');
  togglePasswordBtns.forEach(btn => {
    btn.addEventListener('click', () => {
      const targetId = btn.getAttribute('data-target');
      const input = document.getElementById(targetId);
      const icon = btn.querySelector('i');
      if (input) {
        if (input.type === 'password') {
          input.type = 'text';
          if (icon) {
            icon.classList.remove('bi-eye');
            icon.classList.add('bi-eye-slash');
          }
        } else {
          input.type = 'password';
          if (icon) {

```

```

            icon.classList.remove('bi-eye-slash');
            icon.classList.add('bi-eye');
          }
        }
      });
    });
    const deleteModal =
document.getElementById('deleteConfirmModal');
    if (deleteModal) {
      const deleteForm =
document.getElementById('deleteConfirmForm');
      const deleteItemName =
document.getElementById('deleteItemName');
      const deleteButtons =
document.querySelectorAll('.btn-delete-trigger');
      deleteButtons.forEach(btn => {
        btn.addEventListener('click', () => {
          const actionUrl = btn.getAttribute('data-action');
          const itemName = btn.getAttribute('data-name')
|| 'this item';

          if (deleteForm) deleteForm.action = actionUrl;
          if (deleteItemName)
deleteItemName.textContent = itemName;
        });
      });
    }
    const autoDismissAlerts =
document.querySelectorAll('.alert-auto-dismiss');
    autoDismissAlerts.forEach(alertEl => {
      setTimeout(() => {
        const bsAlert =
bootstrap.Alert.getOrCreateInstance(alertEl);
        if (bsAlert) bsAlert.close();
      }, 5000);
    });
    const tooltipTriggerList =
[].slice.call(document.querySelectorAll('[data-bs-
toggle="tooltip"]'));
    tooltipTriggerList.map(tooltipTriggerEl => new
bootstrap.Tooltip(tooltipTriggerEl));
  });
Admin:
Dashboard.html:
{% extends 'base.html' %}
{% block title %}Admin Dashboard - FitPulse{%
endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Header -->
  <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
    <div>
      <h2 class="text-white fw-bold mb-1">
System Administration <span class="badge bg-
danger fs-6">Admin</span>
      </h2>
      <p class="text-secondary small mb-0">Platform
overview, user management, and global database
monitoring.</p>
    </div>
    <div class="d-flex gap-2 flex-wrap">
      <a href="{{ url_for('admin.create_user') }}"
class="btn btn-emerald btn-sm">
        <i class="bi bi-person-plus-fill me-1"></i>
Create User
      </a>

```

```

    <a href="{{ url_for('admin.list_users') }}"
class="btn btn-outline-custom btn-sm">
    <i class="bi bi-people me-1"></i> Manage
Users
    </a>
</div>
</div>
<!-- Summary Metrics -->
<div class="row g-3 mb-4">
    <div class="col-sm-6 col-xl-3">
        <div class="stat-card">
            <div class="stat-label">Total Users</div>
            <div class="stat-value">{{ total_users }}</div>
            <small class="text-secondary">{{
users_by_role['user'] }} Clients &bull; {{
users_by_role['trainer'] }} Trainers &bull; {{
users_by_role['nutritionist'] }} Nutritionists</small>
        </div>
    </div>
    <div class="col-sm-6 col-xl-3">
        <div class="stat-card stat-violet">
            <div class="stat-label">Total Workouts</div>
            <div class="stat-value">{{ total_workouts
}}</div>
            <small class="text-indigo">Logged platform-
wide</small>
        </div>
    </div>
    <div class="col-sm-6 col-xl-3">
        <div class="stat-card stat-amber">
            <div class="stat-label">Total Goals</div>
            <div class="stat-value">{{ total_goals }}</div>
            <small class="text-warning">Active &amp;
completed goals</small>
        </div>
    </div>
    <div class="col-sm-6 col-xl-3">
        <div class="stat-card stat-rose">
            <div class="stat-label">Weight Records</div>
            <div class="stat-value">{{ total_weights
}}</div>
            <small class="text-rose">Body metrics
logged</small>
        </div>
    </div>
</div>
<!-- Quick Navigation Management Cards -->
<div class="row g-4 mb-4">
    <div class="col-md-3">
        <a href="{{ url_for('admin.list_users') }}"
class="card card-dark card-dark-hover p-3 text-
decoration-none d-block">
            <div class="d-flex align-items-center gap-3">
                <div class="stat-icon" style="background:
rgb(16, 185, 129, 0.15); color: var(--accent-emerald);">
                    <i class="bi bi-person-gear"></i>
                </div>
                <div>
                    <h6 class="text-white fw-bold mb-0">User
Management</h6>
                    <small class="text-secondary">{{
total_users }} registered</small>
                </div>
            </div>
        </a>
    </div>
    <div class="col-md-3">

```

```

    <a href="{{ url_for('admin.list_workouts') }}"
class="card card-dark card-dark-hover p-3 text-
decoration-none d-block">
        <div class="d-flex align-items-center gap-3">
            <div class="stat-icon icon-violet">
                <i class="bi bi-activity"></i>
            </div>
            <div>
                <h6 class="text-white fw-bold mb-
0">Workout Records</h6>
                <small class="text-secondary">{{
total_workouts }} entries</small>
            </div>
        </div>
    </a>
</div>
    <div class="col-md-3">
        <a href="{{ url_for('admin.list_goals') }}"
class="card card-dark card-dark-hover p-3 text-
decoration-none d-block">
            <div class="d-flex align-items-center gap-3">
                <div class="stat-icon icon-amber">
                    <i class="bi bi-trophy"></i>
                </div>
                <div>
                    <h6 class="text-white fw-bold mb-
0">Fitness Goals</h6>
                    <small class="text-secondary">{{
total_goals }} tracked</small>
                </div>
            </div>
        </a>
    </div>
    <div class="col-md-3">
        <a href="{{ url_for('admin.list_weights') }}"
class="card card-dark card-dark-hover p-3 text-
decoration-none d-block">
            <div class="d-flex align-items-center gap-3">
                <div class="stat-icon icon-rose">
                    <i class="bi bi-speedometer"></i>
                </div>
                <div>
                    <h6 class="text-white fw-bold mb-
0">Weight Logs</h6>
                    <small class="text-secondary">{{
total_weights }} entries</small>
                </div>
            </div>
        </a>
    </div>
</div>
<!-- Recent System Activity -->
<div class="row g-4">
    <!-- Recent Users Registered -->
    <div class="col-lg-6">
        <div class="card card-dark">
            <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
                <span class="text-white fw-bold"><i
class="bi bi-person-plus text-emerald me-2"></i> Recent
Registrations</span>
                <a href="{{ url_for('admin.list_users') }}"
class="btn btn-sm btn-outline-custom">All Users</a>
            </div>
            <div class="card-body p-0">
                <div class="table-responsive">
                    <table class="table table-dark-custom mb-
0">
                        <thead>

```

```

<tr><th>User</th><th>Email</th><th>Role</th><th>
class="text-end">Registered</th></tr>
</thead>
<tbody>
  {% for u in recent_users %}
    <tr>
      <td class="fw-bold text-
white">{{ u.name }}</td>
      <td class="text-secondary
small">{{ u.email }}</td>
      <td><span class="badge bg-
secondary">{{ u.role }}</span></td>
      <td class="text-end text-
secondary small">{{ u.created_at.strftime("%b %d, %Y")
}}</td>
    </tr>
  {% endfor %}
</tbody>
</table>
</div>
</div>
</div>
</div>
<!-- Recent Workouts Logged Platform-wide -->
<div class="col-lg-6">
  <div class="card card-dark">
    <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
      <span class="text-white fw-bold"><i>Recent
Workouts (Global)</i></span>
      <a href="{{ url_for('admin.list_workouts')
}}" class="btn btn-sm btn-outline-custom">All
Workouts</a>
    </div>
    <div class="card-body p-0">
      <div class="table-responsive">
        <table class="table table-dark-custom mb-
0">
          <thead>
            <tr><th>User</th><th>Exercise</th><th>Type</th><th>
Duration</th><th class="text-end">Date</th></tr>
          </thead>
          <tbody>
            {% for w in recent_workouts %}
              <tr>
                <td class="text-white">{{
w.user.name }}</td>
                <td class="text-emerald fw-
semibold">{{ w.exercise_name }}</td>
                <td><span class="badge bg-
secondary">{{ w.workout_type }}</span></td>
                <td class="text-secondary">{{
w.duration }} mins</td>
                <td class="text-end text-
secondary small">{{ w.workout_date.strftime("%b %d")
}}</td>
              </tr>
            {% endfor %}
          </tbody>
        </table>
      </div>
    </div>
  </div>
</div>
</div>
</div>
</div>

```

```
{% endblock %}
```

Goals.html:

```

{% extends 'base.html' %}
{% block title %}Goal Management - Admin Portal{%
endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Header -->
  <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
    <div>
      <h2 class="text-white fw-bold mb-1"><i>bi bi-
trophy text-warning me-2"></i> Global Goal
Management</h2>
      <p class="text-secondary small mb-0">Inspect and
monitor fitness goals tracked by all platform athletes.</p>
    </div>
    <a href="{{ url_for('admin.dashboard') }}"
class="btn btn-outline-custom">
      <i>bi bi-arrow-left me-1"></i> Admin
Dashboard
    </a>
  </div>
  <!-- Table -->
  <div class="card card-dark shadow-sm">
    <div class="card-body p-0">
      {% if goals %}
        <div class="table-responsive">
          <table class="table table-dark-custom mb-0">
            <thead>
              <tr>
                <th>ID</th>
                <th>User</th>
                <th>Goal Type</th>
                <th>Target Value</th>
                <th>Current Value</th>
                <th>Start Date</th>
                <th>Target Date</th>
                <th>Status</th>
                <th class="text-end">Actions</th>
              </tr>
            </thead>
            <tbody>
              {% for g in goals %}
                <tr>
                  <td class="text-secondary">#{{
g.id }}</td>
                  <td class="fw-bold text-white">{{
g.user.name }} ({{ g.user.email }})</td>
                  <td><span class="text-cyan">{{
g.goal_type }}</span></td>
                  <td class="fw-semibold text-
white">{{ g.target_value }}</td>
                  <td class="text-secondary
small">{{ g.current_value or '-' }}</td>
                  <td class="text-secondary
small">{{ g.start_date.strftime("%Y-%m-%d") }}</td>
                  <td class="text-secondary
small">{{ g.target_date.strftime("%Y-%m-%d") }}</td>
                  <td><span class="badge bg-{{
g.badge_color }}">{{ g.status }}</span></td>
                  <td class="text-end">
                    <button type="button" class="btn
btn-sm btn-outline-danger btn-delete-trigger" data-bs-
toggle="modal" data-bs-target="#deleteConfirmModal"
data-action="{{ url_for('admin.delete_goal', goal_id=g.id)
}}" data-name="Goal #{{ g.id }} ({{ g.goal_type }})">
                      <i>bi bi-trash"></i>
                    </button>

```

```

        </td>
      </tr>
    {% endfor %}
  </tbody>
</table>
</div>
{% else %}
  <div class="text-center py-5">
    <p class="text-secondary mb-0">No goals
found.</p>
  </div>
{% endif %}
</div>
</div>
</div>
{% endblock %}
User_create.html:
{% extends 'base.html' %}
{% block title %}Create User - Admin Portal{% endblock %}
{% block content %}
<div class="container py-4">
  <div class="row justify-content-center">
    <div class="col-lg-8">
      <div class="card card-dark shadow-lg">
        <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
          <span class="text-white fw-bold"><i>
class="bi bi-person-plus-fill text-emerald me-2"></i>
Create New User Account</span>
          <a href="{{ url_for('admin.list_users') }}"
class="btn btn-sm btn-outline-custom">Back to
Users</a>
        </div>
        <div class="card-body p-4">
          <form method="POST" action="{{
url_for('admin.create_user') }}">
            <div class="row g-3 mb-3">
              <div class="col-md-6">
                <label for="name" class="form-label
form-label-dark">Full Name <span class="text-
danger">*</span></label>
                <input type="text" class="form-
control form-control-dark" id="name" name="name"
value="{{ name or '' }}" placeholder="e.g. John Doe"
required>
              </div>
              <div class="col-md-6">
                <label for="email" class="form-label
form-label-dark">Email Address <span class="text-
danger">*</span></label>
                <input type="email" class="form-
control form-control-dark" id="email" name="email"
value="{{ email or '' }}"
placeholder="john@example.com" required>
              </div>
            </div>
            <div class="row g-3 mb-3">
              <div class="col-md-6">
                <label for="password" class="form-
label form-label-dark">Initial Password <span
class="text-danger">*</span></label>
                <input type="password" class="form-
control form-control-dark" id="password"
name="password" placeholder="Min 6 characters"
minlength="6" required>
              </div>
              <div class="col-md-6">

```

```

        <label for="role" class="form-label
form-label-dark">System Role <span class="text-
danger">*</span></label>
        <select class="form-select form-
select-dark" id="role" name="role" required>
          <option value="user" {% if role ==
'user' %}>selected{% endif %}>User (Client)</option>
          <option value="trainer" {% if role
== 'trainer' %}>selected{% endif %}>Trainer</option>
          <option value="nutritionist" {% if
role == 'nutritionist' %}>selected{% endif
%}>Nutritionist</option>
          <option value="admin" {% if role
== 'admin' %}>selected{% endif
%}>Administrator</option>
        </select>
      </div>
    </div>
    <div class="row g-3 mb-3">
      <div class="col-md-6">
        <label for="gender" class="form-label
form-label-dark">Gender</label>
        <select class="form-select form-
select-dark" id="gender" name="gender">
          <option value="">Select
          <option
value="Male">Male</option>
          <option
value="Female">Female</option>
          <option
value="Other">Other</option>
        </select>
      </div>
      <div class="col-md-6">
        <label for="age" class="form-label
form-label-dark">Age (years)</label>
        <input type="number" class="form-
control form-control-dark" id="age" name="age"
placeholder="e.g. 30" min="5" max="120">
      </div>
    </div>
    <div class="row g-3 mb-4">
      <div class="col-md-6">
        <label for="height" class="form-label
form-label-dark">Height (cm)</label>
        <input type="number" step="0.1"
class="form-control form-control-dark" id="height"
name="height" placeholder="e.g. 175" min="50"
max="260">
      </div>
      <div class="col-md-6">
        <label for="weight" class="form-label
form-label-dark">Weight (kg)</label>
        <input type="number" step="0.1"
class="form-control form-control-dark" id="weight"
name="weight" placeholder="e.g. 70.0" min="20"
max="400">
      </div>
    </div>
    <div class="d-flex justify-content-end gap-
2">
      <a href="{{ url_for('admin.list_users')
}}" class="btn btn-outline-custom">Cancel</a>
      <button type="submit" class="btn btn-
emerald">
        <i class="bi bi-check2-circle me-
1"></i> Create Account
      </button>
    </div>

```

```

        </div>
    </form>
</div>
</div>
</div>
</div>
</div>
{% endblock %}
User_edit.html:
{% extends 'base.html' %}
{% block title %}Edit User #{{ user.id }} - Admin Portal{% endblock %}
{% block content %}
<div class="container py-4">
    <div class="row justify-content-center">
        <div class="col-lg-8">
            <div class="card card-dark shadow-lg">
                <div class="card-header card-header-dark d-flex justify-content-between align-items-center">
                    <span class="text-white fw-bold"><i class="bi bi-pencil-square text-cyan me-2"></i> Edit User: {{ user.name }}</span>
                    <a href="{{ url_for('admin.list_users') }}" class="btn btn-sm btn-outline-custom">Back to Users</a>
                </div>
                <div class="card-body p-4">
                    <form method="POST" action="{{ url_for('admin.edit_user', user_id=user.id) }}">
                        <div class="row g-3 mb-3">
                            <div class="col-md-6">
                                <label for="name" class="form-label form-label-dark">Full Name <span class="text-danger">*</span></label>
                                <input type="text" class="form-control form-control-dark" id="name" name="name" value="{{ user.name }}" required>
                            </div>
                            <div class="col-md-6">
                                <label for="email" class="form-label form-label-dark">Email Address <span class="text-danger">*</span></label>
                                <input type="email" class="form-control form-control-dark" id="email" name="email" value="{{ user.email }}" required>
                            </div>
                        </div>
                        <div class="row g-3 mb-3">
                            <div class="col-md-6">
                                <label for="role" class="form-label form-label-dark">Role Authorization <span class="text-danger">*</span></label>
                                <select class="form-select form-select-dark" id="role" name="role" required>
                                    <option value="user" {% if user.role == 'user' %}>User (Client)</option>
                                    <option value="trainer" {% if user.role == 'trainer' %}>Trainer</option>
                                    <option value="nutritionist" {% if user.role == 'nutritionist' %}>Nutritionist</option>
                                    <option value="admin" {% if user.role == 'admin' %}>Administrator</option>
                                </select>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

        <label for="new_password" class="form-label form-label-dark">Reset Password (Leave blank to keep current)</label>
        <input type="password" class="form-control form-control-dark" id="new_password" name="new_password" placeholder="New password (optional)">
    </div>
</div>
<div class="row g-3 mb-3">
    <div class="col-md-6">
        <label for="gender" class="form-label form-label-dark">Gender</label>
        <select class="form-select form-select-dark" id="gender" name="gender">
            <option value="" {% if not user.gender %}>Select Gender</option>
            <option value="Male" {% if user.gender == 'Male' %}>Male</option>
            <option value="Female" {% if user.gender == 'Female' %}>Female</option>
            <option value="Other" {% if user.gender == 'Other' %}>Other</option>
            <option value="Prefer not to say" {% if user.gender == 'Prefer not to say' %}>Prefer not to say</option>
        </select>
    </div>
    <div class="col-md-6">
        <label for="age" class="form-label form-label-dark">Age (years)</label>
        <input type="number" class="form-control form-control-dark" id="age" name="age" value="{{ user.age or '' }}" min="5" max="120">
    </div>
</div>
<div class="row g-3 mb-4">
    <div class="col-md-6">
        <label for="height" class="form-label form-label-dark">Height (cm)</label>
        <input type="number" step="0.1" class="form-control form-control-dark" id="height" name="height" value="{{ user.height or '' }}" min="50" max="260">
    </div>
    <div class="col-md-6">
        <label for="weight" class="form-label form-label-dark">Weight (kg)</label>
        <input type="number" step="0.1" class="form-control form-control-dark" id="weight" name="weight" value="{{ user.weight or '' }}" min="20" max="400">
    </div>
</div>
<div class="d-flex justify-content-between align-items-center">
    {% if user.id != current_user.id %}
        <button type="button" class="btn btn-outline-danger btn-delete-trigger" data-bs-toggle="modal" data-bs-target="#deleteConfirmModal" data-action="{{ url_for('admin.delete_user', user_id=user.id) }}" data-name="User {{ user.name }}">
            <i class="bi bi-trash me-1"></i> Delete User
        </button>
    </div>

```



```

        <td class="text-secondary
small">{{ u.created_at.strftime('%Y-%m-%d') }}</td>
        <td class="text-end">
            <div class="btn-group btn-group-
sm">
                <a href="{{
url_for('admin.edit_user', user_id=u.id) }}" class="btn
btn-outline-secondary" title="Edit User">
                    <i class="bi bi-pencil"></i>
                </a>
                {% if u.id != current_user.id
%}
                    <button type="button"
class="btn btn-outline-danger btn-delete-trigger" data-bs-
toggle="modal" data-bs-target="#deleteConfirmModal"
data-action="{{ url_for('admin.delete_user', user_id=u.id)
}}" data-name="User {{ u.name }} ({{ u.email }})">
                        <i class="bi bi-
trash"></i>
                    </button>
                    {% endif %}
                </div>
            </td>
        </tr>
        {% endfor %}
    </tbody>
</table>
</div>
{% else %}
    <div class="text-center py-5">
        <p class="text-secondary mb-0">No users
found matching query.</p>
    </div>
    {% endif %}
</div>
</div>
{% endblock %}

```

Weights.html:

```

{% extends 'base.html' %}
{% block title %}Weight Logs Management - Admin
Portal{% endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
    <!-- Header -->
    <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
        <div>
            <h2 class="text-white fw-bold mb-1"><i class="bi
bi-speedometer text-rose me-2"></i> Global Weight Logs
Management</h2>
            <p class="text-secondary small mb-0">Browse
and manage historical body weight measurements across
all users.</p>
        </div>
        <a href="{{ url_for('admin.dashboard') }}"
class="btn btn-outline-custom">
            <i class="bi bi-arrow-left me-1"></i> Admin
Dashboard
        </a>
    </div>
    <!-- Table -->
    <div class="card card-dark shadow-sm">
        <div class="card-body p-0">
            {% if weights %}
                <div class="table-responsive">
                    <table class="table table-dark-custom mb-0">
                        <thead>

```

```

                        <tr>
                            <th>ID</th>
                            <th>User</th>
                            <th>Date</th>
                            <th>Weight (kg)</th>
                            <th>Notes</th>
                            <th class="text-end">Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for w in weights %}
                            <tr>
                                <td class="text-secondary">#{{
w.id }}</td>
                                <td class="fw-bold text-white">{{
w.user.name }} ({{ w.user.email }})</td>
                                <td class="text-secondary">{{
w.date.strftime('%Y-%m-%d') }}</td>
                                <td class="fw-bold text-cyan">{{
w.weight }} kg</td>
                                <td class="text-secondary
small">{{ w.notes or '-' }}</td>
                                <td class="text-end">
                                    <button type="button" class="btn
btn-sm btn-outline-danger btn-delete-trigger" data-bs-
toggle="modal" data-bs-target="#deleteConfirmModal"
data-action="{{ url_for('admin.delete_weight',
record_id=w.id) }}" data-name="Weight Record #{{ w.id
}} ({{ w.weight }} kg)">
                                        <i class="bi bi-trash"></i>
                                    </button>
                                </td>
                            </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
            {% else %}
                <div class="text-center py-5">
                    <p class="text-secondary mb-0">No weight
records found.</p>
                </div>
            {% endif %}
        </div>
    </div>
{% endblock %}
Workouts.html:
{% extends 'base.html' %}
{% block title %}Workout Management - Admin
Portal{% endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
    <!-- Header -->
    <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
        <div>
            <h2 class="text-white fw-bold mb-1"><i class="bi
bi-activity text-cyan me-2"></i> Global Workout
Management</h2>
            <p class="text-secondary small mb-0">View and
audit all workout sessions recorded across all platform
users.</p>
        </div>
        <a href="{{ url_for('admin.dashboard') }}"
class="btn btn-outline-custom">
            <i class="bi bi-arrow-left me-1"></i> Admin
Dashboard
        </a>
    </div>

```

```

</div>
<!-- Filter Bar -->
<div class="card card-dark mb-4 p-3">
  <form method="GET" action="{{ {{ workout_type }}"
url_for('admin.list_workouts') }}" class="row g-2">
    <div class="col-md-9">
      <input type="text" class="form-control form-control-dark" name="type" value="{{ {{ workout_type }}"
placeholder="Filter by workout type (e.g. Running,
Strength Training)...">
    </div>
    <div class="col-md-3 d-flex gap-2">
      <button type="submit" class="btn btn-emerald
flex-grow-1">Filter</button>
      <a href="{{ {{ url_for('admin.list_workouts') }}"
class="btn btn-outline-custom">Reset</a>
    </div>
  </form>
</div>
<!-- Table -->
<div class="card card-dark shadow-sm">
  <div class="card-body p-0">
    {% if workouts %}
    <div class="table-responsive">
      <table class="table table-dark-custom mb-0">
        <thead>
          <tr>
            <th>ID</th>
            <th>User</th>
            <th>Date</th>
            <th>Type</th>
            <th>Exercise</th>
            <th>Duration</th>
            <th>Calories</th>
            <th>Notes</th>
            <th class="text-end">Actions</th>
          </tr>
        </thead>
        <tbody>
          {% for w in workouts %}
            <tr>
              <td class="text-secondary">#{{ {{
w.id }}</td>
              <td class="fw-bold text-white">{{ {{
w.user.name }} {{ {{ w.user.email }}</td>
              <td class="text-secondary">{{ {{
w.workout_date.strftime("%Y-%m-%d") }}</td>
              <td><span class="badge bg-secondary">{{ {{ w.workout_type }}</span></td>
              <td class="text-emerald">{{ {{
w.exercise_name }}</td>
              <td>{{ {{ w.duration }} min</td>
              <td class="text-warning">{{ {{
"{:,0f}".format(w.calories_burned) }} kcal</td>
              <td class="text-secondary
small">{{ {{ w.notes or '-' }}</td>
              <td class="text-end">
                <button type="button" class="btn
btn-sm btn-outline-danger btn-delete-trigger" data-bs-
toggle="modal" data-bs-target="#deleteConfirmModal"
data-action="{{ {{ url_for('admin.delete_workout',
workout_id=w.id) }}" data-name="Workout #{{ {{ w.id }}
{{ {{ w.exercise_name }} }}">
                  <i class="bi bi-trash"></i>
                </button>
              </td>
            </tr>
          {% endfor %}
        </tbody>

```

```

</table>
</div>
{% else %}
  <div class="text-center py-5">
    <p class="text-secondary mb-0">No workout
records found.</p>
  </div>
{% endif %}
</div>
</div>
{% endblock %}
Errors:
403.html:
{% extends 'base.html' %}
{% block title %}403 - Forbidden{% endblock %}
{% block content %}
<div class="container py-5 text-center">
  <div class="card card-dark p-5 mx-auto shadow-lg"
style="max-width: 550px;">
    <div class="stat-icon icon-rose mx-auto mb-3"
style="width: 70px; height: 70px; font-size: 2.2rem;">
      <i class="bi bi-shield-slash-fill"></i>
    </div>
    <h1 class="display-4 fw-bold text-white mb-
2">403</h1>
    <h4 class="text-white fw-semibold mb-3">Access
Forbidden</h4>
    <p class="text-secondary small mb-4">You do not
have the required role permissions to view or perform
actions on this resource.</p>
    <a href="{{ {{ url_for('dashboard.index') }}"
class="btn btn-emerald px-4">
      <i class="bi bi-house-door-fill me-1"></i> Return
to Safety
    </a>
  </div>
</div>
{% endblock %}
404.html:
{% extends 'base.html' %}
{% block title %}404 - Page Not Found{% endblock %}
{% block content %}
<div class="container py-5 text-center">
  <div class="card card-dark p-5 mx-auto shadow-lg"
style="max-width: 550px;">
    <div class="stat-icon icon-amber mx-auto mb-3"
style="width: 70px; height: 70px; font-size: 2.2rem;">
      <i class="bi bi-compass-fill"></i>
    </div>
    <h1 class="display-4 fw-bold text-white mb-
2">404</h1>
    <h4 class="text-white fw-semibold mb-3">Page Not
Found</h4>
    <p class="text-secondary small mb-4">The page or
fitness record you requested could not be located on the
server.</p>
    <a href="{{ {{ url_for('dashboard.index') }}"
class="btn btn-emerald px-4">
      <i class="bi bi-house-door-fill me-1"></i> Return
Home
    </a>
  </div>
</div>
{% endblock %}
500.html:
{% extends 'base.html' %}
{% block title %}500 - Internal Server Error{% endblock %}

```

```
{% block content %}
<div class="container py-5 text-center">
  <div class="card card-dark p-5 mx-auto shadow-lg"
  style="max-width: 550px;">
    <div class="stat-icon icon-rose mx-auto mb-3"
    style="width: 70px; height: 70px; font-size: 2.2rem;">
      <i class="bi bi-cpu-fill"></i>
    </div>
    <h1 class="display-4 fw-bold text-white mb-
    2">500</h1>
    <h4 class="text-white fw-semibold mb-3">Internal
    System Error</h4>
    <p class="text-secondary small mb-4">An
    unexpected error occurred while processing your request.
    The transaction has been rolled back safely.</p>
    <a href="{{ url_for('dashboard.index') }}"
    class="btn btn-emerald px-4">
      <i class="bi bi-house-door-fill me-1"></i> Return
    Home
    </a>
  </div>
</div>
{% endblock %}
Goals:
Add.html:
{% extends 'base.html' %}
{% block title %}Add Goal - FitPulse{% endblock %}
{% block content %}
<div class="container py-4">
  <div class="row justify-content-center">
    <div class="col-lg-8">
      <div class="card card-dark shadow-lg">
        <div class="card-header card-header-dark d-flex
        justify-content-between align-items-center">
          <span class="text-white fw-bold"><i
          class="bi bi-trophy-fill text-warning me-2"></i> Set New
          Fitness Goal</span>
          <a href="{{ url_for('goal.view_goals') }}"
          class="btn btn-sm btn-outline-custom">View Goals</a>
        </div>
        <div class="card-body p-4">
          <form method="POST" action="{{
          url_for('goal.add') }}">
            <div class="row g-3 mb-3">
              <div class="col-md-6">
                <label for="goal_type" class="form-
                label form-label-dark">Goal Type <span class="text-
                danger">*</span></label>
                <select class="form-select form-
                select-dark" id="goal_type" name="goal_type" required>
                  <option value="">Select Goal
                  Type</option>
                  {% for gt in goal_types %}
                    <option value="{{ gt }}" {% if
                    goal_type == gt %}selected{% endif %}>{{ gt
                    }}</option>
                  {% endfor %}
                </select>
              </div>
              <div class="col-md-6">
                <label for="target_value"
                class="form-label form-label-dark">Target Metric <span
                class="text-danger">*</span></label>
                <input type="text" class="form-
                control form-control-dark" id="target_value"
                name="target_value" value="{{ target_value or " }}"
                placeholder="e.g. 70 kg, 10 km, 4 days/week" required>
              </div>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

```
<div class="row g-3 mb-3">
  <div class="col-md-6">
    <label for="current_value"
    class="form-label form-label-dark">Current
    Baseline</label>
    <input type="text" class="form-
    control form-control-dark" id="current_value"
    name="current_value" value="{{ current_value or " }}"
    placeholder="e.g. 76 kg, 3 km">
  </div>
  <div class="col-md-3">
    <label for="start_date" class="form-
    label form-label-dark">Start Date</label>
    <input type="date" class="form-
    control form-control-dark" id="start_date"
    name="start_date" value="{{ start_date or today_date
    }}">
  </div>
  <div class="col-md-3">
    <label for="target_date" class="form-
    label form-label-dark">Target Date <span class="text-
    danger">*</span></label>
    <input type="date" class="form-
    control form-control-dark" id="target_date"
    name="target_date" value="{{ target_date or " }}"
    required>
  </div>
</div>
<div class="mb-4">
  <label for="description" class="form-
  label form-label-dark">Description & Plan</label>
  <textarea class="form-control form-
  control-dark" id="description" name="description"
  rows="3" placeholder="Why is this goal important? What
  is your action plan...">{{ description or " }}"</textarea>
</div>
<div class="d-flex justify-content-end gap-
2">
  <a href="{{ url_for('goal.view_goals')
  }}" class="btn btn-outline-custom">Cancel</a>
  <button type="submit" class="btn btn-
  emerald">
    <i class="bi bi-check2-circle me-
    1"></i> Save Goal
  </button>
</div>
</form>
</div>
</div>
</div>
</div>
</div>
{% endblock %}
Edit.html:
{% extends 'base.html' %}
{% block title %}Edit Goal - FitPulse{% endblock %}
{% block content %}
<div class="container py-4">
  <div class="row justify-content-center">
    <div class="col-lg-8">
      <div class="card card-dark shadow-lg">
        <div class="card-header card-header-dark d-flex
        justify-content-between align-items-center">
          <span class="text-white fw-bold"><i
          class="bi bi-pencil-square text-cyan me-2"></i> Edit
          Fitness Goal</span>
          <a href="{{ url_for('goal.view_goals') }}"
          class="btn btn-sm btn-outline-custom">Back to
          Goals</a>
        </div>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

```

</div>
<div class="card-body p-4">
  <form method="POST" action="{{
url_for('goal.edit', goal_id=goal.id) }}">
    <div class="row g-3 mb-3">
      <div class="col-md-6">
        <label for="goal_type" class="form-label form-label-dark">Goal Type <span class="text-danger">*</span></label>
        <select class="form-select form-select-dark" id="goal_type" name="goal_type" required>
          {% for gt in goal_types %}
            <option value="{{ gt }}" {{ % if goal.goal_type == gt %}}selected{% endif %}}>{{ gt }}</option>
          {% endfor %}
        </select>
      </div>
      <div class="col-md-6">
        <label for="target_value" class="form-label form-label-dark">Target Value <span class="text-danger">*</span></label>
        <input type="text" class="form-control form-control-dark" id="target_value" name="target_value" value="{{ goal.target_value }}" required>
      </div>
    </div>
    <div class="row g-3 mb-3">
      <div class="col-md-6">
        <label for="current_value" class="form-label form-label-dark">Current Value</label>
        <input type="text" class="form-control form-control-dark" id="current_value" name="current_value" value="{{ goal.current_value or '' }}">
      </div>
      <div class="col-md-6">
        <label for="status" class="form-label form-label-dark">Status</label>
        <select class="form-select form-select-dark" id="status" name="status">
          <option value="ACTIVE" {{ % if goal.status == 'ACTIVE' %}}selected{% endif %}}>ACTIVE</option>
          <option value="COMPLETED" {{ % if goal.status == 'COMPLETED' %}}selected{% endif %}}>COMPLETED</option>
          <option value="EXPIRED" {{ % if goal.status == 'EXPIRED' %}}selected{% endif %}}>EXPIRED</option>
        </select>
      </div>
    </div>
    <div class="row g-3 mb-3">
      <div class="col-md-6">
        <label for="start_date" class="form-label form-label-dark">Start Date</label>
        <input type="date" class="form-control form-control-dark" id="start_date" name="start_date" value="{{ goal.start_date.strftime('%Y-%m-%d') }}">
      </div>
      <div class="col-md-6">
        <label for="target_date" class="form-label form-label-dark">Target Date <span class="text-danger">*</span></label>

```

```

        <input type="date" class="form-control form-control-dark" id="target_date" name="target_date" value="{{ goal.target_date.strftime('%Y-%m-%d') }}" required>
      </div>
    </div>
    <div class="mb-4">
      <label for="description" class="form-label form-label-dark">Description</label>
      <textarea class="form-control form-control-dark" id="description" name="description" rows="3">{{ goal.description or '' }}</textarea>
    </div>
    <div class="d-flex justify-content-between align-items-center">
      <button type="button" class="btn btn-outline-danger btn-delete-trigger" data-bs-toggle="modal" data-bs-target="#deleteConfirmModal" data-action="{{ url_for('goal.delete', goal_id=goal.id) }}" data-name="{{ goal.goal_type }}">
        <i class="bi bi-trash me-1"></i>
        Delete
      </button>
      <div class="d-flex gap-2">
        <a href="{{ url_for('goal.view_goals') }}" class="btn btn-outline-custom">Cancel</a>
        <button type="submit" class="btn btn-emerald">
          <i class="bi bi-floppy me-1"></i>
          Update Goal
        </button>
      </div>
    </div>
  </form>
</div>
{% endblock %}
View.html:
{% extends 'base.html' %}
{% block title %}Fitness Goals - FitPulse{% endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Header & Action Button -->
  <div class="d-flex flex-column flex-md-row justify-content-between align-items-md-center gap-3 mb-4">
    <div>
      <h2 class="text-white fw-bold mb-1"><i class="bi bi-trophy-fill text-warning me-2"></i> Fitness Goals</h2>
      <p class="text-secondary small mb-0">Track, manage, and celebrate your fitness milestones.</p>
    </div>
    <a href="{{ url_for('goal.add') }}" class="btn btn-emerald">
      <i class="bi bi-plus-lg me-1"></i> Set New Goal
    </a>
  </div>
  <!-- Summary Metrics & Filter Tabs -->
  <div class="row g-3 mb-4">
    <div class="col-sm-4">
      <div class="stat-card">
        <div class="stat-label">Total Goals Set</div>
        <div class="stat-value">{{ total_goals }}</div>
      </div>
    </div>
    <div class="col-sm-4">

```

```

<div class="stat-card stat-amber">
  <div class="stat-label">In Progress /
Active</div>
  <div class="stat-value">{{ active_count
}}</div>
</div>
</div>
<div class="col-sm-4">
  <div class="stat-card stat-violet">
    <div class="stat-label">Completed Goals</div>
    <div class="stat-value">{{ completed_count }}
<span class="fs-6 text-emerald"><i class="bi bi-check-
circle-fill"></i></span></div>
  </div>
</div>
<!-- Status Tabs -->
<div class="d-flex gap-2 mb-4">
  <a href="{{ url_for('goal.view_goals', status='ALL')
}}" class="btn btn-sm {% if current_status == 'ALL'
%}btn-emerald{% else %}btn-outline-custom{% endif
%}">
    All Goals ({{ total_goals }})
  </a>
  <a href="{{ url_for('goal.view_goals',
status='ACTIVE') }}" class="btn btn-sm {% if
current_status == 'ACTIVE' %}btn-emerald{% else
%}btn-outline-custom{% endif %}">
    Active ({{ active_count }})
  </a>
  <a href="{{ url_for('goal.view_goals',
status='COMPLETED') }}" class="btn btn-sm {% if
current_status == 'COMPLETED' %}btn-emerald{% else
%}btn-outline-custom{% endif %}">
    Completed ({{ completed_count }})
  </a>
</div>
<!-- Goals Cards Grid -->
{% if goals %}
  <div class="row g-4">
    {% for g in goals %}
      <div class="col-md-6 col-lg-4">
        <div class="card card-dark h-100 p-3 d-flex
flex-column border-top border-3 {% if g.status ==
'COMPLETED' %}border-success{% elif g.is_expired
%}border-danger{% else %}border-primary{% endif
%}">
          <div class="d-flex justify-content-between
align-items-start mb-2">
            <h5 class="text-white fw-bold mb-0">{{
g.goal_type }}</h5>
            <span class="badge bg-{{ g.badge_color
}}">{{ g.status }}</span>
          </div>
          <div class="my-2 p-2 rounded"
style="background: rgba(15, 23, 42, 0.6);">
            <div class="d-flex justify-content-
between small text-secondary">
              <span>Target: <strong class="text-
white">{{ g.target_value }}</strong></span>
              {% if g.current_value %}
                <span>Current: <strong
class="text-cyan">{{ g.current_value }}</strong></span>
              {% endif %}
            </div>
          </div>
          <p class="text-secondary small flex-grow-
1 mb-3">

```

```

        {{ g.description or 'No additional
description provided.' }}
      </p>
      <div class="d-flex justify-content-between
align-items-center text-muted small border-top pt-2"
style="border-color: var(--border-color) !important;">
        <span><i class="bi bi-calendar-event
me-1"></i> Target: {{ g.target_date.strftime("%b %d,
%Y") }}</span>
        {% if g.is_expired %}
          <span class="text-danger"><i
class="bi bi-exclamation-octagon me-1"></i>
Expired</span>
        {% endif %}
      </div>
      <div class="d-flex justify-content-between
align-items-center mt-3 pt-2">
        <form method="POST" action="{{ {
url_for('goal.toggle_status', goal_id=g.id) }}">
          {% if g.status == 'ACTIVE' %}
            <button type="submit" class="btn
btn-sm btn-outline-success">
              <i class="bi bi-check-lg me-
1"></i> Mark Done
            </button>
          {% else %}
            <button type="submit" class="btn
btn-sm btn-outline-secondary">
              <i class="bi bi-arrow-
counterclockwise me-1"></i> Reactivate
            </button>
          {% endif %}
        </form>
        <div class="btn-group btn-group-sm">
          <a href="{{ { url_for('goal.edit',
goal_id=g.id) }}" class="btn btn-outline-secondary"
title="Edit">
            <i class="bi bi-pencil"></i>
          </a>
          <button type="button" class="btn btn-
outline-danger btn-delete-trigger" data-bs-toggle="modal"
data-bs-target="#deleteConfirmModal" data-action="{{ {
url_for('goal.delete', goal_id=g.id) }}" data-name="{{ {
g.goal_type }}">
            <i class="bi bi-trash"></i>
          </button>
        </div>
      </div>
    </div>
  </div>
  <div>
    {% endfor %}
  </div>
  {% else %}
    <div class="card card-dark text-center py-5">
      <i class="bi bi-trophy fs-1 text-muted d-block mb-
2"></i>
      <h5 class="text-white">No goals found in this
category</h5>
      <p class="text-secondary small mb-3">Stay
motivated by defining targeted fitness goals.</p>
      <a href="{{ { url_for('goal.add') }}" class="btn btn-
emerald btn-sm mx-auto">Set a Fitness Goal</a>
    </div>
  {% endif %}
</div>
{% endblock %}
Nutritionist :
Dashboard.html:
{% extends 'base.html' %}

```

```
{% block title %}Nutritionist Dashboard - FitPulse{%
endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Header -->
  <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
    <div>
      <h2 class="text-white fw-bold mb-1">
        Nutritionist Portal <span class="badge bg-
warning text-dark fs-6">Nutritionist</span>
      </h2>
      <p class="text-secondary small mb-0">Evaluate
client BMI classifications, weight trends, and provide
clinical dietary guidance.</p>
    </div>
    <a href="{{ url_for('nutritionist.list_users') }}"
class="btn btn-emerald">
      <i class="bi bi-people-fill me-1"></i> Browse
Client Health Profiles
    </a>
  </div>
  <!-- Summary Metrics: BMI Distribution -->
  <div class="row g-3 mb-4">
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card">
        <div class="stat-label">Total Clients</div>
        <div class="stat-value">{{ total_clients
}}</div>
        <small class="text-secondary">Assigned
roster</small>
      </div>
    </div>
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card">
        <div class="stat-label">Normal Weight</div>
        <div class="stat-value text-success">{{
bmi_dist['Normal weight'] }}</div>
        <small class="text-success"><i class="bi bi-
check-circle me-1"></i> Optimal BMI</small>
      </div>
    </div>
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card stat-amber">
        <div class="stat-label">Overweight</div>
        <div class="stat-value text-warning">{{
bmi_dist['Overweight'] }}</div>
        <small class="text-warning"><i class="bi bi-
exclamation-circle me-1"></i> Weight management
candidates</small>
      </div>
    </div>
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card stat-rose">
        <div class="stat-label">Obesity /
Underweight</div>
        <div class="stat-value text-danger">{{
bmi_dist['Obesity'] + bmi_dist['Underweight'] }}</div>
        <small class="text-danger"><i class="bi bi-
heart-pulse me-1"></i> Priority dietary attention</small>
      </div>
    </div>
  </div>
  <!-- Main Grid -->
  <div class="row g-4">
    <!-- Client Health Profiles -->
    <div class="col-lg-8">
      <div class="card card-dark mb-4">
```

```

      <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
        <span class="text-white fw-bold"><i>
class="bi bi-person-lines-fill text-warning me-2"></i>
Client Body Metrics</span>
        <a href="{{ url_for('nutritionist.list_users')
}}" class="btn btn-sm btn-outline-custom">View All</a>
      </div>
      <div class="card-body p-0">
        {% if clients %}
        <div class="table-responsive">
          <table class="table table-dark-custom
mb-0">
            <thead>
              <tr>
                <th>Client</th>
                <th>Height</th>
                <th>Weight</th>
                <th>BMI</th>
                <th>Category</th>
                <th class="text-
end">Action</th>
              </tr>
            </thead>
            <tbody>
              {% for c in clients %}
                <tr>
                  <td class="fw-bold text-
white">{{ c.name }}</td>
                  <td class="text-secondary">{{
c.height or '--' }} cm</td>
                  <td class="text-secondary">{{
c.weight or '--' }} kg</td>
                  <td class="fw-bold text-
cyan">{{ c.bmi or '--' }}</td>
                  <td>
                    <span class="badge bg-{{
c.bmi_badge_color }}">{{ c.bmi_category }}</span>
                  </td>
                  <td class="text-end">
                    <a href="{{
url_for('nutritionist.user_detail', user_id=c.id) }}"
class="btn btn-sm btn-emerald py-1 px-3">
                      Diet Plan <i class="bi bi-
arrow-right ms-1"></i>
                    </a>
                  </td>
                </tr>
              {% endfor %}
            </tbody>
          </table>
        </div>
        {% else %}
          <div class="text-center py-5">
            <p class="text-secondary mb-0">No
client profiles available.</p>
          </div>
        {% endif %}
      </div>
    </div>
    <!-- Recent Nutrition Notes Given -->
    <div class="col-lg-4">
      <div class="card card-dark">
        <div class="card-header card-header-dark">
          <span class="text-white fw-bold"><i>
class="bi bi-journal-text text-emerald me-2"></i> Recent
Dietary Notes</span>
        </div>
```

```

<div class="card-body">
  {% if my_notes %}
    <div class="d-flex flex-column gap-3">
      {% for n in my_notes %}
        <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
          <div class="d-flex justify-content-
between align-items-center mb-1">
            <strong class="text-white
small">{{ n.client.name }}</strong>
            <small class="text-muted">{{
n.created_at.strftime('%b %d') }}</small>
          </div>
          <h6 class="text-warning fw-bold
mb-1 fs-6">{{ n.title }}</h6>
          <p class="text-secondary small mb-
0">{{ n.content }}</p>
        </div>
      {% endfor %}
    </div>
  {% else %}
    <div class="text-center py-4">
      <p class="text-secondary small mb-
0">No dietary recommendations sent yet.</p>
    </div>
  {% endif %}
</div>
</div>
</div>
</div>
{% endblock %}
User_detail.html:
{% extends 'base.html' %}
{% block title %}Nutrition Profile: {{ client.name }} -
Nutritionist Portal{% endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Client Header -->
  <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
    <div>
      <a href="{{ url_for('nutritionist.list_users') }}"
class="text-secondary small text-decoration-none mb-1 d-
inline-block">
        <i class="bi bi-arrow-left me-1"></i> Back to
Client List
      </a>
      <h2 class="text-white fw-bold mb-1">{{
client.name }}'s Nutritional & Weight Profile</h2>
      <div class="d-flex gap-2 align-items-center flex-
wrap">
        <span class="text-secondary small"><i
class="bi bi-envelope me-1"></i> {{ client.email
}}</span>
        <span class="text-muted">&bull;</span>
        <span class="text-secondary small">Age: {{
client.age or 'N/A' }}</span>
        <span class="text-muted">&bull;</span>
        <span class="text-secondary small">Height: {{
client.height or 'N/A' }} cm</span>
        <span class="text-muted">&bull;</span>
        <span class="text-secondary small">Weight: {{
client.weight or 'N/A' }} kg</span>
        {% if client.bmi %}
          <span class="badge bg-{{
client.bmi_badge_color }}">BMI: {{ client.bmi }} ({{
client.bmi_category }})</span>
        </div>
      </div>
    </div>
  </div>

```

```

    {% endif %}
  </div>
</div>
<!-- Summary Metrics -->
<div class="row g-3 mb-4">
  <div class="col-sm-4">
    <div class="stat-card">
      <div class="stat-label">Current Weight</div>
      <div class="stat-value">{{ current_weight or '--'
}} <span class="fs-6 text-muted">kg</span></div>
    </div>
  </div>
  <div class="col-sm-4">
    <div class="stat-card stat-violet">
      <div class="stat-label">Initial Weight
Recorded</div>
      <div class="stat-value">{{ initial_weight or '--'
}} <span class="fs-6 text-muted">kg</span></div>
    </div>
  </div>
  <div class="col-sm-4">
    <div class="stat-card stat-amber">
      <div class="stat-label">Total Weight
Trajectory</div>
      <div class="stat-value">
        {% if weight_change > 0 %}
          <span class="text-warning">+{{
weight_change }} kg</span>
        {% elif weight_change < 0 %}
          <span class="text-success">{{
weight_change }} kg</span>
        {% else %}
          <span class="text-muted">0.0 kg</span>
        {% endif %}
      </div>
    </div>
  </div>
</div>
<!-- Main Grid -->
<div class="row g-4">
  <!-- Left Column: Weight History & Dietary Goals --
>
  <div class="col-lg-8">
    <!-- Weight History Table -->
    <div class="card card-dark mb-4">
      <div class="card-header card-header-dark">
        <span class="text-white fw-bold"><i
class="bi bi-journal-medical text-info me-2"></i> Client
Weight History & Computed BMI</span>
      </div>
      <div class="card-body p-0">
        {% if weight_records %}
          <div class="table-responsive">
            <table class="table table-dark-custom
mb-0">
              <thead>
                <tr>
                  <th>Date</th>
                  <th>Weight (kg)</th>
                  <th>Calculated BMI</th>
                  <th>Notes</th>
                </tr>
              </thead>
              <tbody>
                {% for wr in weight_records %}
                  <tr>
                    <td class="text-secondary
small">{{ wr.date.strftime('%Y-%m-%d') }}</td>

```

```

        <td class="fw-bold text-
white">{{ wr.weight }} kg</td>
        <td>
            <span class="badge bg-
secondary">{{ wr.bmi_for_height(client.height) or '--'
}}</span>
        </td>
        <td class="text-secondary
small">{{ wr.notes or '-' }}</td>
    </tr>
    <tbody>
        <tr>
            <td colspan="3">
                <div class="text-center py-4">
                    <p class="text-secondary small mb-
0">No weight history records logged by this client.</p>
                </div>
            </td>
        </tr>
    </tbody>
</table>
{% else %}
    <div class="text-center py-4">
        <p class="text-secondary small mb-
0">No weight history records logged by this client.</p>
    </div>
{% endif %}
</div>
<!-- Client Goals -->
<div class="card card-dark">
    <div class="card-header card-header-dark">
        <span class="text-white fw-bold"><i>Client
Goals</i></span>
    </div>
    <div class="card-body">
        {% if goals %}
            <div class="row g-3">
                {% for g in goals %}
                    <div class="col-md-6">
                        <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
                            <div class="d-flex justify-
content-between align-items-center mb-1">
                                <strong class="text-white">{{
g.goal_type }}</strong>
                                <span class="badge bg-{{
g.badge_color }}">{{ g.status }}</span>
                            </div>
                            <small class="text-cyan d-block
mb-1">Target: {{ g.target_value }}</small>
                            <p class="text-secondary small
mb-1">{{ g.description or 'No notes.' }}</p>
                            <small class="text-
muted">Target Date: {{ g.target_date.strftime('%b %d,
%Y') }}</small>
                        </div>
                    </div>
                </div>
            </div>
        {% else %}
            <p class="text-secondary small mb-
0">Client has no active goals.</p>
        {% endif %}
    </div>
</div>
</div>
<!-- Right Column: Give Nutrition Recommendation
Form & Existing Recommendations -->
<div class="col-lg-4">
    <!-- Add Recommendation Form -->
    <div class="card card-dark mb-4">
        <div class="card-header card-header-dark">

```

```

            <span class="text-white fw-bold"><i>Prescribe
Nutrition Guidance</i></span>
        </div>
        <div class="card-body">
            <form method="POST" action="{{
url_for('nutritionist.add_recommendation',
user_id=client.id) }}">
                <div class="mb-3">
                    <label for="title" class="form-label
form-label-dark">Guidance Title <span class="text-
danger">*</span></label>
                    <input type="text" class="form-control
form-control-dark" id="title" name="title"
placeholder="e.g. High Protein Deficit Plan" required>
                </div>
                <div class="mb-3">
                    <label for="content" class="form-label
form-label-dark">Nutritional Advice & Meal
Guidance <span class="text-danger">*</span></label>
                    <textarea class="form-control form-
control-dark" id="content" name="content" rows="4"
placeholder="Detail macronutrient targets, hydration
recommendations, or meal structure..."
required></textarea>
                </div>
                <button type="submit" class="btn btn-
emerald w-100">
                    <i class="bi bi-send-fill me-1"></i>
Send Dietary Advice
                </button>
            </form>
        </div>
    </div>
    <!-- Existing Guidance Notes -->
    <div class="card card-dark">
        <div class="card-header card-header-dark">
            <span class="text-white fw-bold"><i>
class="bi bi-chat-quote-fill text-warning me-2"></i>
Dietary Notes ({{ recommendations|length }})</span>
        </div>
        <div class="card-body">
            {% if recommendations %}
                <div class="d-flex flex-column gap-3">
                    {% for r in recommendations %}
                        <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
                            <div class="d-flex justify-content-
between align-items-center mb-1">
                                <h6 class="text-warning fw-bold
mb-0 fs-6">{{ r.title }}</h6>
                                <small class="text-muted">{{
r.created_at.strftime('%b %d') }}</small>
                            </div>
                            <p class="text-secondary small mb-
2">{{ r.content }}</p>
                            <div class="d-flex justify-content-
between align-items-center">
                                <small class="text-muted">By:
{{ r.author.name }}</small>
                                <form method="POST"
action="{{ url_for('nutritionist.delete_recommendation',
rec_id=r.id) }}">
                                    <button type="submit"
class="btn btn-sm btn-outline-danger py-0 px-2"
title="Delete Note">
                                        <i class="bi bi-trash"></i>
                                    </button>
                                </div>
                        </div>
                    </div>
            </div>
        </div>
    </div>

```

```

        </form>
      </div>
    </div>
    {% endfor %}
  </div>
  {% else %}
    <p class="text-secondary small mb-0">No
    dietary guidance notes recorded for this client.</p>
  {% endif %}
</div>
</div>
</div>
</div>
</div>
{% endblock %}
Users.html:
{% extends 'base.html' %}
{% block title %}Client Directory - Nutritionist Portal{%
endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <div class="d-flex flex-column flex-md-row justify-
  content-between align-items-md-center gap-3 mb-4">
    <div>
      <h2 class="text-white fw-bold mb-1"><i class="bi
      bi-people text-warning me-2"></i> Client Nutrition
      & BMI Roster</h2>
      <p class="text-secondary small mb-0">Select a
      client to evaluate weight logs, analyze BMI progression,
      and provide nutritional strategies.</p>
    </div>
    <a href="{{ url_for('nutritionist.dashboard') }}"
    class="btn btn-outline-custom">
      <i class="bi bi-arrow-left me-1"></i> Dashboard
    </a>
  </div>
  <!-- Search Toolbar -->
  <div class="card card-dark mb-4 p-3">
    <form method="GET" action="{{
    url_for('nutritionist.list_users') }}" class="row g-2">
      <div class="col-md-9">
        <div class="input-group">
          <span class="input-group-text bg-dark
          border-secondary text-secondary"><i class="bi bi-
          search"></i></span>
          <input type="text" class="form-control form-
          control-dark" name="search" value="{{ search }}"
          placeholder="Search client by name or email...">
        </div>
      </div>
      <div class="col-md-3 d-flex gap-2">
        <button type="submit" class="btn btn-emerald
        flex-grow-1">Search</button>
        <a href="{{ url_for('nutritionist.list_users') }}"
        class="btn btn-outline-custom">Reset</a>
      </div>
    </form>
  </div>
  <!-- Clients Grid -->
  {% if clients %}
    <div class="row g-4">
      {% for c in clients %}
        <div class="col-md-6 col-lg-4">
          <div class="card card-dark h-100 p-4 card-
          dark-hover d-flex flex-column">
            <div class="d-flex align-items-center gap-3
            mb-3">

```

```

            <div class="stat-icon"
            style="background: rgba(245, 158, 11, 0.15); color: var(--
            accent-amber);">
              <i class="bi bi-person-fill fs-3"></i>
            </div>
            <div>
              <h5 class="text-white fw-bold mb-
              0">{{ c.name }}</h5>
              <small class="text-secondary">{{
              c.email }}</small>
            </div>
          </div>
        </div>
      </div>
      <div class="p-3 rounded-3 mb-3 flex-
      grow-1" style="background: rgba(15, 23, 42, 0.6); border:
      1px solid var(--border-color);">
        <div class="d-flex justify-content-
        between align-items-center mb-2">
          <span class="text-muted
          small">Current BMI</span>
          <span class="badge bg-{{
          c.bmi_badge_color }}">{{ c.bmi or '--' }} ({{
          c.bmi_category }})</span>
        </div>
        <div class="row g-2 text-center border-
        top pt-2" style="border-color: var(--border-color)
        !important;">
          <div class="col-6">
            <small class="text-muted d-
            block">Weight</small>
            <strong class="text-white">{{
            c.weight or '--' }} kg</strong>
          </div>
          <div class="col-6">
            <small class="text-muted d-
            block">Height</small>
            <strong class="text-white">{{
            c.height or '--' }} cm</strong>
          </div>
        </div>
        <a href="{{
        url_for('nutritionist.user_detail', user_id=c.id) }}"
        class="btn btn-emerald w-100">
          <i class="bi bi-egg-fried me-1"></i>
          View Nutrition & Diet
        </a>
      </div>
    </div>
    {% endfor %}
  </div>
  {% else %}
    <div class="card card-dark text-center py-5">
      <p class="text-secondary mb-0">No client records
      found matching search.</p>
    </div>
  {% endif %}
</div>
{% endblock %}
Trainer:
Dashboard.html:
{% extends 'base.html' %}
{% block title %}Trainer Dashboard - FitPulse{%
endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Header -->
  <div class="d-flex flex-column flex-md-row justify-
  content-between align-items-md-center gap-3 mb-4">
    <div>

```

```

<h2 class="text-white fw-bold mb-1">
  Trainer Portal <span class="badge bg-primary
fs-6">Trainer</span>
</h2>
<p class="text-secondary small mb-0">Monitor
assigned clients, inspect workout logs, and provide expert
training guidance.</p>
</div>
<a href="{{ url_for('trainer.list_users') }}"
class="btn btn-emerald">
  <i class="bi bi-people-fill me-1"></i> Browse All
Clients
</a>
</div>
<!-- Summary Metrics -->
<div class="row g-3 mb-4">
  <div class="col-sm-4">
    <div class="stat-card">
      <div class="stat-label">Active Clients</div>
      <div class="stat-value">{{ total_clients
}}</div>
    </div>
  </div>
  <div class="col-sm-4">
    <div class="stat-card stat-violet">
      <div class="stat-label">Client Workouts
Logged</div>
      <div class="stat-value">{{
total_client_workouts }}</div>
    </div>
  </div>
  <div class="col-sm-4">
    <div class="stat-card stat-amber">
      <div class="stat-label">Active Client
Goals</div>
      <div class="stat-value">{{ active_client_goals
}}</div>
    </div>
  </div>
</div>
<!-- Main Grid -->
<div class="row g-4">
  <!-- Clients List Table -->
  <div class="col-lg-8">
    <div class="card card-dark mb-4">
      <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
        <span class="text-white fw-bold"><i
class="bi bi-people text-cyan me-2"></i> Client
Roster</span>
        <a href="{{ url_for('trainer.list_users') }}"
class="btn btn-sm btn-outline-custom">View All</a>
      </div>
      <div class="card-body p-0">
        {% if clients %}
          <table class="table-responsive">
            <table class="table table-dark custom
mb-0">
              <thead>
                <tr>
                  <th>Name</th>
                  <th>Email</th>
                  <th>Workouts</th>
                  <th>BMI</th>
                  <th class="text-
end">Action</th>
                </tr>
              </thead>
              <tbody>

```

```

        {% for c in clients %}
          <tr>
            <td class="fw-bold text-
white">{{ c.name }}</td>
            <td class="text-secondary
small">{{ c.email }}</td>
            <td><span class="badge bg-
secondary">{{ c.workouts|length }}
sessions</span></td>
            <td>
              {% if c.bmi %}
                <span class="badge bg-{{
c.bmi_badge_color }}">{{ c.bmi }} ({{ c.bmi_category
}})</span>
              {% else %}
                <span class="text-muted
small">Not recorded</span>
              {% endif %}
            </td>
            <td class="text-end">
              <a href="{{
url_for('trainer.user_detail', user_id=c.id) }}" class="btn
btn-sm btn-emerald py-1 px-3">
                Inspect & Guide <i
class="bi bi-arrow-right ms-1"></i>
              </a>
            </td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
  {% else %}
    <div class="text-center py-5">
      <p class="text-secondary mb-0">No
registered client users found.</p>
    </div>
  {% endif %}
</div>
</div>
<!-- Recent Guidance Notes Given -->
<div class="col-lg-4">
  <div class="card card-dark">
    <div class="card-header card-header-dark">
      <span class="text-white fw-bold"><i
class="bi bi-chat-left-text text-emerald me-2"></i>
Recent Guidance Prescribed</span>
    </div>
    <div class="card-body">
      {% if my_notes %}
        <div class="d-flex flex-column gap-3">
          {% for n in my_notes %}
            <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
              <div class="d-flex justify-content-
between align-items-center mb-1">
                <strong class="text-white
small">{{ n.client.name }}</strong>
                <small class="text-muted">{{
n.created_at.strftime('%b %d') }}</small>
              </div>
              <h6 class="text-emerald fw-bold
mb-1 fs-6">{{ n.title }}</h6>
              <p class="text-secondary small mb-
0">{{ n.content }}</p>
            </div>
          {% endfor %}
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
    {% else %}
    <div class="text-center py-4">
      <p class="text-secondary small mb-0">You have not prescribed any workout notes yet.</p>
    </div>
    {% endif %}
  </div>
</div>
</div>
</div>
</div>
</div>
{% endblock %}
User_detail.html:
{% extends 'base.html' %}
{% block title %}Client: {{ client.name }} - Trainer Portal{% endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Client Header -->
  <div class="d-flex flex-column flex-md-row justify-content-between align-items-md-center gap-3 mb-4">
    <div>
      <a href="{{ url_for('trainer.list_users') }}"
        class="text-secondary small text-decoration-none mb-1 d-inline-block">
        <i class="bi bi-arrow-left me-1"></i> Back to Client Directory
      </a>
      <h2 class="text-white fw-bold mb-1">{{ client.name }}'s Fitness Profile</h2>
      <div class="d-flex gap-2 align-items-center flex-wrap">
        <span class="text-secondary small"><i>{{ client.email }}</i>
        <span class="text-muted">&bull;</span>
        <span class="text-secondary small">Age: {{ client.age or 'N/A' }}</span>
        <span class="text-muted">&bull;</span>
        <span class="text-secondary small">Height: {{ client.height or 'N/A' }} cm</span>
        <span class="text-muted">&bull;</span>
        <span class="text-secondary small">Weight: {{ client.weight or 'N/A' }} kg</span>
        {% if client.bmi %}
          <span class="badge bg-{{ client.bmi_badge_color }}">BMI: {{ client.bmi }} ({{ client.bmi_category }})</span>
        {% endif %}
      </div>
    </div>
  </div>
  <!-- Metrics Row -->
  <div class="row g-3 mb-4">
    <div class="col-sm-4">
      <div class="stat-card">
        <div class="stat-label">Logged Workouts</div>
        <div class="stat-value">{{ workouts|length }}</div>
      </div>
    </div>
    <div class="col-sm-4">
      <div class="stat-card stat-violet">
        <div class="stat-label">Total Training Time</div>
        <div class="stat-value">{{ total_minutes }}</div>
      </div>
    </div>
  </div>
</div>
</div>
</div>

```

```

    </div>
  </div>
  <div class="col-sm-4">
    <div class="stat-card stat-amber">
      <div class="stat-label">Total Calories Burned</div>
      <div class="stat-value">{{ "{:,.0f}".format(total_calories) }}</div>
    </div>
  </div>
</div>
<!-- Main Grid -->
<div class="row g-4">
  <!-- Left Column: Workouts & Goals -->
  <div class="col-lg-8">
    <!-- Workouts Table -->
    <div class="card card-dark mb-4">
      <div class="card-header card-header-dark">
        <span class="text-white fw-bold"><i>Client Workout History</i></span>
      </div>
      <div class="card-body p-0">
        {% if workouts %}
          <div class="table-responsive">
            <table class="table table-dark-custom mb-0">
              <thead>
                <tr>
                  <th>Date</th>
                  <th>Type</th>
                  <th>Exercise</th>
                  <th>Duration</th>
                  <th>Sets/Reps</th>
                  <th>Calories</th>
                  <th>Notes</th>
                </tr>
              </thead>
              <tbody>
                {% for w in workouts %}
                  <tr>
                    <td class="text-secondary small">{{ w.workout_date.strftime('%Y-%m-%d') }}</td>
                    <td><span class="badge bg-secondary">{{ w.workout_type }}</span></td>
                    <td class="fw-bold text-white">{{ w.exercise_name }}</td>
                    <td>{{ w.duration }} min</td>
                    <td>{{ w.sets or 0 }} / {{ w.repetitions or 0 }}</td>
                    <td class="text-warning">{{ "{:,.0f}".format(w.calories_burned) }} kcal</td>
                    <td class="text-secondary small">{{ w.notes or '-' }}</td>
                </tr>
                {% endfor %}
              </tbody>
            </table>
          </div>
        {% else %}
          <div class="text-center py-4">
            <p class="text-secondary small mb-0">Client has not logged any workouts yet.</p>
          </div>
        {% endif %}
      </div>
    </div>
  </div>

```

```

    </div>
    <!-- Active Goals -->
    <div class="card card-dark">
      <div class="card-header card-header-dark">
        <span class="text-white fw-bold"><i
class="bi bi-trophy-fill text-warning me-2"></i> Fitness
Goals</span>
      </div>
      <div class="card-body">
        {% if goals %}
        <div class="row g-3">
          {% for g in goals %}
            <div class="col-md-6">
              <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
                <div class="d-flex justify-
content-between align-items-center mb-1">
                  <strong class="text-white">{{
g.goal_type }}</strong>
                  <span class="badge bg-{{
g.badge_color }}">{{ g.status }}</span>
                </div>
                <small class="text-cyan d-block
mb-1">Target: {{ g.target_value }}</small>
                <p class="text-secondary small
mb-1">{{ g.description or 'No notes.' }}</p>
                <small class="text-
muted">Target Date: {{ g.target_date.strftime('%b %d,
%Y') }}</small>
              </div>
            </div>
          {% endfor %}
        </div>
        {% else %}
        <p class="text-secondary small mb-
0">Client has not set any goals yet.</p>
        {% endif %}
      </div>
    </div>
  <!-- Right Column: Give Workout Recommendation
Form & Existing Recommendations -->
  <div class="col-lg-4">
    <!-- Add Recommendation Form -->
    <div class="card card-dark mb-4">
      <div class="card-header card-header-dark">
        <span class="text-white fw-bold"><i
class="bi bi-pencil-square text-emerald me-2"></i>
Prescribe Workout Guidance</span>
      </div>
      <div class="card-body">
        <form method="POST" action="{{ {
url_for('trainer.add_recommendation', user_id=client.id)
}} }}">
          <div class="mb-3">
            <label for="title" class="form-label
form-label-dark">Guidance Title <span class="text-
danger">*</span></label>
            <input type="text" class="form-control
form-control-dark" id="title" name="title"
placeholder="e.g. Progressive Overload Plan" required>
          </div>
          <div class="mb-3">
            <label for="content" class="form-label
form-label-dark">Workout Advice & Instructions
<span class="text-danger">*</span></label>
            <textarea class="form-control form-
control-dark" id="content" name="content" rows="4"

```

```

placeholder="Provide specific reps, rest periods, or
workout adjustments..." required></textarea>
          </div>
          <button type="submit" class="btn btn-
emerald w-100">
            <i class="bi bi-send-fill me-1"></i>
Send Workout Guidance
          </button>
        </form>
      </div>
    </div>
    <!-- Existing Guidance Notes -->
    <div class="card card-dark">
      <div class="card-header card-header-dark">
        <span class="text-white fw-bold"><i
class="bi bi-chat-quote-fill text-primary me-2"></i>
Prescribed Notes ({{ recommendations|length }})</span>
      </div>
      <div class="card-body">
        {% if recommendations %}
        <div class="d-flex flex-column gap-3">
          {% for r in recommendations %}
            <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
              <div class="d-flex justify-content-
between align-items-center mb-1">
                <h6 class="text-emerald fw-bold
mb-0 fs-6">{{ r.title }}</h6>
                <small class="text-muted">{{
r.created_at.strftime('%b %d') }}</small>
              </div>
              <p class="text-secondary small mb-
2">{{ r.content }}</p>
              <div class="d-flex justify-content-
between align-items-center">
                <small class="text-muted">By:
                {{ r.author.name }}</small>
                <form method="POST"
action="{{ { url_for('trainer.delete_recommendation',
rec_id=r.id) } }}">
                  <button type="submit"
class="btn btn-sm btn-outline-danger py-0 px-2"
title="Delete Note">
                    <i class="bi bi-trash"></i>
                  </button>
                </form>
              </div>
            </div>
          {% endfor %}
        </div>
        {% else %}
        <p class="text-secondary small mb-0">No
workout guidance notes recorded for this client.</p>
        {% endif %}
      </div>
    </div>
  </div>
  {% endblock %}
Users.html:
{% extends 'base.html' %}
{% block title %}Client Directory - Trainer Portal{%
endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">

```

```

<div>
  <h2 class="text-white fw-bold mb-1"><i class="bi
bi-people text-primary me-2"></i> Client Directory</h2>
  <p class="text-secondary small mb-0">Select a
client to review workout logs, inspect goals, and provide
personalized training advice.</p>
</div>
  <a href="{{ url_for('trainer.dashboard') }}"
class="btn btn-outline-custom">
    <i class="bi bi-arrow-left me-1"></i> Trainer
Dashboard
  </a>
</div>
<!-- Search Bar -->
<div class="card card-dark mb-4 p-3">
  <form method="GET" action="{{
url_for('trainer.list_users') }}" class="row g-2">
    <div class="col-md-9">
      <div class="input-group">
        <span class="input-group-text bg-dark
border-secondary text-secondary"><i class="bi bi-
search"></i></span>
        <input type="text" class="form-control form-
control-dark" name="search" value="{{ search }}"
placeholder="Search client by name or email...">
      </div>
    </div>
    <div class="col-md-3 d-flex gap-2">
      <button type="submit" class="btn btn-emerald
flex-grow-1">Search</button>
      <a href="{{ url_for('trainer.list_users') }}"
class="btn btn-outline-custom">Reset</a>
    </div>
  </form>
</div>
<!-- Clients Grid -->
{% if clients %}
  <div class="row g-4">
    {% for c in clients %}
      <div class="col-md-6 col-lg-4">
        <div class="card card-dark h-100 p-4 card-
dark-hover d-flex flex-column">
          <div class="d-flex align-items-center gap-3
mb-3">
            <div class="stat-icon"
style="background: rgba(59, 130, 246, 0.15); color: var(--
accent-blue);">
              <i class="bi bi-person-fill fs-3"></i>
            </div>
            <div>
              <h5 class="text-white fw-bold mb-
0">{{ c.name }}</h5>
              <small class="text-secondary">{{ {
c.email }}</small>
            </div>
          </div>
          <div class="p-3 rounded-3 mb-3 flex-
grow-1" style="background: rgba(15, 23, 42, 0.6); border:
1px solid var(--border-color);">
            <div class="row g-2 text-center">
              <div class="col-4">
                <small class="text-muted d-
block">Workouts</small>
                <strong class="text-white">{{ {
c.workouts|length }}</strong>
              </div>
              <div class="col-4">
                <small class="text-muted d-
block">Weight</small>

```

```

                <strong class="text-white">{{ {
c.weight or '-' }} kg</strong>
              </div>
            </div>
            <div class="col-4">
              <small class="text-muted d-
block">BMI</small>
              <strong class="text-cyan">{{ { c.bmi
or '-' }}</strong>
            </div>
          </div>
          <a href="{{ url_for('trainer.user_detail',
user_id=c.id) }}" class="btn btn-emerald w-100">
            <i class="bi bi-eye me-1"></i> View
Client Details
          </a>
        </div>
      </div>
    {% endfor %}
  </div>
  {% else %}
    <div class="card card-dark text-center py-5">
      <p class="text-secondary mb-0">No client records
found matching your search.</p>
    </div>
  {% endif %}
</div>
{% endblock %}
Weight:
Add.html:
{% extends 'base.html' %}
{% block title %}Log Weight - FitPulse{% endblock %}
{% block content %}
<div class="container py-4">
  <div class="row justify-content-center">
    <div class="col-lg-6">
      <div class="card card-dark shadow-lg">
        <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
          <span class="text-white fw-bold"><i
class="bi bi-speedometer2 text-info me-2"></i> Log
Weight Measurement</span>
          <a href="{{ url_for('weight.history') }}"
class="btn btn-sm btn-outline-custom">History</a>
        </div>
        <div class="card-body p-4">
          <form method="POST" action="{{
url_for('weight.add') }}">
            <div class="mb-3">
              <label for="weight" class="form-label
form-label-dark">Weight (kg) <span class="text-
danger">*</span></label>
              <div class="input-group">
                <input type="number" step="0.1"
class="form-control form-control-dark" id="weight"
name="weight" value="{{ weight or (user.weight if user
else '') }}" placeholder="e.g. 72.5" min="20" max="400"
required autofocus>
                <span class="input-group-text bg-dark
border-secondary text-secondary">kg</span>
              </div>
            </div>
            <div class="mb-3">
              <label for="date" class="form-label
form-label-dark">Measurement Date <span class="text-
danger">*</span></label>
              <input type="date" class="form-control
form-control-dark" id="date" name="date" value="{{ date
or today_date }}" required>

```

```

        </div>
        <div class="mb-4">
            <label for="notes" class="form-label
form-label-dark">Notes (Optional)</label>
            <input type="text" class="form-control
form-control-dark" id="notes" name="notes" value="{{
notes or " }}" placeholder="e.g. Morning weighing, post-
workout">
        </div>
        <div class="d-flex justify-content-end gap-
2">
            <a href="{{ url_for('weight.history') }}"
class="btn btn-outline-custom">Cancel</a>
            <button type="submit" class="btn btn-
emerald">
                <i class="bi bi-check2-circle me-
1"></i> Save Weight Entry
            </button>
        </div>
    </form>
</div>
</div>
</div>
</div>
</div>
</div>
{% endblock %}
Edit.html:
{% extends 'base.html' %}
{% block title %}Edit Weight Entry - FitPulse{%
endblock %}
{% block content %}
<div class="container py-4">
    <div class="row justify-content-center">
        <div class="col-lg-6">
            <div class="card card-dark shadow-lg">
                <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
                    <span class="text-white fw-bold"><i
class="bi bi-pencil-square text-cyan me-2"></i> Edit
Weight Record</span>
                    <a href="{{ url_for('weight.history') }}"
class="btn btn-sm btn-outline-custom">History</a>
                </div>
                <div class="card-body p-4">
                    <form method="POST" action="{{ {
url_for('weight.edit', record_id=record.id) }}">
                        <div class="mb-3">
                            <label for="weight" class="form-label
form-label-dark">Weight (kg) <span class="text-
danger">*</span></label>
                            <div class="input-group">
                                <input type="number" step="0.1"
class="form-control form-control-dark" id="weight"
name="weight" value="{{ record.weight }}" min="20"
max="400" required>
                                <span class="input-group-text bg-dark
border-secondary text-secondary">kg</span>
                            </div>
                        </div>
                        <div class="mb-3">
                            <label for="date" class="form-label
form-label-dark">Date <span class="text-
danger">*</span></label>
                            <input type="date" class="form-control
form-control-dark" id="date" name="date" value="{{ {
record.date.strftime('%Y-%m-%d') }}" required>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

        <label for="notes" class="form-label
form-label-dark">Notes</label>
        <input type="text" class="form-control
form-control-dark" id="notes" name="notes" value="{{ {
record.notes or " }}">
    </div>
    <div class="d-flex justify-content-between
align-items-center">
        <button type="button" class="btn btn-
outline-danger btn-delete-trigger" data-bs-toggle="modal"
data-bs-target="#deleteConfirmModal" data-action="{{ {
url_for('weight.delete', record_id=record.id) }}" data-
name="{{ { record.weight }} kg on {{ {
record.date.strftime('%Y-%m-%d') }}">
            <i class="bi bi-trash me-1"></i>
        Delete
    </button>
    <div class="d-flex gap-2">
        <a href="{{ { url_for('weight.history')
}} " class="btn btn-outline-custom">Cancel</a>
        <button type="submit" class="btn
btn-emerald">
            <i class="bi bi-floppy me-1"></i>
        Update
    </button>
    </div>
</div>
</form>
</div>
</div>
</div>
</div>
</div>
</div>
{% endblock %}
History.html:
{% extends 'base.html' %}
{% block title %}Weight History - FitPulse{% endblock
%}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
    <!-- Header & Action Button -->
    <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
        <div>
            <h2 class="text-white fw-bold mb-1"><i class="bi
bi-speedometer2 text-info me-2"></i> Weight History &
Trends</h2>
            <p class="text-secondary small mb-0">Track
historical body weight logs and calculated BMI
progression.</p>
        </div>
        <a href="{{ { url_for('weight.add') }}" class="btn btn-
emerald">
            <i class="bi bi-plus-lg me-1"></i> Log Weight
Entry
        </a>
    </div>
    <!-- Summary Metrics -->
    <div class="row g-3 mb-4">
        <div class="col-sm-4">
            <div class="stat-card">
                <div class="stat-label">Current Weight</div>
                <div class="stat-value">{{ { current_weight or '--'
}} <span class="fs-6 text-muted">kg</span></div>
            </div>
        </div>
        <div class="col-sm-4">
            <div class="stat-card stat-violet">

```



```

        </div>
        <div class="col-md-6">
            <label for="exercise_name"
class="form-label form-label-dark">Exercise / Routine
Name <span class="text-danger">*</span></label>
            <input type="text" class="form-
control form-control-dark" id="exercise_name"
name="exercise_name" value="{{
workout.exercise_name }}" required>
        </div>
    </div>
    <div class="row g-3 mb-3">
        <div class="col-md-4">
            <label for="duration" class="form-
label form-label-dark">Duration (minutes) <span
class="text-danger">*</span></label>
            <input type="number" class="form-
control form-control-dark" id="duration"
name="duration" value="{{ workout.duration }}"
min="1" max="1440" required>
        </div>
        <div class="col-md-4">
            <label for="sets" class="form-label
form-label-dark">Sets</label>
            <input type="number" class="form-
control form-control-dark" id="sets" name="sets"
value="{{ workout.sets or '0' }}" min="0">
        </div>
        <div class="col-md-4">
            <label for="repetitions" class="form-
label form-label-dark">Reps / Distance</label>
            <input type="number" class="form-
control form-control-dark" id="repetitions"
name="repetitions" value="{{ workout.repetitions or '0'
}}" min="0">
        </div>
    </div>
    <div class="row g-3 mb-3">
        <div class="col-md-6">
            <label for="calories_burned"
class="form-label form-label-dark">Calories Burned
(kcal)</label>
            <input type="number" step="0.1"
class="form-control form-control-dark"
id="calories_burned" name="calories_burned" value="{{
workout.calories_burned }}" min="0">
        </div>
        <div class="col-md-6">
            <label for="workout_date"
class="form-label form-label-dark">Workout Date <span
class="text-danger">*</span></label>
            <input type="date" class="form-
control form-control-dark" id="workout_date"
name="workout_date" value="{{
workout.workout_date.strftime('%Y-%m-%d') }}"
required>
        </div>
    </div>
    <div class="mb-4">
        <label for="notes" class="form-label
form-label-dark">Notes</label>
        <textarea class="form-control form-
control-dark" id="notes" name="notes" rows="3">{{
workout.notes or " " }}</textarea>
    </div>
    <div class="d-flex justify-content-between
align-items-center">
        <button type="button" class="btn btn-
outline-danger btn-delete-trigger" data-bs-toggle="modal"

```

```

data-bs-target="#deleteConfirmModal" data-action="{{
url_for('workout.delete', workout_id=workout.id) }}"
data-name="{{ workout.exercise_name }}">
        <i class="bi bi-trash me-1"></i>
Delete
        </button>
        <div class="d-flex gap-2">
            <a href="{{ url_for('workout.history')
}}" class="btn btn-outline-custom">Cancel</a>
            <button type="submit" class="btn
btn-emerald">
                <i class="bi bi-floppy me-1"></i>
Update Workout
            </button>
        </div>
    </div>
</form>
</div>
</div>
</div>
</div>
</div>
</div>
{% endblock %}
History.html:
{% extends 'base.html' %}
{% block title %}Workout History - FitPulse{% endblock
%}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
    <!-- Header & Action Button -->
    <div class="d-flex flex-column flex-md-row justify-
content-between align-items-md-center gap-3 mb-4">
        <div>
            <h2 class="text-white fw-bold mb-1"><i class="bi
bi-clock-history text-cyan me-2"></i> Workout
History</h2>
            <p class="text-secondary small mb-0">Browse
and manage your historical exercise sessions.</p>
        </div>
        <a href="{{ url_for('workout.add') }}" class="btn
btn-emerald">
            <i class="bi bi-plus-lg me-1"></i> Log New
Workout
        </a>
    </div>
    <!-- Summary Metrics -->
    <div class="row g-3 mb-4">
        <div class="col-sm-4">
            <div class="stat-card">
                <div class="stat-label">Total Workouts</div>
                <div class="stat-value">{{ total_workouts
}}</div>
            </div>
        </div>
        <div class="col-sm-4">
            <div class="stat-card stat-violet">
                <div class="stat-label">Total Duration</div>
                <div class="stat-value">{{ total_duration }}
<span class="fs-6 text-muted">mins</span></div>
            </div>
        </div>
        <div class="col-sm-4">
            <div class="stat-card stat-amber">
                <div class="stat-label">Total Calories</div>
                <div class="stat-value">{{
"{:,0f}".format(total_calories) }} <span class="fs-6 text-
muted">kcal</span></div>
            </div>
        </div>
    </div>

```

```

</div>
<!-- Filter & Search Toolbar -->
<div class="card card-dark mb-4 p-3">
  <form method="GET" action="{{
url_for('workout.history') }}" class="row g-2 align-items-
center">
    <div class="col-md-5">
      <div class="input-group">
        <span class="input-group-text bg-dark
border-secondary text-secondary"><i class="bi bi-
search"></i></span>
        <input type="text" class="form-control form-
control-dark" name="search" value="{{ search_query }}"
placeholder="Search exercise or notes...">
      </div>
    </div>
    <div class="col-md-4">
      <select class="form-select form-select-dark"
name="type">
        <option value="">All Workout
Types</option>
        {% for wt in workout_types %}
          <option value="{{ wt }}" {% if
selected_type == wt %>selected{% endif %}>{{ wt
}}</option>
        {% endfor %}
      </select>
    </div>
    <div class="col-md-3 d-flex gap-2">
      <button type="submit" class="btn btn-emerald
flex-grow-1">Filter</button>
      <a href="{{ url_for('workout.history') }}"
class="btn btn-outline-custom">Reset</a>
    </div>
  </form>
</div>
<!-- Workout History Table -->
<div class="card card-dark shadow-sm">
  <div class="card-body p-0">
    {% if workouts %}
      <div class="table-responsive">
        <table class="table table-dark-custom mb-0">
          <thead>
            <tr>
              <th>Date</th>
              <th>Type</th>
              <th>Exercise</th>
              <th>Duration</th>
              <th>Sets</th>
              <th>Reps</th>
              <th>Calories</th>
              <th>Notes</th>
              <th class="text-end">Actions</th>
            </tr>
          </thead>
          <tbody>
            {% for w in workouts %}
              <tr>
                <td class="text-secondary">{{
w.workout_date.strftime('%Y-%m-%d') }}</td>
                <td>
                  <span class="badge bg-
secondary">
                    <i class="bi {{ w.icon }}" me-
1"></i> {{ w.workout_type }}
                  </span>
                </td>
                <td class="fw-bold text-white">{{
w.exercise_name }}</td>

```

```

                <td>{{ w.duration }} mins</td>
                <td>{{ w.sets or '-' }}</td>
                <td>{{ w.repetitions or '-' }}</td>
                <td class="text-warning fw-
semibold">{{ "{:,0f}".format(w.calories_burned) }}
kcal</td>
                <td class="text-secondary small"
style="max-width: 200px; text-overflow: ellipsis;
overflow: hidden; white-space: nowrap;">
                  {{ w.notes or '-' }}
                </td>
              </tr>
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>
  <div class="btn-group btn-group-
sm">
    <a href="{{
url_for('workout.edit', workout_id=w.id) }}" class="btn
btn-outline-secondary" title="Edit">
      <i class="bi bi-pencil"></i>
    </a>
    <button type="button"
class="btn btn-outline-danger btn-delete-trigger" data-bs-
toggle="modal" data-bs-target="#deleteConfirmModal"
data-action="{{ url_for('workout.delete',
workout_id=w.id) }}" data-name="{{ w.exercise_name
}}" on="{{ w.workout_date.strftime('%Y-%m-%d') }}">
      <i class="bi bi-trash"></i>
    </button>
  </div>
</div>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
  <div class="text-center py-5">
    <i class="bi bi-journal-x fs-1 text-muted d-
block mb-2"></i>
    <h5 class="text-white">No workouts
found</h5>
    <p class="text-secondary small mb-3">Try
adjusting your filter criteria or log a new workout.</p>
    <a href="{{ url_for('workout.add') }}"
class="btn btn-emerald btn-sm">Log New Workout</a>
  </div>
{% endif %}
</div>
</div>
{% endblock %}
Base.html:
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>{% block title %}FitPulse - Modern Fitness &
Workout Tracker{% endblock %}</title>

  <!-- Meta tags for SEO -->
  <meta name="description" content="FitPulse -
Comprehensive full-stack fitness and workout tracking
system for workouts, goals, weight logs, BMI analytics,
and specialist guidance.">

  <!-- Bootstrap 5 CSS -->

```

```

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/cs
s/bootstrap.min.css" rel="stylesheet">
<!-- Bootstrap Icons -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css"
rel="stylesheet">

<!-- Custom Theme CSS -->
<link rel="stylesheet" href="{{ url_for('static',
filename='css/style.css') }}">

{% block extra_css %}{% endblock %}
</head>
<body>
<!-- Main Navigation Bar -->
<nav class="navbar navbar-expand-lg navbar-custom">
  <div class="container-fluid px-lg-4">
    <a class="navbar-brand brand-logo" href="{{
url_for('dashboard.index') }}">
      <i class="bi bi-heart-pulse-fill"></i> FitPulse
    </a>
    <button class="navbar-toggler text-white border-
0" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarMain">
      <i class="bi bi-list fs-2"></i>
    </button>
    <div class="collapse navbar-collapse"
id="navbarMain">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0
ms-lg-3">
        {% if current_user %}
          {% if current_user.role == 'user' %}
            <li class="nav-item">
              <a class="nav-link nav-link-custom
{% if request.endpoint == 'dashboard.user_dashboard'
}%active{% endif %}" href="{{
url_for('dashboard.user_dashboard') }}">
                <i class="bi bi-grid-fill me-1"></i>
Dashboard
              </a>
            </li>
            <li class="nav-item dropdown">
              <a class="nav-link nav-link-custom
dropdown-toggle {% if 'workout' in request.endpoint
}%active{% endif %}" href="#" role="button" data-bs-
toggle="dropdown">
                <i class="bi bi-activity me-1"></i>
Workouts
              </a>
              <ul class="dropdown-menu
dropdown-menu-dark">
                <li><a class="dropdown-item"
href="{{ url_for('workout.add') }}"><i class="bi bi-plus-
circle me-2 text-success"></i> Add Workout</a></li>
                <li><a class="dropdown-item"
href="{{ url_for('workout.history') }}"><i class="bi bi-
clock-history me-2 text-info"></i> Workout
History</a></li>
              </ul>
            </li>
            <li class="nav-item dropdown">
              <a class="nav-link nav-link-custom
dropdown-toggle {% if 'goal' in request.endpoint
}%active{% endif %}" href="#" role="button" data-bs-
toggle="dropdown">
                <i class="bi bi-trophy-fill me-
1"></i> Goals
              </a>

```

```

              <ul class="dropdown-menu
dropdown-menu-dark">
                <li><a class="dropdown-item"
href="{{ url_for('goal.add') }}"><i class="bi bi-plus-
circle me-2 text-success"></i> Add Goal</a></li>
                <li><a class="dropdown-item"
href="{{ url_for('goal.view_goals') }}"><i class="bi bi-
card-checklist me-2 text-warning"></i> View
Goals</a></li>
              </ul>
            </li>
            <li class="nav-item dropdown">
              <a class="nav-link nav-link-custom
dropdown-toggle {% if 'weight' in request.endpoint
}%active{% endif %}" href="#" role="button" data-bs-
toggle="dropdown">
                <i class="bi bi-speedometer me-
1"></i> Weight
              </a>
              <ul class="dropdown-menu
dropdown-menu-dark">
                <li><a class="dropdown-item"
href="{{ url_for('weight.add') }}"><i class="bi bi-plus-
circle me-2 text-success"></i> Add Weight</a></li>
                <li><a class="dropdown-item"
href="{{ url_for('weight.history') }}"><i class="bi bi-
journal-text me-2 text-info"></i> Weight
History</a></li>
              </ul>
            </li>
            <li class="nav-item">
              <a class="nav-link nav-link-custom
{% if request.endpoint == 'bmi.view_calculator'
}%active{% endif %}" href="{{
url_for('bmi.view_calculator') }}">
                <i class="bi bi-calculator-fill me-
1"></i> BMI
              </a>
            </li>
            <li class="nav-item">
              <a class="nav-link nav-link-custom
{% if request.endpoint == 'progress.view_progress'
}%active{% endif %}" href="{{
url_for('progress.view_progress') }}">
                <i class="bi bi-graph-up-arrow me-
1"></i> Progress
              </a>
            </li>
            {% elif current_user.role == 'trainer' %}
            <li class="nav-item">
              <a class="nav-link nav-link-custom
{% if request.endpoint == 'trainer.dashboard' %}active{%
endif %}" href="{{ url_for('trainer.dashboard') }}">
                <i class="bi bi-speedometer2 me-
1"></i> Trainer Dashboard
              </a>
            </li>
            <li class="nav-item">
              <a class="nav-link nav-link-custom
{% if request.endpoint == 'trainer.list_users' %}active{%
endif %}" href="{{ url_for('trainer.list_users') }}">
                <i class="bi bi-people-fill me-
1"></i> Client Directory
              </a>
            </li>
            {% elif current_user.role == 'nutritionist'
%}
            <li class="nav-item">

```

```

        <a class="nav-link nav-link-custom
{% if request.endpoint == 'nutritionist.dashboard'
}%}active{% endif %}" href="{{
url_for('nutritionist.dashboard') }}">
        <i class="bi bi-speedometer2 me-
1"></i> Nutritionist Dashboard
    </a>
</li>
<li class="nav-item">
    <a class="nav-link nav-link-custom
{% if request.endpoint == 'nutritionist.list_users'
}%}active{% endif %}" href="{{
url_for('nutritionist.list_users') }}">
        <i class="bi bi-people-fill me-
1"></i> Client Profiles & BMI
    </a>
    {% elif current_user.role == 'admin' %}
    <li class="nav-item">
        <a class="nav-link nav-link-custom
{% if request.endpoint == 'admin.dashboard' %}active{%
endif %}" href="{{ url_for('admin.dashboard') }}">
            <i class="bi bi-shield-lock-fill me-
1"></i> Admin Dashboard
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link nav-link-custom
{% if request.endpoint == 'admin.list_users' %}active{%
endif %}" href="{{ url_for('admin.list_users') }}">
            <i class="bi bi-person-gear me-
1"></i> Users
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link nav-link-custom
{% if request.endpoint == 'admin.list_workouts'
}%}active{% endif %}" href="{{
url_for('admin.list_workouts') }}">
            <i class="bi bi-activity me-1"></i>
Workouts
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link nav-link-custom
{% if request.endpoint == 'admin.list_goals' %}active{%
endif %}" href="{{ url_for('admin.list_goals') }}">
            <i class="bi bi-trophy-fill me-
1"></i> Goals
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link nav-link-custom
{% if request.endpoint == 'admin.list_weights'
}%}active{% endif %}" href="{{
url_for('admin.list_weights') }}">
            <i class="bi bi-speedometer me-
1"></i> Weights
        </a>
    </li>
    {% endif %}
    <li class="nav-item">
        <a class="nav-link nav-link-custom"
href="{{ url_for('dashboard.index')
}}#features">Features</a>
    </li>
</li class="nav-item">

```

```

        <a class="nav-link nav-link-custom"
href="{{ url_for('dashboard.index')
}}#roles">Specialists</a>
    </li>
    <li class="nav-item">
        <a class="nav-link nav-link-custom"
href="{{ url_for('bmi.view_calculator') }}">BMI
Calculator</a>
    </li>
    {% endif %}
</ul>
<div class="d-flex align-items-center gap-3">
    {% if current_user %}
    <div class="dropdown">
        <button class="btn btn-outline-custom
dropdown-toggle d-flex align-items-center gap-2"
type="button" data-bs-toggle="dropdown">
            <i class="bi bi-person-circle fs-5 text-
emerald"></i>
            <span>{{ current_user.name
}}</span>
            <span class="badge bg-secondary
role-badge">{{ current_user.role }}</span>
        </button>
        <ul class="dropdown-menu dropdown-
menu-dark dropdown-menu-end shadow-lg">
            <li>
                <a class="dropdown-item"
href="{{ url_for('profile.view_profile') }}">
                    <i class="bi bi-person me-
2"></i> My Profile
                </a>
            </li>
            <li><hr class="dropdown-
divider"></li>
            <li>
                <a class="dropdown-item text-
danger" href="{{ url_for('auth.logout') }}">
                    <i class="bi bi-box-arrow-right
me-2"></i> Log Out
                </a>
            </li>
        </ul>
    </div>
    {% else %}
    <a href="{{ url_for('auth.login') }}"
class="btn btn-outline-custom">Log In</a>
    <a href="{{ url_for('auth.register') }}"
class="btn btn-emerald">Get Started</a>
    {% endif %}
</div>
</div>
</nav>
<!-- Flash Notification Alerts -->
<div class="container-fluid px-lg-4 mt-3">
    {% with messages =
get_flashed_messages(with_categories=true) %}
    {% if messages %}
        {% for category, message in messages %}
            <div class="alert alert-{{ category if category
!= 'error' else 'danger' }} alert-custom alert-dismissible
fade show alert-auto-dismiss" role="alert">
                <i class="bi {% if category == 'success'
}%}bi-check-circle-fill{% elif category == 'danger' %}bi-
exclamation-triangle-fill{% elif category == 'warning'
}%}bi-exclamation-circle-fill{% else %}bi-info-circle-
fill{% endif %} me-2"></i>
                {{ message }}
            </div>
        {% endfor %}
    {% endif %}

```

```

        <button type="button" class="btn-close"
data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
    {% endfor %}
    {% endif %}
    {% endwith %}
</div>
<!-- Main Page Content -->
<main class="flex-grow-1">
    {% block content %}{% endblock %}
</main>
<!-- Global Delete Confirmation Modal -->
<div class="modal fade" id="deleteConfirmModal"
tabindex="-1" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content modal-content-dark">
            <div class="modal-header modal-header-dark">
                <h5 class="modal-title text-danger"><i
class="bi bi-exclamation-triangle-fill me-2"></i> Confirm
Deletion</h5>
                <button type="button" class="btn-close btn-
close-white" data-bs-dismiss="modal" aria-
label="Close"></button>
            </div>
            <div class="modal-body">
                Are you sure you want to permanently delete
<strong id="deleteItemName" class="text-white">this
item</strong>? This action cannot be undone.
            </div>
            <div class="modal-footer modal-footer-dark">
                <button type="button" class="btn btn-outline-
custom" data-bs-dismiss="modal">Cancel</button>
                <form id="deleteConfirmForm"
method="POST" action="" class="d-inline">
                    <button type="submit" class="btn btn-
danger">Confirm Delete</button>
                </form>
            </div>
        </div>
    </div>
</div>
</div>
<!-- Application Footer -->
<footer>
    <div class="container-fluid px-lg-4 d-flex flex-
column flex-md-row justify-content-between align-items-
center gap-2">
        <div>
            <span class="fw-bold text-white"><i class="bi
bi-heart-pulse-fill text-success"></i> FitPulse</span>
            &copy; {{ current_year }} Fitness & Workout Tracker.
            All rights reserved.
        </div>
        <div class="d-flex gap-3">
            <a href="{{ url_for('bmi.view_calculator') }}"
class="text-muted text-decoration-none">BMI
Calculator</a>
            <span class="text-muted">&bull;</span>
            <span class="text-muted">Flask &
SQLAlchemy Powered</span>
        </div>
    </div>
</footer>
<!-- Bootstrap 5 JS Bundle -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/
bootstrap.bundle.min.js"></script>
<!-- Main Application Scripts -->
<script src="{{ url_for('static', filename='js/main.js')
}}"></script>

```

```

    {% block extra_js %}{% endblock %}
</body>
</html>
Bmi.html:
{% extends 'base.html' %}
{% block title %}Interactive BMI Calculator - FitPulse{%
endblock %}
{% block content %}
<div class="container py-4">
    <div class="row g-4">
        <!-- Left Column: Interactive Calculator -->
        <div class="col-lg-7">
            <div class="card card-dark p-4 shadow-lg">
                <div class="d-flex align-items-center gap-3 mb-
4">
                    <div class="stat-icon" style="background:
rgba(6, 182, 212, 0.15); color: var(--accent-cyan);">
                        <i class="bi bi-calculator-fill fs-3"></i>
                    </div>
                    <div>
                        <h4 class="text-white fw-bold mb-
0">Body Mass Index (BMI) Calculator</h4>
                        <p class="text-secondary small mb-
0">Calculate your BMI using World Health Organization
(WHO) standards.</p>
                    </div>
                </div>
                <!-- Sliders & Inputs -->
                <div class="mb-4">
                    <div class="d-flex justify-content-between
align-items-center mb-1">
                        <label for="bmiHeightInput" class="form-
label form-label-dark mb-0">Height (cm)</label>
                        <div class="input-group" style="width:
140px;">
                            <input type="number" class="form-
control form-control-dark text-end" id="bmiHeightInput"
value="{{ initial_height }}" min="80" max="250"
step="0.5">
                            <span class="input-group-text bg-dark
border-secondary text-secondary">cm</span>
                        </div>
                    </div>
                    <div class="mb-4">
                        <div class="d-flex justify-content-between
align-items-center mb-1">
                            <label for="bmiWeightInput" class="form-
label form-label-dark mb-0">Weight (kg)</label>
                            <div class="input-group" style="width:
140px;">
                                <input type="number" class="form-
control form-control-dark text-end" id="bmiWeightInput"
value="{{ initial_weight }}" min="30" max="250"
step="0.5">
                                <span class="input-group-text bg-dark
border-secondary text-secondary">kg</span>
                            </div>
                        </div>
                        <div class="mb-4">
                            <input type="range" class="form-range"
id="bmiHeightSlider" min="100" max="230" step="0.5"
value="{{ initial_height }}">
                        </div>
                    </div>
                </div>
            </div>
        </div>
        <!-- Result Card -->

```



```

        <td class="text-end fw-bold
text-cyan">{{ bh.bmi or '--' }}</td>
      </tr>
    </tbody>
  </table>
</div>
{% else %}
  <div class="text-center py-4">
    <p class="text-secondary small mb-2">No weight logs to generate BMI timeline.</p>
    <a href="{{ url_for('weight.add') }}"
class="btn btn-outline-custom btn-sm">Log Weight</a>
  </div>
{% endif %}
</div>
</div>
{% endif %}
</div>
</div>
</div>
{% endblock %}
{% block extra_js %}
<script src="{{ url_for('static', filename='js/bmi.js') }}"></script>
{% endblock %}
Dashboard.html:
{% extends 'base.html' %}
{% block title %}Dashboard - FitPulse{% endblock %}
{% block content %}
<div class="container-fluid px-lg-4 py-4">
  <!-- Welcome Header & Quick Action Buttons -->
  <div class="d-flex flex-column flex-md-row justify-content-between align-items-md-center gap-3 mb-4">
    <div>
      <h2 class="text-white fw-bold mb-1">
        Hello, {{ user.name }} <span class="badge bg-emerald text-dark fw-bold fs-6">User</span>
      </h2>
      <p class="text-secondary small mb-0">Here is your daily fitness summary and activity metrics.</p>
    </div>
    <div class="d-flex gap-2 flex-wrap">
      <a href="{{ url_for('workout.add') }}" class="btn btn-emerald btn-sm">
        <i class="bi bi-plus-lg me-1"></i> Log Workout
      </a>
      <a href="{{ url_for('goal.add') }}" class="btn btn-outline-custom btn-sm">
        <i class="bi bi-trophy me-1"></i> Add Goal
      </a>
      <a href="{{ url_for('weight.add') }}" class="btn btn-outline-custom btn-sm">
        <i class="bi bi-speedometer me-1"></i> Log Weight
      </a>
    </div>
  </div>
  <!-- Stat Metrics Row -->
  <div class="row g-3 mb-4">
    <!-- Metric 1: Workouts -->
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card">
        <div class="d-flex align-items-center justify-content-between">
          <div>
            <div class="stat-label">Total Workouts</div>

```

```

          <div class="stat-value">{{ total_workouts }}</div>
        </div>
        <small class="text-emerald"><i class="bi bi-calendar-check me-1"></i> {{ weekly_workouts_count }} this week</small>
      </div>
      <div class="stat-icon">
        <i class="bi bi-activity"></i>
      </div>
    </div>
    <!-- Metric 2: Calories Burned -->
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card stat-amber">
        <div class="d-flex align-items-center justify-content-between">
          <div>
            <div class="stat-label">Calories Burned</div>
            <div class="stat-value">{{ "{:,.0f}".format(total_calories) }} <span class="fs-6 text-muted">kcal</span></div>
          </div>
          <small class="text-warning"><i class="bi bi-fire me-1"></i> {{ "{:,.0f}".format(weekly_calories) }} kcal this week</small>
        </div>
        <div class="stat-icon icon-amber">
          <i class="bi bi-fire"></i>
        </div>
      </div>
    </div>
    <!-- Metric 3: Time Spent -->
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card stat-violet">
        <div class="d-flex align-items-center justify-content-between">
          <div>
            <div class="stat-label">Active Exercise Time</div>
            <div class="stat-value">{{ total_minutes }} <span class="fs-6 text-muted">mins</span></div>
          </div>
          <small class="text-indigo"><i class="bi bi-clock me-1"></i> Total training time</small>
        </div>
        <div class="stat-icon icon-violet">
          <i class="bi bi-stopwatch"></i>
        </div>
      </div>
    </div>
    <!-- Metric 4: Weight & BMI -->
    <div class="col-sm-6 col-xl-3">
      <div class="stat-card stat-rose">
        <div class="d-flex align-items-center justify-content-between">
          <div>
            <div class="stat-label">Current Weight / BMI</div>
            <div class="stat-value">
              {{ user.weight or '--' }} <span class="fs-6 text-muted">kg</span>
            </div>
          </div>
          <small>
            BMI: <strong class="text-white">{{ bmi or '--' }}</strong>
          </small>
        </div>

```

```

        <span class="badge bg-{{ bmi_color }}
ms-1" style="font-size: 0.7rem;">{{ bmi_category
}}</span>
        </small>
    </div>
    <div class="stat-icon icon-rose">
    <i class="bi bi-heart-pulse"></i>
    </div>
    </div>
    </div>
    <!-- Main Content Row -->
    <div class="row g-4">

        <!-- Left Column: Recent Workouts &
Recommendations -->
        <div class="col-lg-8">
            <!-- Recent Workouts Card -->
            <div class="card card-dark mb-4">
                <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
                    <span class="text-white fw-bold"><i
class="bi bi-clock-history text-cyan me-2"></i> Recent
Workouts</span>
                    <a href="{{ url_for('workout.history') }}"
class="btn btn-sm btn-outline-custom">View All</a>
                </div>
                <div class="card-body p-0">
                    {% if latest_workouts %}
                    <div class="table-responsive">
                        <table class="table table-dark-custom">
                            <thead>
                                <tr>
                                    <th>Date</th>
                                    <th>Workout</th>
                                    <th>Type</th>
                                    <th>Duration</th>
                                    <th>Calories</th>
                                    <th class="text-
end">Action</th>
                                </tr>
                            </thead>
                            <tbody>
                                {% for w in latest_workouts %}
                                <tr>
                                    <td class="text-secondary">{{
w.workout_date.strftime('%b %d, %Y') }}</td>
                                    <td class="fw-semibold text-
white">
                                        <i class="bi {{ w.icon }}"
me-1 text-emerald"></i> {{ w.exercise_name }}
                                    </td>
                                    <td><span class="badge bg-
secondary">{{ w.workout_type }}</span></td>
                                    <td>{{ w.duration }}
mins</td>
                                    <td class="text-warning">{{
"{:,0f}".format(w.calories_burned) }} kcal</td>
                                    <td class="text-end">
                                        <a href="{{
url_for('workout.edit', workout_id=w.id) }}" class="btn
btn-sm btn-outline-secondary py-0 px-2" title="Edit">
                                        <i class="bi bi-
pencil"></i>
                                    </a>
                                    </td>
                                </tr>
                                {% endfor %}
                            </tbody>
                        </table>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        </tbody>
    </table>
</div>
{% else %}
    <div class="text-center py-5">
        <i class="bi bi-clipboard2-plus fs-1 text-
muted d-block mb-2"></i>
        <p class="text-secondary mb-3">No
workouts recorded yet.</p>
        <a href="{{ url_for('workout.add') }}"
class="btn btn-emerald btn-sm">Log Your First
Workout</a>
    </div>
{% endif %}
</div>
</div>
<!-- Specialist Recommendations Received -->
{% if recommendations %}
    <div class="card card-dark">
        <div class="card-header card-header-dark">
            <span class="text-white fw-bold"><i
class="bi bi-chat-quote-fill text-emerald me-2"></i>
Specialist Guidance & Advice</span>
        </div>
        <div class="card-body">
            <div class="row g-3">
                {% for rec in recommendations %}
                <div class="col-md-6">
                    <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.6); border: 1px
solid var(--border-color);">
                        <div class="d-flex justify-
content-between align-items-center mb-2">
                            <span class="badge bg-{{
rec.badge_color }}">{{ rec.note_type }}</span>
                            <small class="text-muted">{{
rec.created_at.strftime('%b %d, %Y') }}</small>
                        </div>
                        <h6 class="text-white fw-bold
mb-1">{{ rec.title }}</h6>
                        <p class="text-secondary small
mb-2">{{ rec.content }}</p>
                        <small class="text-muted"><i
class="bi bi-person-check me-1"></i> From: {{
rec.author.name }} ({{ rec.author.role|capitalize
}})</small>
                    </div>
                </div>
                {% endfor %}
            </div>
        </div>
    </div>
{% endif %}
</div>
<!-- Right Column: Goals & Weight Overview -->
<div class="col-lg-4">
    <!-- Active Goals Card -->
    <div class="card card-dark mb-4">
        <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
            <span class="text-white fw-bold"><i
class="bi bi-trophy-fill text-warning me-2"></i> Active
Goals</span>
            <a href="{{ url_for('goal.view_goals') }}"
class="btn btn-sm btn-outline-custom">Manage</a>
        </div>
        <div class="card-body">
            {% if active_goals %}
            <div class="d-flex flex-column gap-3">

```

```

        {% for g in active_goals %}
        <div class="p-3 rounded-3"
style="background: rgba(15, 23, 42, 0.5); border: 1px
solid var(--border-color);">
        <div class="d-flex justify-content-
between align-items-start mb-1">
        <h6 class="text-white fw-bold
mb-0">{{ g.goal_type }}</h6>
        <span class="badge bg-
primary">{{ g.target_value }}</span>
        </div>
        <p class="text-secondary small mb-
2">{{ g.description or 'No description provided' }}</p>
        <div class="d-flex justify-content-
between align-items-center">
        <small class="text-
muted">Target: {{ g.target_date.strftime("%b %d, %Y")
}}</small>
        <form method="POST"
action="{{ url_for('goal.toggle_status', goal_id=g.id) }}">
        <button type="submit"
class="btn btn-sm btn-outline-success py-0 px-2"
title="Mark as Completed">
        <i class="bi bi-
check2"></i> Done
        </button>
        </form>
        </div>
        </div>
        {% endfor %}
    </div>
    <div class="text-center py-4">
    <i class="bi bi-flag fs-2 text-muted d-
block mb-2"></i>
    <p class="text-secondary small mb-
3">No active goals currently set.</p>
    <a href="{{ url_for('goal.add') }}"
class="btn btn-outline-custom btn-sm">Set a Goal</a>
    </div>
    {% endif %}
</div>
</div>
<!-- Recent Weight Logs Card -->
<div class="card card-dark">
    <div class="card-header card-header-dark d-flex
justify-content-between align-items-center">
        <span class="text-white fw-bold"><i
class="bi bi-speedometer text-info me-2"></i> Weight
Logs</span>
        <a href="{{ url_for('weight.history') }}"
class="btn btn-sm btn-outline-custom">History</a>
    </div>
    <div class="card-body p-0">
        {% if recent_weights %}
        <table class="table-responsive">
            <table class="table table-dark-custom">
                <thead>
                    <tr>
                        <th>Date</th>
                        <th>Weight</th>
                        <th class="text-end">BMI</th>
                    </tr>
                </thead>
                <tbody>
                    {% for rw in recent_weights %}
                    <tr>
                        <td class="text-secondary
small">{{ rw.date.strftime('%b %d') }}</td>

```

```

                        <td class="fw-bold text-
white">{{ rw.weight }} kg</td>
                        <td class="text-end">
                            <span class="badge bg-
secondary">{{ rw.bmi_for_height(user.height) or '--'
}}</span>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
        {% else %}
        <div class="text-center py-4">
        <p class="text-secondary small mb-
2">No weight history logged.</p>
        <a href="{{ url_for('weight.add') }}"
class="btn btn-outline-custom btn-sm">Add Weight</a>
        </div>
        {% endif %}
    </div>
</div>
</div>
</div>
{% endblock %}
Index.html:
{% extends 'base.html' %}
{% block title %}FitPulse - Modern Fitness & Workout
Tracker{% endblock %}
{% block content %}
<div class="container-fluid px-lg-5">

    <!-- Hero Section -->
    <section class="hero-section text-center py-5">
        <div class="row justify-content-center">
            <div class="col-lg-8">
                <span class="badge bg-emerald text-dark fw-
bold px-3 py-2 rounded-pill mb-3" style="background:
var(--gradient-emerald);">
                <i class="bi bi-stars"></i> Intelligent Fitness
Ecosystem
            </span>
            <h1 class="hero-title text-white">
                Transform Your Fitness Journey With <span
style="background: var(--gradient-emerald); -webkit-
background-clip: text; -webkit-text-fill-color:
transparent;">Precision Tracking</span>
            </h1>
            <p class="hero-subtitle mx-auto">
                Seamlessly record daily workouts, track body
weight trends, calculate dynamic BMI, set ambitious
fitness goals, and receive professional guidance from
trainers and nutritionists.
            </p>
            <div class="d-flex justify-content-center gap-3
flex-wrap mt-4">
                <a href="{{ url_for('auth.register') }}"
class="btn btn-emerald btn-lg px-4 py-3">
                    <i class="bi bi-rocket-takeoff-fill me-
2"></i> Get Started Free
                </a>
                <a href="{{ url_for('bmi.view_calculator')
}}" class="btn btn-outline-custom btn-lg px-4 py-3">
                    <i class="bi bi-calculator me-2"></i>
Calculate BMI
                </a>
            </div>
        </div>
    </div>

```

```

    </div>
  </section>
  <!-- Core Features Grid -->
  <section id="features" class="py-5">
    <div class="text-center mb-5">
      <h6 class="text-emerald text-uppercase fw-bold letter-spacing-1">Features</h6>
      <h2 class="text-white fw-bold">Everything You Need to Reach Your Peak</h2>
    </div>
    <div class="row g-4">
      <!-- Feature 1 -->
      <div class="col-md-6 col-lg-3">
        <div class="card card-dark card-dark-hover h-100 p-4">
          <div class="feature-icon-box bg-emerald text-dark" style="background: rgba(16, 185, 129, 0.15); color: var(--accent-emerald);">
            <i class="bi bi-activity"></i>
          </div>
          <h5 class="text-white fw-bold mb-2">Workout Tracking</h5>
          <p class="text-secondary small mb-0">Log running, strength, cycling, yoga, and HIIT sessions with duration, sets, reps, and calories burned.</p>
        </div>
      </div>
      <!-- Feature 2 -->
      <div class="col-md-6 col-lg-3">
        <div class="card card-dark card-dark-hover h-100 p-4">
          <div class="feature-icon-box" style="background: rgba(99, 102, 241, 0.15); color: var(--accent-indigo);">
            <i class="bi bi-trophy-fill"></i>
          </div>
          <h5 class="text-white fw-bold mb-2">Goal Management</h5>
          <p class="text-secondary small mb-0">Set customized targets for weight loss, muscle gain, running mileage, or workout frequency with target dates.</p>
        </div>
      </div>
      <!-- Feature 3 -->
      <div class="col-md-6 col-lg-3">
        <div class="card card-dark card-dark-hover h-100 p-4">
          <div class="feature-icon-box" style="background: rgba(6, 182, 212, 0.15); color: var(--accent-cyan);">
            <i class="bi bi-speedometer2"></i>
          </div>
          <h5 class="text-white fw-bold mb-2">Weight & BMI Logs</h5>
          <p class="text-secondary small mb-0">Log your weight over time, view historical trends, and calculate precise WHO-standard BMI classifications.</p>
        </div>
      </div>
      <!-- Feature 4 -->
      <div class="col-md-6 col-lg-3">
        <div class="card card-dark card-dark-hover h-100 p-4">
          <div class="feature-icon-box" style="background: rgba(245, 158, 11, 0.15); color: var(--accent-amber);">
            <i class="bi bi-graph-up-arrow"></i>
          </div>

```

```

      <h5 class="text-white fw-bold mb-2">Visual Analytics</h5>
      <p class="text-secondary small mb-0">Interactive Chart.js visual analytics for weight trajectories, workout intensity, and goal completion rates.</p>
    </div>
  </div>
</section>
<!-- Multi-Role Specialist Ecosystem -->
<section id="roles" class="py-5 border-top" style="border-color: var(--border-color) !important;">
  <div class="text-center mb-5">
    <h6 class="text-cyan text-uppercase fw-bold letter-spacing-1">Ecosystem</h6>
    <h2 class="text-white fw-bold">Built for Athletes & Professionals</h2>
  </div>
  <div class="row g-4">
    <div class="col-lg-4">
      <div class="card card-dark p-4 h-100 border-start border-3 border-success">
        <div class="d-flex align-items-center gap-3 mb-3">
          <i class="bi bi-person-fill fs-2 text-success"></i>
          <div>
            <h5 class="text-white fw-bold mb-0">Fitness Users</h5>
            <small class="text-muted">Self-directed tracking</small>
          </div>
        </div>
        <p class="text-secondary small">Maintain workout history, log weekly weight, monitor personal BMI, and track milestones with real-time feedback.</p>
      </div>
    </div>
    <div class="col-lg-4">
      <div class="card card-dark p-4 h-100 border-start border-3 border-primary">
        <div class="d-flex align-items-center gap-3 mb-3">
          <i class="bi bi-person-workspace fs-2 text-primary"></i>
          <div>
            <h5 class="text-white fw-bold mb-0">Certified Trainers</h5>
            <small class="text-muted">Workout guidance</small>
          </div>
        </div>
        <p class="text-secondary small">Inspect client exercise logs, monitor consistency, and prescribe structured workout guidance notes directly.</p>
      </div>
    </div>
    <div class="col-lg-4">
      <div class="card card-dark p-4 h-100 border-start border-3 border-warning">
        <div class="d-flex align-items-center gap-3 mb-3">
          <i class="bi bi-egg-fried fs-2 text-warning"></i>
          <div>
            <h5 class="text-white fw-bold mb-0">Clinical Nutritionists</h5>

```

```

        <small class="text-muted">Dietary
optimization</small>
      </div>
    </div>
    <p class="text-secondary small">Analyze
client weight progression, BMI classifications, and deliver
tailored meal & nutrition advice.</p>
  </div>
</div>
</section>
<!-- CTA Section -->
<section class="my-5 p-5 card-dark rounded-4 text-
center text-white" style="background: linear-
gradient(135deg, rgba(16, 185, 129, 0.15) 0%, rgba(15,
23, 42, 0.9) 100%); border: 1px solid var(--border-
highlight);">
  <h2 class="fw-bold mb-3">Ready to Take Control of
Your Health?</h2>
  <p class="text-secondary mx-auto mb-4"
style="max-width: 550px;">
    Join thousands of users tracking their strength,
cardio, weight, and wellness goals on FitPulse today.
  </p>
  <a href="{ { url_for('auth.register') } }" class="btn
btn-emerald btn-lg px-4">
    <i class="bi bi-person-plus-fill me-2"></i> Create
Free Account
  </a>
</section>
</div>
{% endblock %}
Login.html:
{% extends 'base.html' %}
{% block title %}Log In - FitPulse{% endblock %}
{% block content %}
<div class="container py-5">
  <div class="row justify-content-center">
    <div class="col-md-7 col-lg-5">
      <div class="card card-dark p-4 p-md-5 shadow-lg
border-highlight">
        <div class="text-center mb-4">
          <div class="stat-icon mx-auto mb-3"
style="width: 56px; height: 56px; font-size: 1.6rem;">
            <i class="bi bi-shield-lock-fill"></i>
          </div>
          <h3 class="fw-bold text-white mb-
1">Welcome Back</h3>
          <p class="text-secondary small">Enter your
credentials to access your fitness dashboard</p>
        </div>
        <form method="POST" action="{ {
url_for('auth.login') } }">
          <div class="mb-3">
            <label for="email" class="form-label form-
label-dark">Email Address</label>
            <div class="input-group">
              <span class="input-group-text bg-
transparent border-secondary text-secondary"><i
class="bi bi-envelope"></i></span>
              <input type="email" class="form-control
form-control-dark" id="email" name="email" value="{ {
email or ' ' } }" placeholder="name@example.com"
required autofocus>
            </div>
            <div class="mb-4">
              <label for="password" class="form-label
form-label-dark">Password</label>

```

```

        <div class="input-group">
          <span class="input-group-text bg-
transparent border-secondary text-secondary"><i
class="bi bi-key"></i></span>
          <input type="password" class="form-
control form-control-dark" id="password"
name="password" placeholder="Enter your password"
required>
          <button type="button" class="btn btn-
outline-custom toggle-password-btn" data-
target="password" type="button">
            <i class="bi bi-eye"></i>
          </button>
        </div>
      </div>
      <button type="submit" class="btn btn-
emerald w-100 py-2 mb-3">
        <i class="bi bi-box-arrow-in-right me-
2"></i> Log In
      </button>
    </form>
    <div class="text-center mt-3 pt-3 border-top"
style="border-color: var(--border-color) !important;">
      <p class="text-secondary small mb-0">
        Don't have an account yet?
        <a href="{ { url_for('auth.register') } }"
class="text-emerald fw-semibold text-decoration-
none">Sign Up</a>
      </p>
    </div>
  </div>
<!-- Demo Helper Accordion -->
<div class="card card-dark mt-4 p-3">
  <div class="d-flex align-items-center justify-
content-between">
    <span class="small text-secondary"><i
class="bi bi-info-circle text-cyan me-1"></i> Quick Test
Credentials:</span>
    <button class="btn btn-sm btn-outline-
custom" type="button" data-bs-toggle="collapse" data-bs-
target="#demoAccounts">
      View Demo Logins
    </button>
  </div>
  <div class="collapse mt-3"
id="demoAccounts">
    <div class="table-responsive">
      <table class="table table-sm table-dark-
custom mb-0 small">
        <thead>
          <tr><th>Role</th><th>Email</th><th>Password</th></tr>
        </thead>
        <tbody>
          <tr><td><span class="badge bg-
success">User</span></td><td>user@fitness.com</td><td>
User@123</td></tr>
          <tr><td><span class="badge bg-
primary">Trainer</span></td><td>trainer@fitness.com</td><td>
Trainer@123</td></tr>
          <tr><td><span class="badge bg-
warning text-
dark">Nutritionist</span></td><td>nutritionist@fitness.c
om</td><td>Nutritionist@123</td></tr>
          <tr><td><span class="badge bg-
danger">Admin</span></td><td>admin@fitness.com</td><td>
Admin@123</td></tr>
        </tbody>
      </table>
    </div>
  </div>
</div>

```



```

        <option value="Male" {% if gender
== 'Male' %}>selected{% endif %}>Male</option>
        <option value="Female" {% if gender
== 'Female' %}>selected{% endif %}>Female</option>
        <option value="Other" {% if gender
== 'Other' %}>selected{% endif %}>Other</option>
        <option value="Prefer not to say" {%
if gender == 'Prefer not to say' %}>selected{% endif
%}>Prefer not to say</option>
    </select>
</div>
<div class="col-md-6">
    <label for="age" class="form-label
form-label-dark">Age (years)</label>
    <input type="number" class="form-
control form-control-dark" id="age" name="age"
value="{{ age or '' }}" placeholder="e.g. 28" min="5"
max="120">
    </div>
</div>
<div class="row g-3 mb-4">
    <div class="col-md-6">
        <label for="height" class="form-label
form-label-dark">Height (cm)</label>
        <input type="number" step="0.1"
class="form-control form-control-dark" id="height"
name="height" value="{{ height or '' }}"
placeholder="e.g. 175" min="50" max="260">
        </div>
        <div class="col-md-6">
            <label for="weight" class="form-label
form-label-dark">Current Weight (kg)</label>
            <input type="number" step="0.1"
class="form-control form-control-dark" id="weight"
name="weight" value="{{ weight or '' }}"
placeholder="e.g. 72.5" min="20" max="400">
            </div>
        </div>
        <button type="submit" class="btn btn-
emerald w-100 py-2 mb-3">
            <i class="bi bi-check2-circle me-2"></i>
Register Account
        </button>
    </form>
    <div class="text-center mt-3 pt-3 border-top"
style="border-color: var(--border-color) !important;">
        <p class="text-secondary small mb-0">
            Already have an account?
            <a href="{{ url_for('auth.login') }}"
class="text-emerald fw-semibold text-decoration-
none">Log In</a>
        </p>
    </div>
</div>
</div>
</div>
</div>
{% endblock %}
Conftest.py:
import os
import sys
import pytest
sys.path.insert(0,
os.path.abspath(os.path.join(os.path.dirname(__file__),
'..')))
from app import create_app
from models import db, User, Workout, Goal,
WeightRecord
@pytest.fixture

```

```

def app():
    """Create and configure a clean testing application."""
    test_app = create_app('testing')
    with test_app.app_context():
        db.create_all()

    # Create standard test accounts
    user = User(
        name='Test Athlete',
        email='athlete@test.com',
        role='user',
        age=25,
        gender='Male',
        height=180.0,
        weight=75.0
    )
    user.set_password('Secret123')
    db.session.add(user)
    other_user = User(
        name='Other Athlete',
        email='other@test.com',
        role='user',
        age=30,
        gender='Female',
        height=165.0,
        weight=60.0
    )
    other_user.set_password('Secret123')
    db.session.add(other_user)
    trainer = User(
        name='Test Trainer',
        email='trainer@test.com',
        role='trainer',
        age=35,
        gender='Male',
        height=185.0,
        weight=85.0
    )
    trainer.set_password('Secret123')
    db.session.add(trainer)
    nutritionist = User(
        name='Test Nutritionist',
        email='nutritionist@test.com',
        role='nutritionist',
        age=32,
        gender='Female',
        height=170.0,
        weight=62.0
    )
    nutritionist.set_password('Secret123')
    db.session.add(nutritionist)
    admin = User(
        name='Test Admin',
        email='admin@test.com',
        role='admin',
        age=40,
        gender='Other',
        height=175.0,
        weight=70.0
    )
    admin.set_password('Secret123')
    db.session.add(admin)
    db.session.commit()
    yield test_app
    db.session.remove()
    db.drop_all()
@pytest.fixture
def client(app):
    """A test client for the app."""

```

```

    return app.test_client()
class AuthActions:
    def __init__(self, client):
        self._client = client
    def login(self, email='athlete@test.com',
password='Secret123'):
        return self._client.post(
            '/login',
            data={'email': email, 'password': password},
            follow_redirects=True
        )
    def logout(self):
        return self._client.get('/logout',
follow_redirects=True)
@pytest.fixture
def auth(client):
    return AuthActions(client)
Test_auth.py:
from models import User
def test_register_success(client, app):
    """Test successful user registration."""
    response = client.post('/register', data={
        'name': 'New Runner',
        'email': 'runner@test.com',
        'password': 'RunnerPass123',
        'confirm_password': 'RunnerPass123',
        'gender': 'Female',
        'age': '24',
        'height': '168.0',
        'weight': '58.5'
    }, follow_redirects=True)
    assert response.status_code == 200
    assert b'Account registered successfully' in
response.data
    with app.app_context():
        user =
User.query.filter_by(email='runner@test.com').first()
    assert user is not None
    assert user.name == 'New Runner'
    assert user.role == 'user'
    assert user.check_password('RunnerPass123') is True
    assert user.weight == 58.5
def test_register_trainer_and_nutritionist(client, app):
    """Test registering as a trainer and nutritionist."""
    # Register Trainer
    res_trainer = client.post('/register', data={
        'name': 'Coach Mike',
        'email': 'mike@trainer.com',
        'role': 'trainer',
        'password': 'CoachPassword123',
        'confirm_password': 'CoachPassword123'
    }, follow_redirects=True)
    assert res_trainer.status_code == 200
    assert b'Account registered successfully' in
res_trainer.data
    # Register Nutritionist
    res_nutri = client.post('/register', data={
        'name': 'Dr. Anna Nutri',
        'email': 'anna@nutrition.com',
        'role': 'nutritionist',
        'password': 'NutriPassword123',
        'confirm_password': 'NutriPassword123'
    }, follow_redirects=True)
    assert res_nutri.status_code == 200
    assert b'Account registered successfully' in
res_nutri.data
    with app.app_context():
        trainer =
User.query.filter_by(email='mike@trainer.com').first()

```

```

        nutritionist =
User.query.filter_by(email='anna@nutrition.com').first()
    assert trainer is not None and trainer.role == 'trainer'
    assert nutritionist is not None and nutritionist.role ==
'nutritionist'
def test_register_duplicate_email(client):
    """Test duplicate email rejection."""
    response = client.post('/register', data={
        'name': 'Duplicate User',
        'email': 'athlete@test.com', # existing in confest
        'password': 'Password123',
        'confirm_password': 'Password123'
    }, follow_redirects=True)
    assert b'An account with this email address already
exists' in response.data
def test_register_password_mismatch(client):
    """Test password and confirm password mismatch."""
    response = client.post('/register', data={
        'name': 'Mismatch User',
        'email': 'mismatch@test.com',
        'password': 'Password123',
        'confirm_password': 'DifferentPassword456'
    }, follow_redirects=True)
    assert b'Passwords do not match' in response.data
def test_login_success(auth):
    """Test valid login authentication."""
    response = auth.login('athlete@test.com', 'Secret123')
    assert response.status_code == 200
    assert b>Welcome back, Test Athlete' in response.data
def test_login_invalid_password(auth):
    """Test invalid password rejection."""
    response = auth.login('athlete@test.com',
'WrongPassword')
    assert response.status_code == 200
    assert b'Invalid email address or password' in
response.data
def test_logout(auth):
    """Test logout destroys session."""
    auth.login('athlete@test.com', 'Secret123')
    response = auth.logout()
    assert response.status_code == 200
    assert b'You have been logged out successfully' in
response.data
Test_goals.py:
from datetime import date, timedelta
from models import db, Goal, User
def test_add_goal(client, auth, app):
    """Test adding a new fitness goal."""
    auth.login('athlete@test.com', 'Secret123')
    target_date = (date.today() +
timedelta(days=30)).strftime('%Y-%m-%d')
    response = client.post('/goals/add', data={
        'goal_type': 'Lose Weight',
        'target_value': '70 kg',
        'current_value': '75 kg',
        'start_date': date.today().strftime('%Y-%m-%d'),
        'target_date': target_date,
        'description': 'Targeting 5 kg reduction.'
    }, follow_redirects=True)
    assert response.status_code == 200
    assert b'Lose Weight' in response.data
    with app.app_context():
        user =
User.query.filter_by(email='athlete@test.com').first()
        goal = Goal.query.filter_by(user_id=user.id,
goal_type='Lose Weight').first()
    assert goal is not None
    assert goal.target_value == '70 kg'
    assert goal.status == 'ACTIVE'

```

```

def test_toggle_goal_status(client, auth, app):
    """Test toggling goal status to COMPLETED."""
    auth.login('athlete@test.com', 'Secret123')
    with app.app_context():
        user =
        User.query.filter_by(email='athlete@test.com').first()
        g = Goal(
            user_id=user.id,
            goal_type='Run Distance',
            target_value='10 km',
            start_date=date.today(),
            target_date=date.today() + timedelta(days=15),
            status='ACTIVE'
        )
        db.session.add(g)
        db.session.commit()
        g_id = g.id
        response = client.post(f'/goals/{g_id}/status',
            follow_redirects=True)
        assert response.status_code == 200
        assert b'marked as Completed' in response.data
        with app.app_context():
            updated = db.session.get(Goal, g_id)
            assert updated.status == 'COMPLETED'
def test_delete_goal(client, auth, app):
    """Test deleting a goal."""
    auth.login('athlete@test.com', 'Secret123')
    with app.app_context():
        user =
        User.query.filter_by(email='athlete@test.com').first()
        g = Goal(
            user_id=user.id,
            goal_type='Workout Frequency',
            target_value='20 Workouts',
            start_date=date.today(),
            target_date=date.today() + timedelta(days=30)
        )
        db.session.add(g)
        db.session.commit()
        g_id = g.id
        response = client.post(f'/goals/{g_id}/delete',
            follow_redirects=True)
        assert response.status_code == 200
        assert b'Goal deleted successfully' in response.data
        with app.app_context():
            assert db.session.get(Goal, g_id) is None
Test_roles.py:
from models import db, User, Recommendation
def test_user_forbidden_access(client, auth):
    """Test standard user is blocked from accessing
    specialized portals."""
    auth.login('athlete@test.com', 'Secret123')
    res_admin = client.get('/admin/dashboard',
        follow_redirects=True)
    assert b'You do not have permission to access this
    resource' in res_admin.data
    res_trainer = client.get('/trainer/dashboard',
        follow_redirects=True)
    assert b'You do not have permission to access this
    resource' in res_trainer.data
    res_nutr = client.get('/nutritionist/dashboard',
        follow_redirects=True)
    assert b'You do not have permission to access this
    resource' in res_nutr.data
def test_trainer_portal_and_recommendation(client, auth,
    app):
    """Test trainer can view dashboard and prescribe
    workout guidance."""
    auth.login('trainer@test.com', 'Secret123')

```

```

        res = client.get('/trainer/dashboard')
        assert res.status_code == 200
        assert b'Trainer Portal' in res.data
        with app.app_context():
            athlete =
            User.query.filter_by(email='athlete@test.com').first()
            athlete_id = athlete.id
            # Prescribe workout recommendation
            post_res =
            client.post(f'/trainer/users/{athlete_id}/recommend',
                data={
                    'title': 'Form Check: Squats',
                    'content': 'Keep your chest elevated and push through
                    mid-foot.'
                }, follow_redirects=True)
            assert post_res.status_code == 200
            assert b'Workout recommendation provided' in
            post_res.data
            with app.app_context():
                rec =
                Recommendation.query.filter_by(user_id=athlete_id,
                    note_type='WORKOUT').first()
                assert rec is not None
                assert rec.title == 'Form Check: Squats'
def test_nutritionist_portal_and_recommendation(client,
    auth, app):
    """Test nutritionist can view dashboard and prescribe
    dietary guidance."""
    auth.login('nutritionist@test.com', 'Secret123')
    res = client.get('/nutritionist/dashboard')
    assert res.status_code == 200
    assert b'Nutritionist Portal' in res.data
    with app.app_context():
        athlete =
        User.query.filter_by(email='athlete@test.com').first()
        athlete_id = athlete.id
        # Prescribe dietary recommendation
        post_res =
        client.post(f'/nutritionist/users/{athlete_id}/recommend',
            data={
                'title': 'Electrolyte Strategy',
                'content': 'Add 500mg sodium in 750ml water prior to
                tempo runs.'
            }, follow_redirects=True)
        assert post_res.status_code == 200
        assert b'Nutrition guidance sent' in post_res.data
        with app.app_context():
            rec =
            Recommendation.query.filter_by(user_id=athlete_id,
                note_type='NUTRITION').first()
            assert rec is not None
            assert rec.title == 'Electrolyte Strategy'
def test_admin_full_management(client, auth, app):
    """Test admin can view dashboard, create, edit, and
    delete users."""
    auth.login('admin@test.com', 'Secret123')
    res = client.get('/admin/dashboard')
    assert res.status_code == 200
    assert b'System Administration' in res.data
    # Admin create new user
    res_create = client.post('/admin/users/create', data={
        'name': 'Admin Created User',
        'email': 'admincreated@test.com',
        'password': 'Password123',
        'role': 'trainer'
    }, follow_redirects=True)
    assert res_create.status_code == 200
    assert b'Admin Created User' in res_create.data
    with app.app_context():

```

```

u =
User.query.filter_by(email='admincreated@test.com').first()
()
    assert u is not None
    assert u.role == 'trainer'
    u_id = u.id
    # Admin delete created user
    res_delete = client.post(f'/admin/users/{u_id}/delete',
follow_redirects=True)
    assert res_delete.status_code == 200
    assert b'deleted successfully' in res_delete.data
    with app.app_context():
        assert db.session.get(User, u_id) is None
Test_weight_bmi.py:
from datetime import date
from models import User
from routes.bmi import calculate_bmi_details
def test_add_weight_and_user_sync(client, auth, app):
    """Test logging weight and ensuring user current
weight syncs."""
    auth.login('athlete@test.com', 'Secret123')
    response = client.post('/weight/add', data={
        'weight': '73.2',
        'date': date.today().strftime('%Y-%m-%d'),
        'notes': 'Post morning run'
    }, follow_redirects=True)
    assert response.status_code == 200
    assert b'73.2 kg logged successfully' in response.data
    with app.app_context():
        user =
User.query.filter_by(email='athlete@test.com').first()
        assert user.weight == 73.2
        # BMI for 73.2 kg & 180 cm => 73.2 / (1.8^2) =
22.59 => 22.6
        assert user.bmi == 22.6
        assert user.bmi_category == 'Normal weight'
def test_bmi_math_and_categories():
    """Test WHO standard BMI categories and
mathematical precision."""
    # Underweight (< 18.5)
    underweight = calculate_bmi_details(50.0, 180.0)
    assert underweight['bmi'] == 15.4
    assert underweight['category'] == 'Underweight'
    # Normal weight (18.5 - 24.9)
    normal = calculate_bmi_details(70.0, 175.0)
    assert normal['bmi'] == 22.9
    assert normal['category'] == 'Normal weight'
    # Overweight (25.0 - 29.9)
    overweight = calculate_bmi_details(85.0, 175.0)
    assert overweight['bmi'] == 27.8
    assert overweight['category'] == 'Overweight'
    # Obesity (>= 30.0)
    obese = calculate_bmi_details(105.0, 175.0)
    assert obese['bmi'] == 34.3
    assert obese['category'] == 'Obesity'
def test_bmi_api_endpoint(client):
    """Test the JSON API calculation endpoint."""
    response = client.post('/api/bmi/calculate', json={
        'height': 170.0,
        'weight': 68.0
    })
    assert response.status_code == 200
    data = response.get_json()
    assert data['bmi'] == 23.5
    assert data['category'] == 'Normal weight'
    assert 'min_healthy_weight' in data

```

Test_workout.py:

```

from datetime import date
from models import db, Workout, User

```

```

def test_add_workout(client, auth, app):
    """Test adding a new workout."""
    auth.login('athlete@test.com', 'Secret123')
    response = client.post('/workouts/add', data={
        'workout_type': 'Running',
        'exercise_name': 'Evening 5K',
        'duration': '30',
        'sets': '1',
        'repetitions': '5',
        'calories_burned': '320.0',
        'workout_date': '2026-03-15',
        'notes': 'Great cadence and breathing.'
    }, follow_redirects=True)
    assert response.status_code == 200
    assert b'Evening 5K' in response.data
    with app.app_context():
        user =
User.query.filter_by(email='athlete@test.com').first()
        workout = Workout.query.filter_by(user_id=user.id,
exercise_name='Evening 5K').first()
        assert workout is not None
        assert workout.duration == 30
        assert workout.calories_burned == 320.0
        assert workout.workout_type == 'Running'
def test_edit_workout(client, auth, app):
    """Test editing an existing workout."""
    auth.login('athlete@test.com', 'Secret123')
    # Add initial workout
    with app.app_context():
        user =
User.query.filter_by(email='athlete@test.com').first()
        w = Workout(
            user_id=user.id,
            workout_type='Strength Training',
            exercise_name='Bench Press',
            duration=45,
            sets=4,
            repetitions=10,
            calories_burned=280.0,
            workout_date=date(2026, 3, 10)
        )
        db.session.add(w)
        db.session.commit()
        w_id = w.id
    # Edit workout
    response = client.post(f'/workouts/{w_id}/edit', data={
        'workout_type': 'Strength Training',
        'exercise_name': 'Heavy Bench Press',
        'duration': '50',
        'sets': '5',
        'repetitions': '5',
        'calories_burned': '310.0',
        'workout_date': '2026-03-10',
        'notes': 'New personal best!'
    }, follow_redirects=True)
    assert response.status_code == 200
    assert b'Heavy Bench Press' in response.data
    with app.app_context():
        updated = db.session.get(Workout, w_id)
        assert updated.exercise_name == 'Heavy Bench
Press'
        assert updated.duration == 50
def test_delete_workout(client, auth, app):
    """Test deleting a workout."""
    auth.login('athlete@test.com', 'Secret123')
    with app.app_context():
        user =
User.query.filter_by(email='athlete@test.com').first()
        w = Workout(

```

```

        user_id=user.id,
        workout_type='Yoga',
        exercise_name='Morning Flow',
        duration=30,
        calories_burned=120.0,
        workout_date=date.today()
    )
    db.session.add(w)
    db.session.commit()
    w_id = w.id
    response = client.post(f'/workouts/{w_id}/delete',
follow_redirects=True)
    assert response.status_code == 200
    assert b'Workout record deleted successfully' in
response.data
    with app.app_context():
        deleted = db.session.get(Workout, w_id)
        assert deleted is None
def test_workout_isolation(client, auth, app):
    """Test user cannot delete or edit other user's
workout."""
    with app.app_context():
        other_user =
User.query.filter_by(email='other@test.com').first()
        w = Workout(
            user_id=other_user.id,
            workout_type='Cycling',
            exercise_name='Private Ride',
            duration=60,
            calories_burned=400.0,
            workout_date=date.today()
        )
        db.session.add(w)
        db.session.commit()
        other_workout_id = w.id
    # Log in as different user
    auth.login('athlete@test.com', 'Secret123')
    # Attempt to delete other user's workout
    response =
client.post(f'/workouts/{other_workout_id}/delete',
follow_redirects=True)
    assert b'You are not authorized to delete this workout'
in response.data
    with app.app_context():
        # Ensure it still exists
        assert db.session.get(Workout, other_workout_id) is
not None
Add.py:
import os
from datetime import datetime
from flask import Flask, render_template, session
from config import config_by_name
from models import db, User
def create_app(config_name='development'):
    """Application factory for the Fitness & Workout
Tracker."""
    app = Flask(__name__)
    config_class = config_by_name.get(config_name,
config_by_name['default'])
    app.config.from_object(config_class)
    try:
        os.makedirs(app.instance_path, exist_ok=True)
    except OSError:
        pass
    # Initialize extensions
    db.init_app(app)
    # Register blueprints
    from routes.auth import auth_bp
    from routes.dashboard import dashboard_bp

```

```

    from routes.profile import profile_bp
    from routes.workout import workout_bp
    from routes.goal import goal_bp
    from routes.weight import weight_bp
    from routes.bmi import bmi_bp
    from routes.progress import progress_bp
    from routes.trainer import trainer_bp
    from routes.nutritionist import nutritionist_bp
    from routes.admin import admin_bp
    app.register_blueprint(auth_bp)
    app.register_blueprint(dashboard_bp)
    app.register_blueprint(profile_bp)
    app.register_blueprint(workout_bp)
    app.register_blueprint(goal_bp)
    app.register_blueprint(weight_bp)
    app.register_blueprint(bmi_bp)
    app.register_blueprint(progress_bp)
    app.register_blueprint(trainer_bp)
    app.register_blueprint(nutritionist_bp)
    app.register_blueprint(admin_bp)
    # Context processors
    @app.context_processor
    def inject_globals():
        current_user = None
        user_id = session.get('user_id')
        if user_id:
            current_user = db.session.get(User, user_id)
        return {
            'current_user': current_user,
            'current_year': datetime.now().year
        }
    # Error handlers
    @app.errorhandler(403)
    def forbidden_error(error):
        return render_template('errors/403.html'), 403
    @app.errorhandler(404)
    def not_found_error(error):
        return render_template('errors/404.html'), 404
    @app.errorhandler(500)
    def internal_error(error):
        db.session.rollback()
        return render_template('errors/500.html'), 500
    # CLI commands
    @app.cli.command('init-db')
    def init_db_command():
        """Initialize the database tables."""
        db.create_all()
        print('Initialized the database tables.')
    return app
app = create_app(os.environ.get('FLASK_ENV',
'development'))
if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(host='127.0.0.1', port=5000, debug=True)
Config.py:
import os
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    """Base application configuration."""
    SECRET_KEY = os.environ.get('SECRET_KEY',
'fitness-tracker-dev-secret-key-2026-secure')
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = 'Lax'
class DevelopmentConfig(Config):
    """Development configuration."""
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = os.environ.get(

```

```

        'DEV_DATABASE_URL',
        f'sqlite:/// {os.path.join(BASE_DIR, 'instance',
        'fitness.db')}"
    )
class TestingConfig(Config):
    """Testing configuration."""
    TESTING = True
    DEBUG = False
    SQLALCHEMY_DATABASE_URI =
    'sqlite:///memory:'
    WTF_CSRF_ENABLED = False
    SECRET_KEY = 'test-secret-key'
class ProductionConfig(Config):
    """Production configuration."""
    DEBUG = False
    SQLALCHEMY_DATABASE_URI = os.environ.get(
        'DATABASE_URL',
        f'sqlite:/// {os.path.join(BASE_DIR, 'instance',
        'fitness.db')}"
    )
config_by_name = {
    'development': DevelopmentConfig,
    'testing': TestingConfig,
    'production': ProductionConfig,
    'default': DevelopmentConfig
}
Speed.py:
from datetime import date, timedelta
import random
from app import create_app
from models import db, User, Workout, Goal,
WeightRecord, Recommendation
def seed_database():
    """Populate database with sample development and test
    data."""
    app = create_app('development')

    with app.app_context():
        # Drop and recreate all tables for a clean seed
        db.drop_all()
        db.create_all()
        print("Dropped and recreated database tables.")
        # 1. Create Core Users
        users_data = [
            {
                'name': 'Alex Johnson',
                'email': 'user@fitness.com',
                'password': 'User@123',
                'role': 'user',
                'age': 28,
                'gender': 'Male',
                'height': 178.0,
                'weight': 74.5
            },
            {
                'name': 'Sarah Miller',
                'email': 'sarah@fitness.com',
                'password': 'Sarah@123',
                'role': 'user',
                'age': 26,
                'gender': 'Female',
                'height': 165.0,
                'weight': 62.0
            },
            {
                'name': 'David Kim',
                'email': 'david@fitness.com',
                'password': 'David@123',
                'role': 'user',

```

```

                'age': 31,
                'gender': 'Male',
                'height': 180.0,
                'weight': 82.0
            },
            {
                'name': 'Marcus Vance (CPT)',
                'email': 'trainer@fitness.com',
                'password': 'Trainer@123',
                'role': 'trainer',
                'age': 34,
                'gender': 'Male',
                'height': 185.0,
                'weight': 86.0
            },
            {
                'name': 'Dr. Elena Rostova (RD)',
                'email': 'nutritionist@fitness.com',
                'password': 'Nutritionist@123',
                'role': 'nutritionist',
                'age': 32,
                'gender': 'Female',
                'height': 168.0,
                'weight': 59.0
            },
            {
                'name': 'System Administrator',
                'email': 'admin@fitness.com',
                'password': 'Admin@123',
                'role': 'admin',
                'age': 40,
                'gender': 'Other',
                'height': 175.0,
                'weight': 75.0
            }
        ]
    created_users = {}
    for u in users_data:
        user = User(
            name=u['name'],
            email=u['email'],
            role=u['role'],
            age=u['age'],
            gender=u['gender'],
            height=u['height'],
            weight=u['weight']
        )
        user.set_password(u['password'])
        db.session.add(user)
        db.session.flush()
        created_users[u['email']] = user
        print(f"Created user: {u['name']} ({u['email']}) as
        [{u['role']}]")
        alex = created_users['user@fitness.com']
        sarah = created_users['sarah@fitness.com']
        trainer = created_users['trainer@fitness.com']
        nutritionist =
        created_users['nutritionist@fitness.com']
        # 2. Seed Weight Logs for Alex (Showing gradual
        progression)
        today = date.today()
        base_weight = 77.8
        for i in range(30, -1, -3):
            log_date = today - timedelta(days=i)
            # Gradual weight loss curve with minor natural
            fluctuations
            wt = round(base_weight - ((30 - i) * 0.11) +
            random.uniform(-0.2, 0.2), 1)
            weight_entry = WeightRecord(

```

```

        user_id=alex.id,
        weight=wt,
        date=log_date,
        notes='Morning fasted weight measurement' if i
% 6 == 0 else None
    )
    db.session.add(weight_entry)
# 3. Seed Workouts for Alex (Across past 30 days)
sample_workouts = [
    {'type': 'Running', 'name': 'Morning 5K Pace Run',
'duration': 28, 'sets': 1, 'reps': 5, 'cals': 310, 'notes':
'Targeted 5:36 min/km pace, felt strong.'},
    {'type': 'Strength Training', 'name': 'Upper Body
Push (Chest & Shoulders)', 'duration': 55, 'sets': 16, 'reps':
12, 'cals': 420, 'notes': 'Bench Press 75kg 4x8, Overhead
Press 45kg.'},
    {'type': 'Cycling', 'name': 'Outdoor Tempo Ride',
'duration': 45, 'sets': 1, 'reps': 18, 'cals': 480, 'notes': 'Good
elevation, high cadence training.'},
    {'type': 'Yoga', 'name': 'Vinyasa Flow & Hip
Mobility', 'duration': 35, 'sets': 1, 'reps': 1, 'cals': 160,
'notes': 'Focus on recovery and hamstrings.'},
    {'type': 'HIIT', 'name': 'Full Body Tabata Intervals',
'duration': 30, 'sets': 8, 'reps': 20, 'cals': 380, 'notes':
'Burpees, kettlebell swings, box jumps.'},
    {'type': 'Strength Training', 'name': 'Leg Day
(Squats & Deadlifts)', 'duration': 60, 'sets': 18, 'reps': 10,
'cals': 510, 'notes': 'Squats 95kg 5x5, Romanian Deadlifts
80kg.'},
    {'type': 'Swimming', 'name': 'Freestyle Lap
Intervals', 'duration': 40, 'sets': 12, 'reps': 50, 'cals': 390,
'notes': '1000m total distance.'},
    {'type': 'Walking', 'name': 'Evening Recovery
Walk', 'duration': 45, 'sets': 1, 'reps': 4, 'cals': 180, 'notes':
'Active recovery session.'}
]
for day_offset in [28, 26, 24, 21, 19, 17, 14, 12, 10,
8, 6, 4, 2, 0]:
    w_template = random.choice(sample_workouts)
    w_date = today - timedelta(days=day_offset)
    workout = Workout(
        user_id=alex.id,
        workout_type=w_template['type'],
        exercise_name=w_template['name'],
        duration=w_template['duration'],
        sets=w_template['sets'],
        repetitions=w_template['reps'],
        calories_burned=w_template['cals'],
        workout_date=w_date,
        notes=w_template['notes']
    )
    db.session.add(workout)
# 4. Seed Goals for Alex
goals_data = [
    {
        'goal_type': 'Lose Weight',
        'target_value': '72.0 kg',
        'current_value': '74.5 kg',
        'start_date': today - timedelta(days=20),
        'target_date': today + timedelta(days=30),
        'status': 'ACTIVE',
        'description': 'Target healthy sustainable fat loss
with consistent caloric deficit.'
    },
    {
        'goal_type': 'Run Distance',
        'target_value': '10.0 km',
        'current_value': '6.5 km',
        'start_date': today - timedelta(days=15),

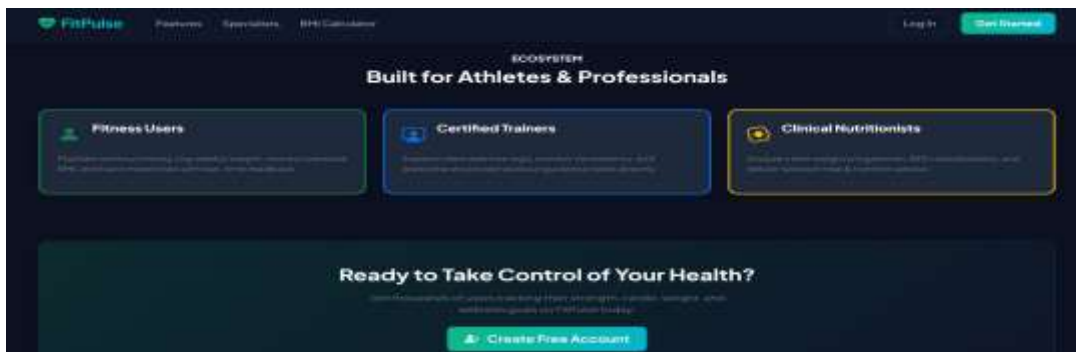
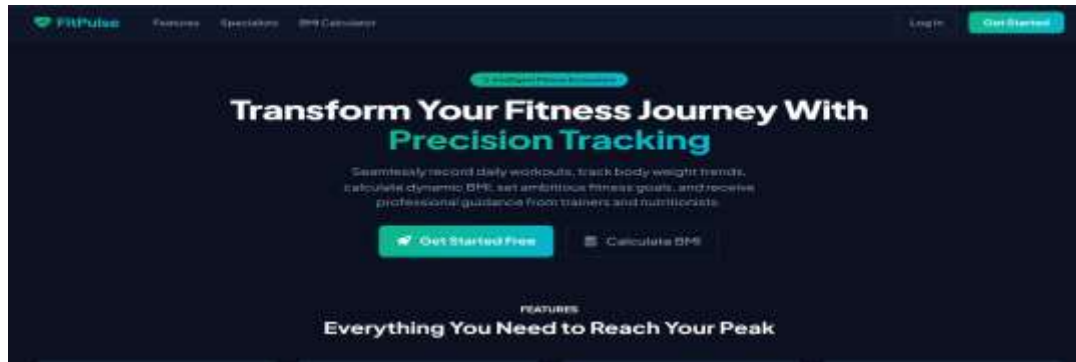
```

```

        'target_date': today + timedelta(days=45),
        'status': 'ACTIVE',
        'description': 'Train for upcoming community
10K road race.'
    },
    {
        'goal_type': 'Workout Frequency',
        'target_value': '16 Workouts / Month',
        'current_value': '16 Workouts',
        'start_date': today - timedelta(days=30),
        'target_date': today,
        'status': 'COMPLETED',
        'description': 'Maintain 4 workouts per week
consistency.'
    },
    {
        'goal_type': 'Improve Strength',
        'target_value': '100 kg Bench Press',
        'current_value': '80 kg',
        'start_date': today - timedelta(days=10),
        'target_date': today + timedelta(days=60),
        'status': 'ACTIVE',
        'description': 'Progressive overload on barbell
flat bench press.'
    }
]
for g in goals_data:
    goal = Goal(
        user_id=alex.id,
        goal_type=g['goal_type'],
        target_value=g['target_value'],
        current_value=g['current_value'],
        start_date=g['start_date'],
        target_date=g['target_date'],
        status=g['status'],
        description=g['description']
    )
    db.session.add(goal)
# 5. Seed Trainer Guidance Notes
rec_trainer = Recommendation(
    user_id=alex.id,
    author_id=trainer.id,
    note_type='WORKOUT',
    title='Strength Progression & Rest Intervals',
    content='Great cadence on your upper body push
workouts! Ensure you take a full 90-120 seconds of rest
between your 5x5 compound sets to maximize central
nervous system recovery.'
)
db.session.add(rec_trainer)
# 6. Seed Nutritionist Dietary Notes
rec_nutrition = Recommendation(
    user_id=alex.id,
    author_id=nutritionist.id,
    note_type='NUTRITION',
    title='Post-Workout Protein & Hydration Timing',
    content='Your weight trajectory is trending
steadily downwards at a safe rate of 0.4kg/week. Aim for
25-30g of fast-absorbing protein within 45 minutes after
strength sessions and drink at least 3 liters of water daily.'
)
db.session.add(rec_nutrition)
db.session.commit()
print("Successfully seeded all realistic test data!")
if __name__ == '__main__':
    seed_database()

```

DEMONSTRATION:



FitPulse Features Specialists BMI Calculator Login [Get Started](#)

Body Mass Index (BMI) Calculator

Calculate your BMI using height and weight to assess your health status.

Height (cm): cm

Weight (kg): kg

YOUR CALCULATED BMI

20.3

Normal weight

Weight range: 47.4 kg – 83.7 kg

WHO BMI Classification

Classification	BMI Range	Health Status
Underweight	< 18.5	Malnutrition, weak bones
Normal weight	18.5 – 24.9	Lowest risk
Overweight	25.0 – 29.9	Moderate risk
Obesity	≥ 30.0	High risk

FitPulse Features Specialists BMI Calculator Login [Get Started](#)

Welcome Back

Enter your email and password to access your account.

Email Address:

Password:

[Forgot Password?](#)

[Log In](#)

[Don't have an account? Register](#)

FitPulse [Nutritionist Dashboard](#) [Client Profiles & BMI](#) [New Client](#) [Logout](#)

Nutritionist Portal

Manage client BMI, track progress, weight trends, and provide dietary guidance.

[Browse Client Health Profiles](#)

Total Clients: **3**

Assigned Clients: [View All](#)

Normal Weight: **2**

[View All](#)

Overweight: **1**

[View All](#)

Obesity / Underweight: **0**

[View All](#)

Client Body Metrics

Client	Height	Weight	BMI	Category	Action
Alex Johnson	180 cm	82 kg	23.8	Normal weight	View Profile
David Kim	175 cm	85 kg	27.8	Overweight	View Profile
Sarah Miller	160 cm	67 kg	26.6	Overweight	View Profile

Recent Dietary Notes

No dietary notes found for this client.

FitPulse [Nutritionist Dashboard](#) [Client Profiles & BMI](#) [New Client](#) [Logout](#)

Alex Johnson's Nutritional & Weight Profile

Client ID: 12345 | Age: 35 | Height: 180 cm | Weight: 82 kg **(BMI: 23.8)**

Current Weight: **74.5**

Target Weight: **77.9**

Total Weight Target: **+3.4 kg**

Client Weight History & Computed BMI

Date	Weight (kg)	Computed BMI	Notes
2024-01-15	74.4 kg	23.8	Monitoring weight trend.
2024-02-15	74.7 kg	23.9	Monitoring weight trend.
2024-03-15	75.0 kg	24.0	Monitoring weight trend.
2024-04-15	75.4 kg	24.2	Monitoring weight trend.
2024-05-15	75.8 kg	24.4	Monitoring weight trend.

Prescribe Nutrition Guidance

Guidance Title:

Nutritional Advice & Food Guidance:

[Send Dietary Advice](#)

FitPulse | Nutritional Dashboard | **Client Profiles & SPM** | **Menu** | **ADMIN**

Fitness Profile

Karl
 30 years old | Member since Nov 2006 | **95.30 kg (current weight)**

Update Personal Information

Full Name	Karl	Email Address (Preferred)	karl@fitpulse.com
Age	30	Phone Number	90
Gender	Male	Current Weight (kg)	95.30
Height (cm)	180.0		

[Cancel](#) [Save Profile Changes](#)

FitPulse | **Dashboard** | **Goals** | **Weight** | **SPM** | **Progress** | **Join** | **ADMIN**

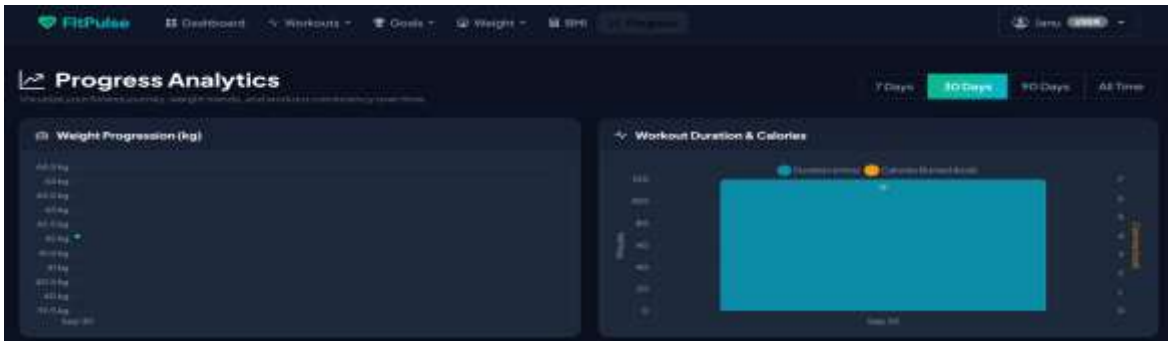
Workout History

Stores and manages your workout exercise history

Total Workouts 1 | **Total Duration** 120 | **Total Calories** 7

All Workout Types [Filter](#) [Reset](#)

ID	DATE	TYPE	EXERCISE	DURATION	SETS	REPS	CALORIES	NOTES	ACTIONS
1	2023-10-30	Exercise	Morning cycling	120 mins	2	5	7 kcal	Good	Edit Delete



FitPulse | **Nutritional Dashboard** | **Users** | **Workouts** | **Goals** | **Weights** | **System Administrator** | **ADMIN**

System Administration

Perform important user management and system administration tasks

[Create User](#) | [Manage Users](#)

Active Users 8
Active users in system & all related devices

Logged-in Users 15
Logged platform-wide

Active & Completed Goals 4

Weight Records 13
Body metrics logged

User Management (4 users)

Workout Records (1 workout)

Fitness Goals (1 goal)

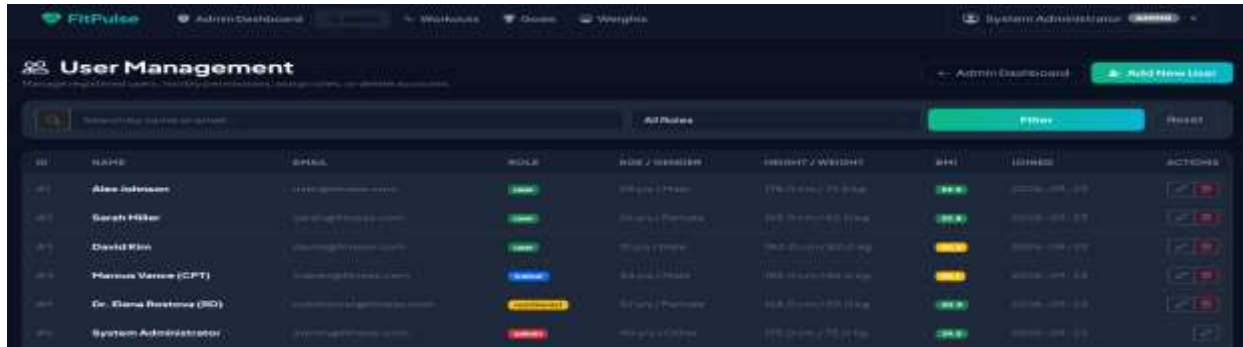
Weight Logs (13 records)

Recent Registrations

USER	EMAIL	ROLE	REGISTERED
John	john@fitpulse.com	ADMIN	2023-09-20 10:00
Karl	karl@fitpulse.com	ADMIN	2023-09-20 10:00
System Administrator	admin@fitpulse.com	ADMIN	2023-09-20 10:00

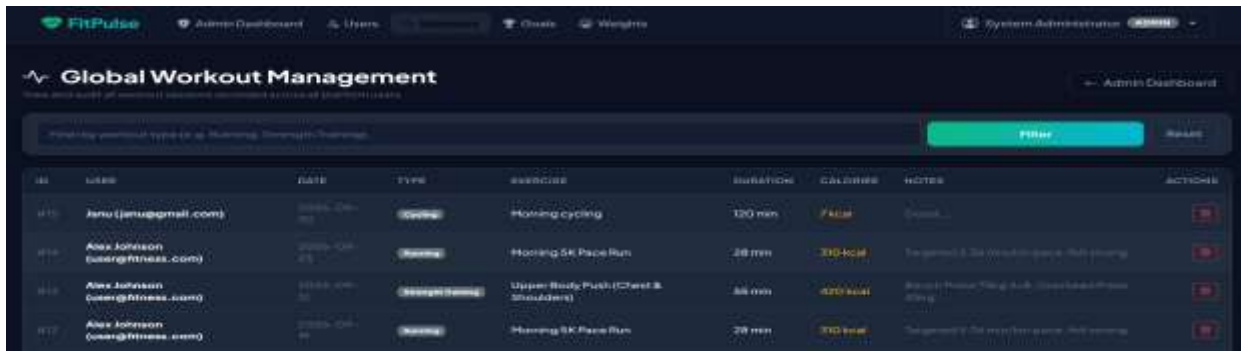
Recent Workouts (Global)

USER	EXERCISE	TYPE	DURATION	SETS
John	Morning cycling	Exercise	120 mins	2
Anna Johnson	Morning 5K Pace Run	Exercise	30 mins	1



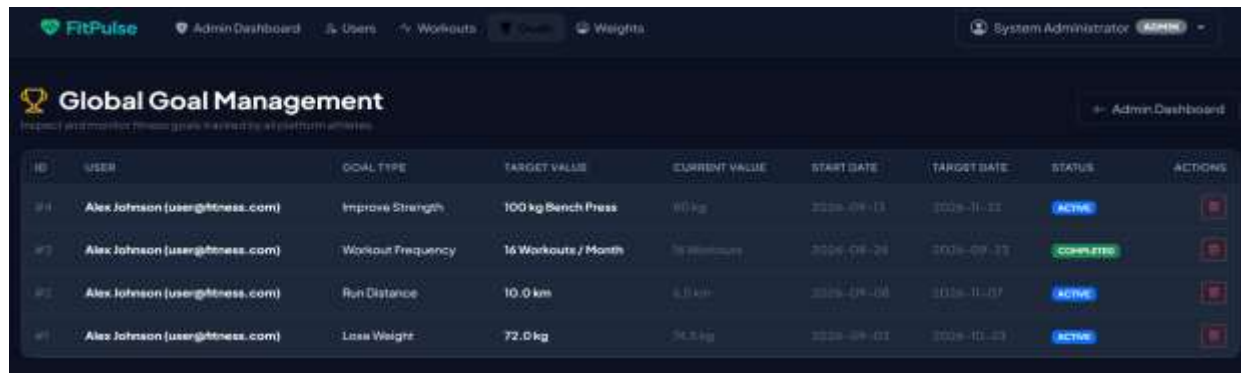
The screenshot shows the 'User Management' section of the FitPulse Admin Dashboard. It features a navigation bar with 'FitPulse', 'Admin Dashboard', 'Users', 'Workouts', 'Goals', and 'Weights'. The main header includes a search bar, 'All Users', 'Filter', and 'Reset' buttons. Below is a table listing users with columns for ID, Name, Email, Role, Role / Permission, Height / Weight, BMI, Gender, and Actions. The table contains six rows of user data, including a System Administrator.

ID	NAME	EMAIL	ROLE	ROLE / PERMISSION	HEIGHT / WEIGHT	BMI	GENDER	ACTIONS
#1	Alex Johnson	alexjohnson@fitness.com	Admin	Admin / Editor	175 (5'9") / 75.5 kg	24.9	2020-09-15	Edit Delete
#2	Sarah Miller	sarahmiller@fitness.com	Admin	Admin / Editor	165 (5'5") / 65.0 kg	23.9	2020-09-17	Edit Delete
#3	David Kim	davidkim@fitness.com	Admin	Admin / Editor	180 (5'11") / 80.0 kg	25.0	2020-09-19	Edit Delete
#4	Thomas Vance (CPT)	thomasmvance@fitness.com	Trainer	Trainer / Coach	185 (6'1") / 85.0 kg	26.0	2020-09-20	Edit Delete
#5	Dr. Diana Restone (MD)	dianarestone@fitness.com	Physician	Trainer / Personal	160 (5'3") / 60.0 kg	26.0	2020-09-22	Edit Delete
#6	System Administrator	systemadmin@fitness.com	Admin	Admin / Editor	175 (5'9") / 75.5 kg	24.9	2020-09-23	Edit



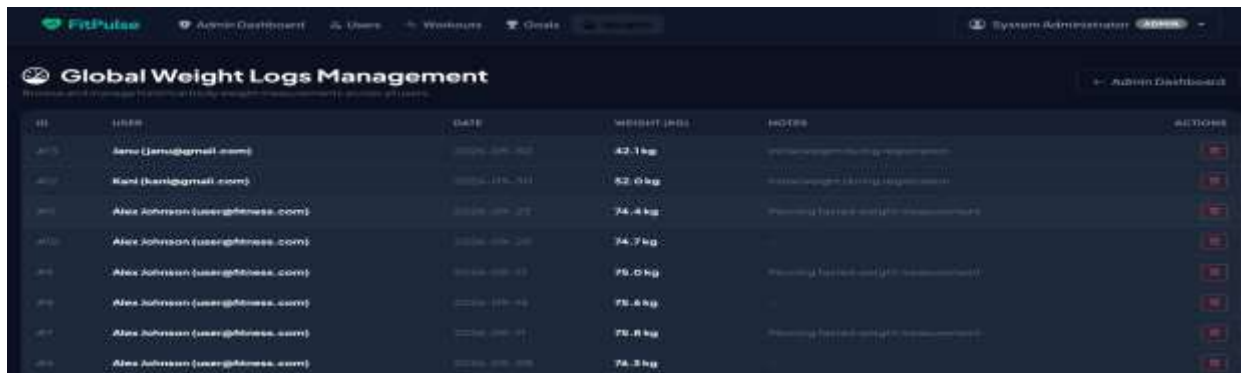
The screenshot shows the 'Global Workout Management' section of the FitPulse Admin Dashboard. It features a navigation bar with 'FitPulse', 'Admin Dashboard', 'Users', 'Workouts', 'Goals', and 'Weights'. The main header includes a search bar, 'Filter', and 'Reset' buttons. Below is a table listing workouts with columns for ID, User, Date, Type, Exercise, Duration, Calories, Notes, and Actions. The table contains four rows of workout data.

ID	USER	DATE	TYPE	EXERCISE	DURATION	CALORIES	NOTES	ACTIONS
#10	Jane (jane@gmail.com)	2020-09-25	Cardio	Morning cycling	120 min	740 kcal	Good...	Edit
#11	Alex Johnson (user@fitness.com)	2020-09-25	Running	Morning 5K Pace Run	28 min	310 kcal	The general 5.24 minutes pace. Not running.	Edit
#12	Alex Johnson (user@fitness.com)	2020-09-26	Strength Training	Upper Body Push (Chest & Shoulders)	35 min	420 kcal	Pushed Power 75kg x 6. Goodhead Press 25kg	Edit
#13	Alex Johnson (user@fitness.com)	2020-09-26	Running	Morning 5K Pace Run	28 min	310 kcal	The general 5.24 minutes pace. Not running.	Edit



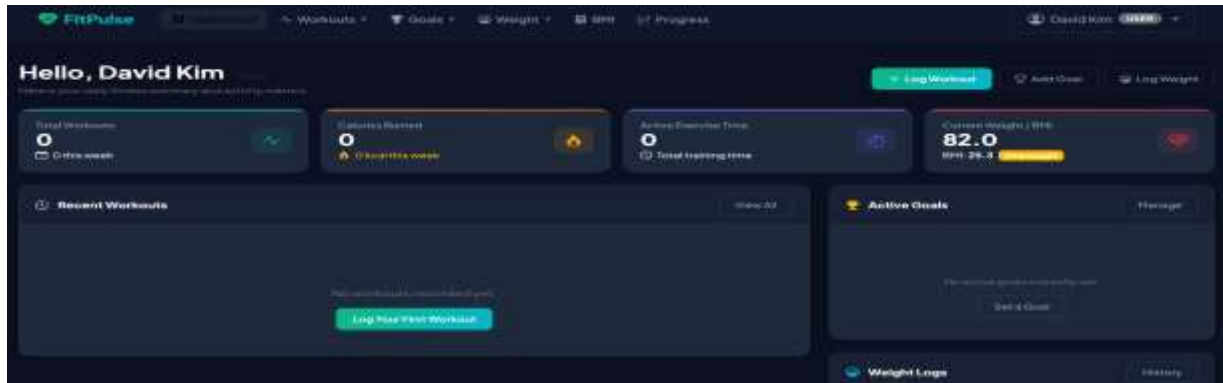
The screenshot shows the 'Global Goal Management' section of the FitPulse Admin Dashboard. It features a navigation bar with 'FitPulse', 'Admin Dashboard', 'Users', 'Workouts', 'Goals', and 'Weights'. The main header includes a search bar, 'Filter', and 'Reset' buttons. Below is a table listing goals with columns for ID, User, Goal Type, Target Value, Current Value, Start Date, Target Date, Status, and Actions. The table contains four rows of goal data.

ID	USER	GOAL TYPE	TARGET VALUE	CURRENT VALUE	START DATE	TARGET DATE	STATUS	ACTIONS
#1	Alex Johnson (user@fitness.com)	Improve Strength	100 kg Bench Press	60 kg	2020-09-13	2020-11-22	Active	Edit
#2	Alex Johnson (user@fitness.com)	Workout Frequency	16 Workouts / Month	16 Workouts	2020-09-24	2020-09-30	Completed	Edit
#3	Alex Johnson (user@fitness.com)	Run Distance	10.0 km	6.0 km	2020-09-06	2020-11-27	Active	Edit
#4	Alex Johnson (user@fitness.com)	Loss Weight	72.0 kg	74.3 kg	2020-09-01	2020-10-29	Active	Edit



The screenshot shows the 'Global Weight Logs Management' section of the FitPulse Admin Dashboard. It features a navigation bar with 'FitPulse', 'Admin Dashboard', 'Users', 'Workouts', 'Goals', and 'Weights'. The main header includes a search bar, 'Filter', and 'Reset' buttons. Below is a table listing weight logs with columns for ID, User, Date, Weight (kg), Notes, and Actions. The table contains eight rows of weight log data.

ID	USER	DATE	WEIGHT (kg)	NOTES	ACTIONS
#15	Jane (jane@gmail.com)	2020-09-30	62.1 kg	Weight gain during pregnancy	Edit
#16	Karl (karl@gmail.com)	2020-10-11	62.0 kg	Weight gain during pregnancy	Edit
#17	Alex Johnson (user@fitness.com)	2020-09-25	74.4 kg	Morning fasted weight measurement	Edit
#18	Alex Johnson (user@fitness.com)	2020-09-26	74.7 kg	-	Edit
#19	Alex Johnson (user@fitness.com)	2020-09-27	75.0 kg	Morning fasted weight measurement	Edit
#20	Alex Johnson (user@fitness.com)	2020-09-28	75.6 kg	-	Edit
#21	Alex Johnson (user@fitness.com)	2020-09-29	75.8 kg	Morning fasted weight measurement	Edit
#22	Alex Johnson (user@fitness.com)	2020-09-30	74.3 kg	-	Edit



E-RESOURCE:

<https://github.com/prateekshaa29/FitPulse>